SAMSUNG

Samsung Innovation Campus

Vạn vật kết nối – IoT

Together for Tomorrow!
Enabling People
Education for Future Generations

Chương 3.

Quản lý phiên bản với Git & GitHub

Vạn vật kết nối – IoT

Mô tả chương học

Mục tiêu của chương học

- Hiểu về tính năng Quản lý phiên bản (Version Control) bằng Git và GitHub.
- Hiểu về giao diện người dùng của GitHub và các chức năng liên quan.
- Hiểu cách phân bố phần mềm với GitHub.
- Thông qua một mini-project để hiểu về cách phối hợp làm việc trên Git và GitHub.

Nội dung chương học

- Bài 1. Quản lý phiên bản là gì?
- Bài 2. Git : Tổng quan & Cài đặt
- Bài 3. Tổng quan về GitHub và các thuật ngữ
- Bài 4. Cách sử dụng GitHub
- Bài 5. Phối hợp trong công việc bằng GitHub

Bài 1.

Quản lý phiên bản là gì?

1.1. Giới thiệu về tính năng Quản lý phiên bản

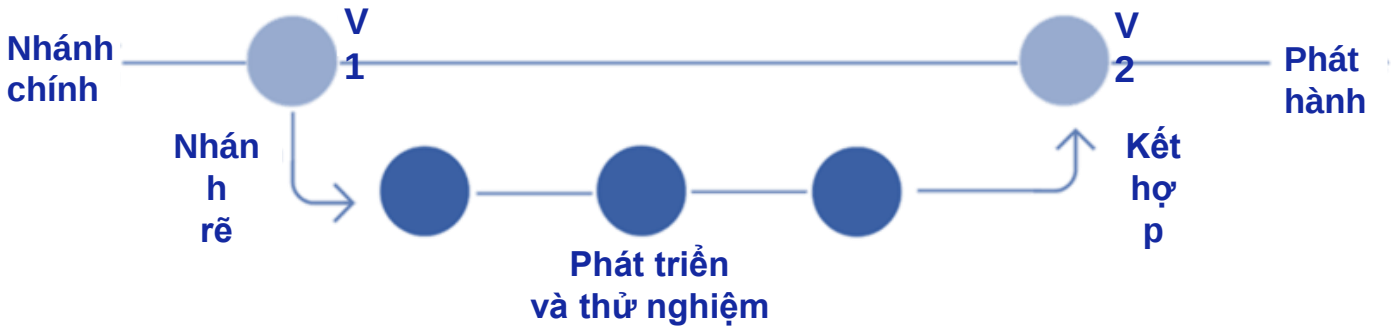
1.2. Phân loại Quản lý phiên bản

1.3. Lợi ích khi sử dụng tính năng Quản lý phiên bản

Quản lý phiên bản

'Quản lý phiên bản' là một trong những kỹ năng thiết yếu trong kỹ thuật phần mềm bởi vì tất cả các phần mềm đều có mã nguồn và chức năng được phát triển thường xuyên liên tục.

- Quản lý phiên bản có nghĩa là quản lý nhiều phiên bản khác nhau của mã nguồn. Và không chỉ là mã nguồn mà còn cả những thay đổi trong bộ tài liệu liên quan ,bao gồm cả tài liệu thiết kế, cũng được phân loại bằng cách cung cấp cho chúng một 'phiên bản' ký hiệu từ chữ số hoặc chữ cái.
  - Thông qua 'phiên bản' đã được xác định, có thể theo dõi thời gian thay đổi, các nội dung đã thay đổi và người đã thực hiện thay đổi đó.
- Với việc dung lượng của phần mềm ngày càng lớn hơn, phức tạp hơn và đa dạng hơn, tất cả các nhà phát triển phần mềm cần hiểu về Quản lý phiên bản để sử dụng nó một cách hiệu quả nhất có thể.



1.1. Giới thiệu về tính năng Quản lý phiên bản

Hệ thống Quản lý phiên bản (Version Control system - VCS)

- Các hệ thống Quản lý phiên bản là những công cụ phát triển phần mềm được thiết kế để giúp các nhà phát triển có thể theo dõi bất kỳ thay đổi nào với mã nguồn của bất kỳ ứng dụng cụ thể nào.

Sự cần thiết của Quản lý phiên bản

- Giúp phục hồi lại công việc khi có sự cố xảy ra.
- Có thể quay ngược lại phần công việc đã thực hiện ở một thời điểm nào đó trong một dự án.
- Tự động đồng bộ các thay đổi được thực hiện bởi mỗi cá nhân khi có nhiều người cùng tham gia vào một dự án.
- Theo dõi các thay đổi với mã nguồn và biết được ai đã thực hiện các thay đổi đó.
- Giúp việc điều chỉnh quy mô lớn trở nên an toàn và đảm bảo hơn.
- Được ứng dụng trong nhiều dự án mã nguồn mở dưới hình thức này hay hình thức khác.
- Giúp theo dõi lý do tại sao một đoạn mã lệnh cụ thể được viết theo mà nó đang có.

1.1. Giới thiệu về tính năng Quản lý phiên bản

Các hệ thống Quản lý phiên bản phổ biến nhất

- Phổ biến nhất hiện nay là Git, được hàng triệu nhà phát triển và lập trình viên trên toàn thế giới sử dụng.

Một số hệ thống khác được sử dụng nhiều:

- Perforce Helix Core
- Apache Subversion
- AWS CodeCommit



Bài 1.

Quản lý phiên bản là gì?

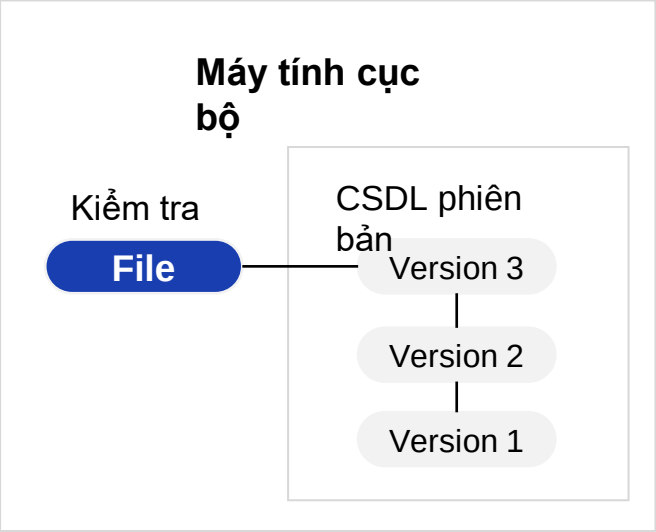
1.1. Giới thiệu về tính năng Quản lý phiên bản

1.2. Phân loại Quản lý phiên bản

1.3. Lợi ích khi sử dụng tính năng Quản lý phiên bản

3 nhóm hệ thống Quản lý phiên bản

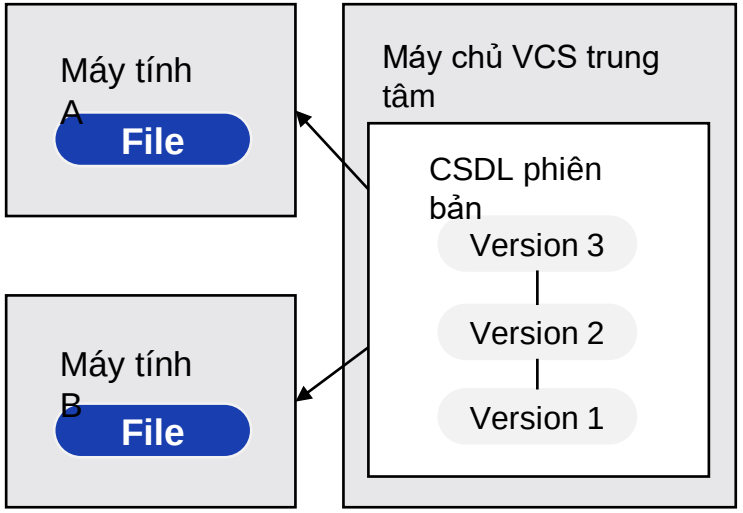
Các hệ thống Quản lý phiên bản cục bộ



- LVCS (LVCS – Local VCS) sử dụng một cơ sở dữ liệu đơn giản để quản lý các thay đổi trên file.
- Trong các hệ thống Quản lý phiên bản (VCS) được sử dụng rộng rãi, hệ thống quản lý hiệu chỉnh (RCS – Revision Control System) đang được nhiều công ty lựa chọn sử dụng. RCS về cơ bản quản lý bộ các bản vá (phần có thay đổi trong file).
- Các bản vá được lưu dưới dạng file có định dạng đặc biệt. Bạn có thể khôi phục tất cả các file về phiên bản ở một thời điểm cụ thể bằng cách áp dụng một loạt các bản vá.

1.2. Phân loại Quản lý phiên bản

Các hệ thống Quản lý phiên bản trung tâm

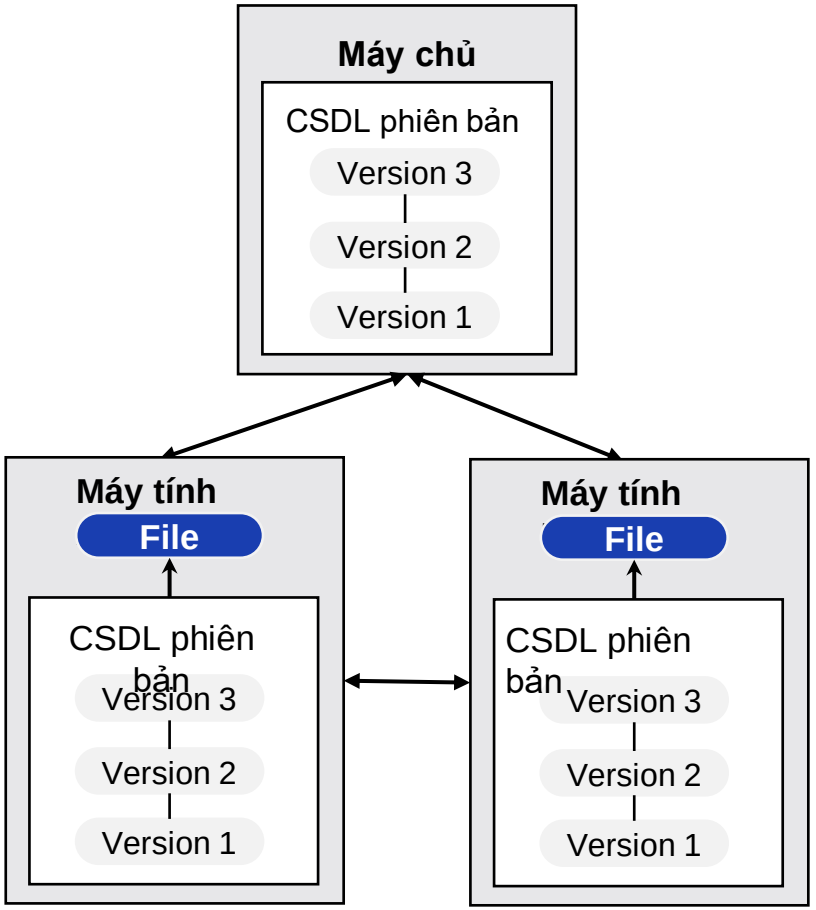


Hệ thống
Quản lý phiên bản trung tâm

- Các hệ thống Quản lý phiên bản trung tâm (CVCS – Centralized Version Control Systems) là một loại hệ thống Quản lý phiên bản với một máy chủ duy nhất chứa tất cả các phiên bản của các file mã nguồn, số lượng nhân sự tham gia và các file sao lưu cho dự án chính.
- CVCS giúp giao tiếp và quản lý nhân sự dễ dàng hơn dựa trên các nhiệm vụ cụ thể do nhân sự cùng nhóm thực hiện dự án sẽ biết những gì người khác đang làm.
- Nhược điểm chính của CVCS là tất cả các file mã nguồn và file sao lưu của bạn được lưu trữ trên một máy chủ tập trung. Nếu có sự cố xảy ra với máy chủ, toàn bộ các file sẽ có nguy cơ biến mất.
- Các công cụ Subversion và Perforce là những ví dụ về CVCS.

1.2. Phân loại Quản lý phiên bản

Các hệ thống Quản lý phiên bản phân tán



- Hệ thống Quản lý phiên bản phân tán (DVCS - Distributed Version Control Systems), chẳng hạn như Git, Mercurial, hoặc Bazaar, là các hệ thống Quản lý phiên bản với một hoặc nhiều máy chủ chính chứa các file mã nguồn.
- Trong DVCS, các file mã nguồn được ánh xạ trên máy tính của các nhân sự dự án khác với CVCS. Điều này có nghĩa là mỗi mã nguồn có phiên bản riêng, bao gồm toàn bộ lịch sử làm việc cục bộ.
- Ngay cả khi máy chủ dừng hoạt động, bộ nhớ lưu trữ của nhân sự dự án có thể được sao chép trở lại máy chủ và lịch sử của các file mã nguồn có thể được khôi phục. Thực hiện việc này rất đơn giản vì mọi bản sao của file mã nguồn gốc thực chất là bản sao lưu đầy đủ của tất cả dữ liệu từ dự án.
- DVCS cho phép nhân sự từ các địa điểm địa lý khác nhau cộng tác đồng thời trong cùng một dự án. Đây là một lợi thế không thể có trong một hệ thống tập trung.

Bài 1.

Quản lý phiên bản là gì?

1.1. Giới thiệu về tính năng Quản lý phiên bản

1.2. Phân loại Quản lý phiên bản

1.3. Lợi ích khi sử dụng tính năng Quản lý phiên bản

Lợi ích khi sử dụng hệ thống Quản lý phiên bản (1/2)

Tạo các bản sao lưu

- Lợi ích quan trọng nhất của việc sử dụng hệ thống Quản lý phiên bản là bất cứ khi nào nhân sự dự án tiến hành sao chép dữ liệu được lưu trữ, hệ thống sẽ tạo ra một bản sao lưu cho phiên bản hiện tại của kho lưu trữ đó.

Kiểm tra và Thử nghiệm

- Khi cùng thực hiện một dự án phần mềm, các nhóm lập trình thường có các bản sao khác nhau của dự án chính để phát triển và thử nghiệm tính năng mới. Trước khi thêm các tính năng mới này vào dự án chính của bạn, hệ thống sẽ đảm bảo rằng sản phẩm đang hoạt động bình thường. Việc này giúp tiết kiệm thời gian vì bạn có thể phát triển các tính năng khác nhau của chương trình đồng thời cùng lúc.

Lợi ích khi sử dụng hệ thống Quản lý phiên bản (2/2)

Lưu lịch sử và Theo dõi thay đổi

- Lưu trữ bản ghi chép các thay đổi trong một file mã nguồn cụ thể sẽ giúp bạn và những nhân sự mới tham gia biết được có thay đổi nào với phần mã nguồn đó.
  Ngoài ra, nếu việc bổ sung một số tính năng nhất định theo thời gian gây khó khăn cho việc đánh giá hoặc mở rộng quy mô dự án, khi này tính năng Quản lý phiên bản cho phép nhà phát triển theo dõi một số tính năng nhất định và thay đổi hoặc loại bỏ chúng mà không ảnh hưởng đến chức năng tổng thể của dự án.

Cộng tác và Đóng góp

- Một trong những lợi ích chính khi sử dụng hệ thống Quản lý phiên bản, đặc biệt là DVCS, là các bạn có thể đóng góp công sức cho các dự án yêu thích, ngay cả khi các bạn ở các quốc gia khác nhau.

Bài 2.

Git : Tổng quan & Cài đặt

2.1. Tổng quan về Git

2.2. Cài đặt Git & GitHub

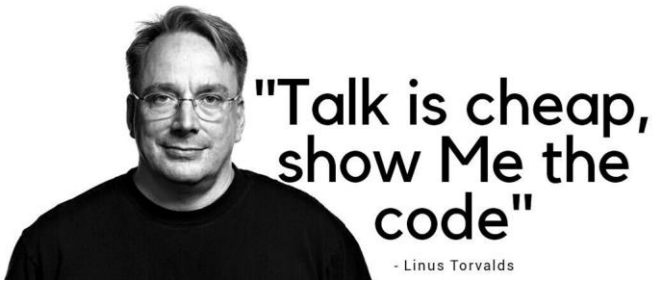
Git là gì?

- Khi phát triển phần mềm, rất khó để quản lý phiên bản mã nguồn mới nhất giữa các thành viên trong nhóm thực hiện. Vì lý do này, công cụ quản lý cấu hình (configuaration management tool) đã được tạo ra để giải quyết các vấn đề của Quản lý phiên bản xảy ra thường xuyên trong một dự án. Một trong những công cụ quản lý cấu hình này là Git, được phát triển bởi Linus Torvalds vào năm 2005 như một hệ thống quản lý phiên bản cho dự án phát triển lõi (kernel) Linux.

- Git tiếp cận dễ dàng vì nó ít phức tạp hơn so với công cụ cấu hình SVN (Subversion) hiện có. Đó là một hệ thống quản lý phiên bản phân tán, trong đó tất cả các nhân sự đều có bản gốc của mã nguồn và tiến độ thực hiện công việc rất nhanh.

- Git có thể thực hiện hầu hết các công việc trong mạng cục bộ. Nó là một công cụ quản lý cấu hình được sử dụng rộng rãi do ưu điểm về tốc độ khi thao tác với nhánh / thẻ / master và các bản chỉnh sửa khác.

Git Logo & Linus Torvalds



Đặc điểm của Git

- Là một hệ thống quản lý phiên bản phân tán cho phép người dùng duy trì và đồng bộ hóa kho dữ liệu Git từ xa.
- Git thực thi hầu hết tất cả các lệnh một cách cục bộ, đồng thời việc tạo / chuyển đổi / xử lý nhánh rất nhanh.
- Git quản lý bản sao lưu toàn bộ hệ thống khác với các hệ thống Quản lý phiên bản khác, chỉ lưu trữ các thay đổi cho từng file riêng lẻ.
- Chức năng chính
  - Các tính năng của máy chủ và máy khách trong hệ thống quản lý phiên bản tài liệu và mã nguồn
  - Các thao tác cục bộ như add, commit, reset, branch, checkout, merge, rebase, etc.
  - Các thao tác từ xa như push, pull, fetch, etc.

Bài 2.

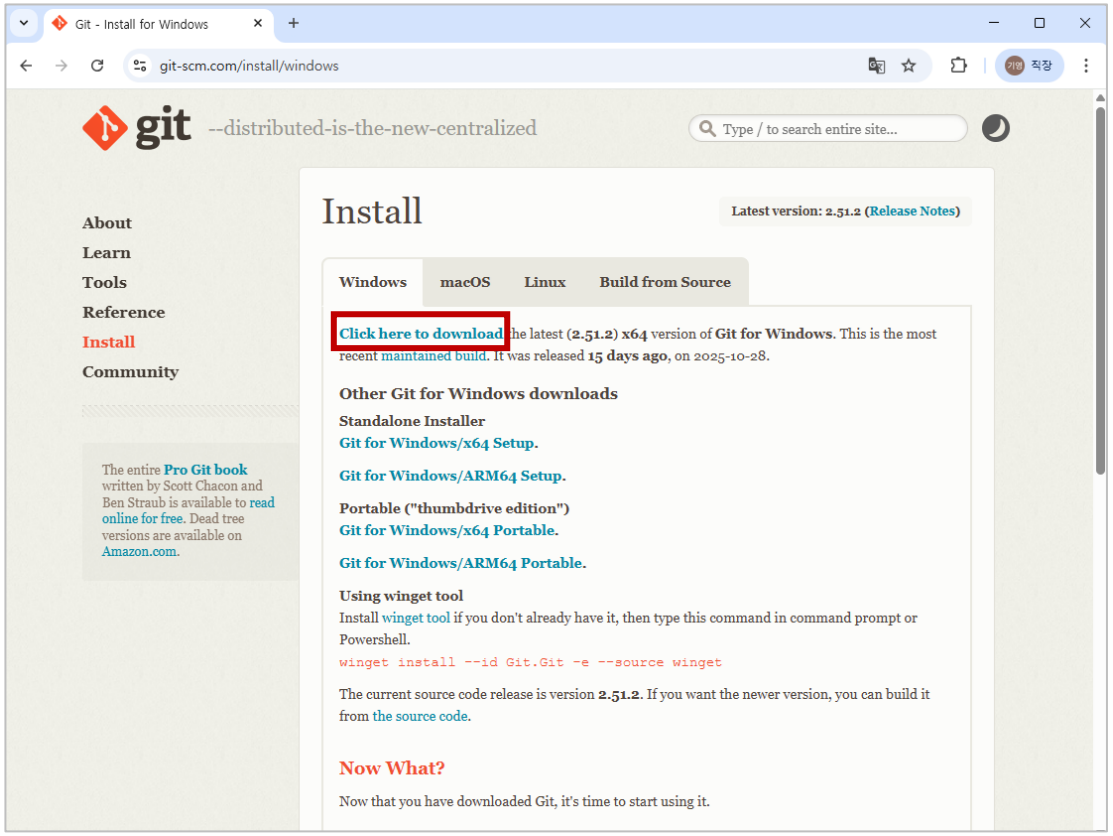
Git : Tổng quan & Cài đặt

2.1. Git Overview

2.2. Git : Tổng quan & Cài đặt

Cài đặt (Windows 11)

Hướng dẫn cài đặt Git

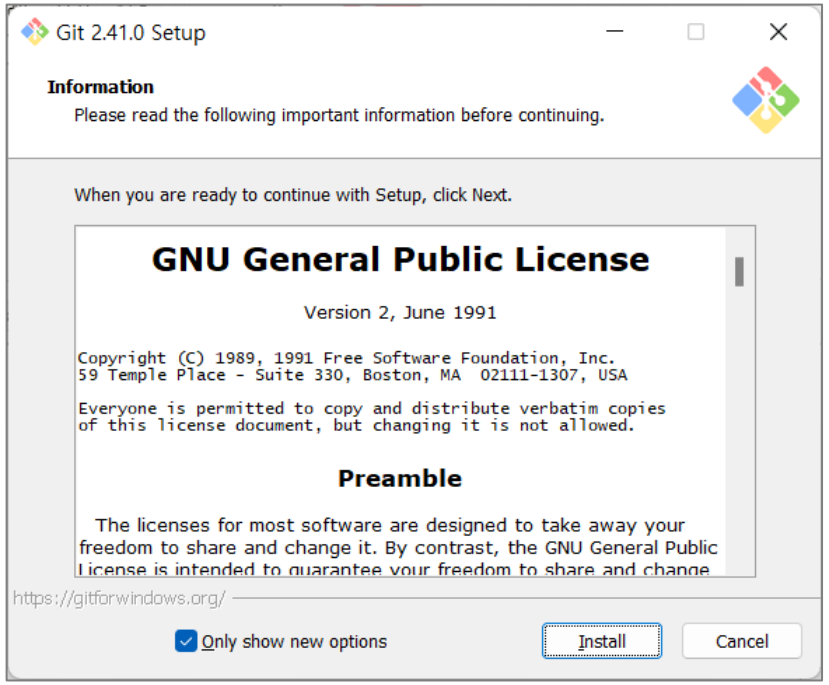


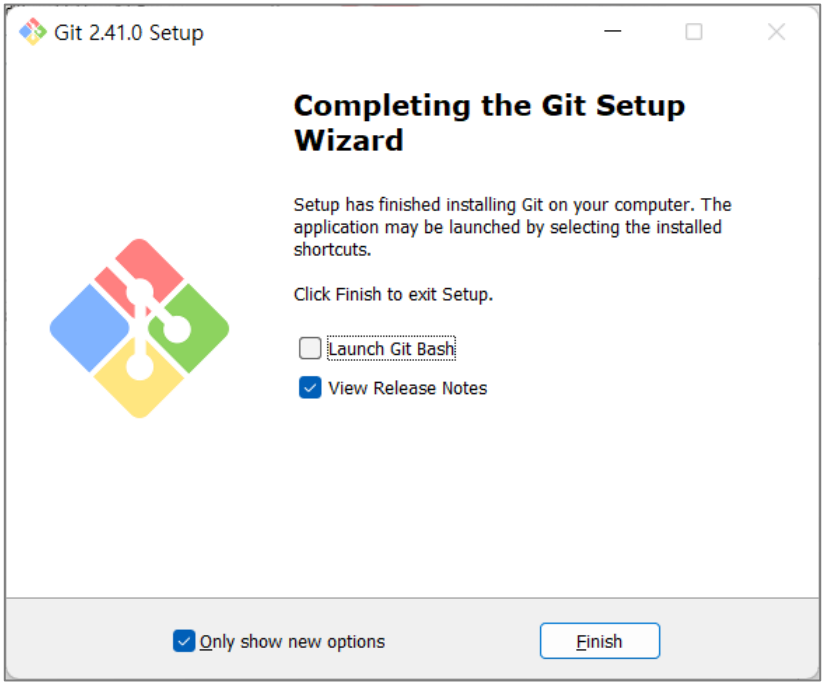
- Truy cập tới **https://git-scm.com/install/windows** để tải bản cài và tiến hành cài đặt
- Nhấp vào liên kết **[Click here to download]**, sau đó bạn có thể tải xuống tệp cài đặt.

Cài đặt (Windows 11)

Hướng dẫn cài đặt Git

- Chạy trình cài đặt đã tải xuống để tiếp tục quá trình cài đặt.

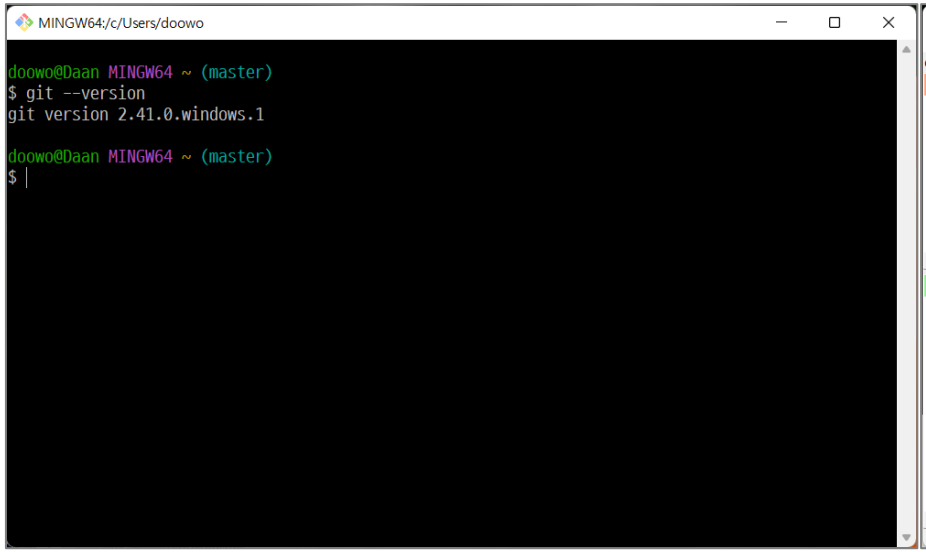




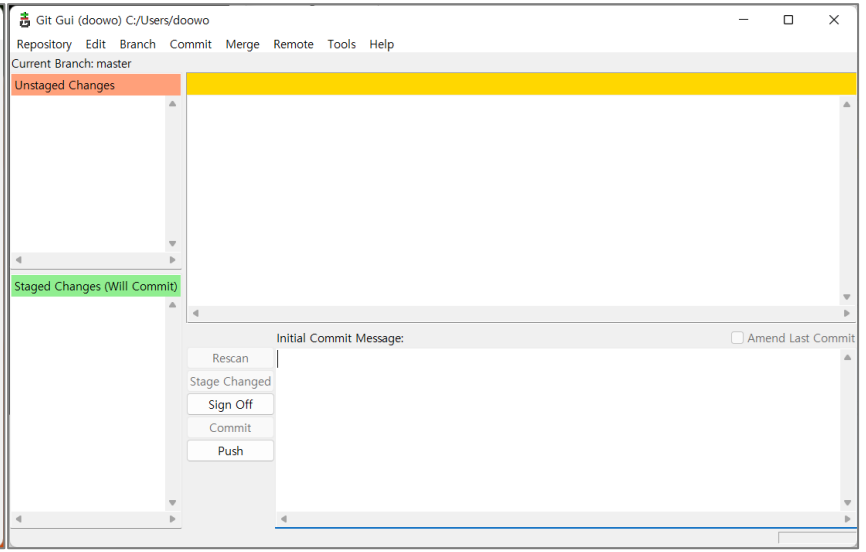
Cài đặt (Windows 11)

2 cách sử dụng Git

- CLI (Giao diện dòng lệnh)



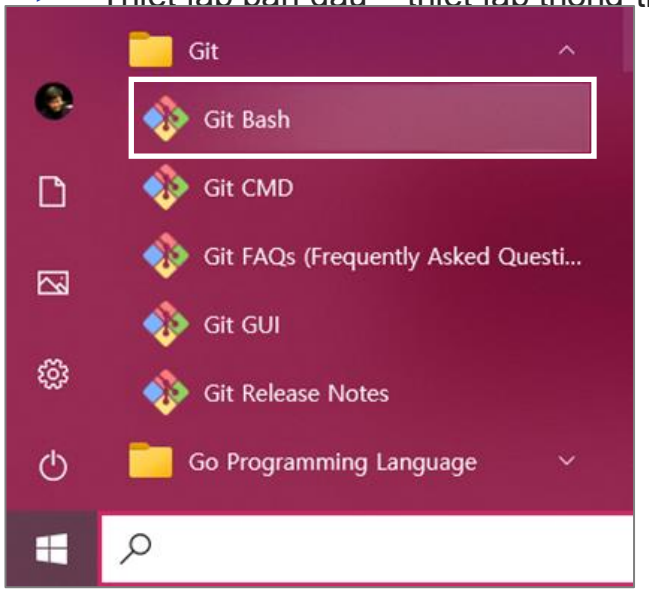
- GUI (Giao diện người dùng đồ họa)

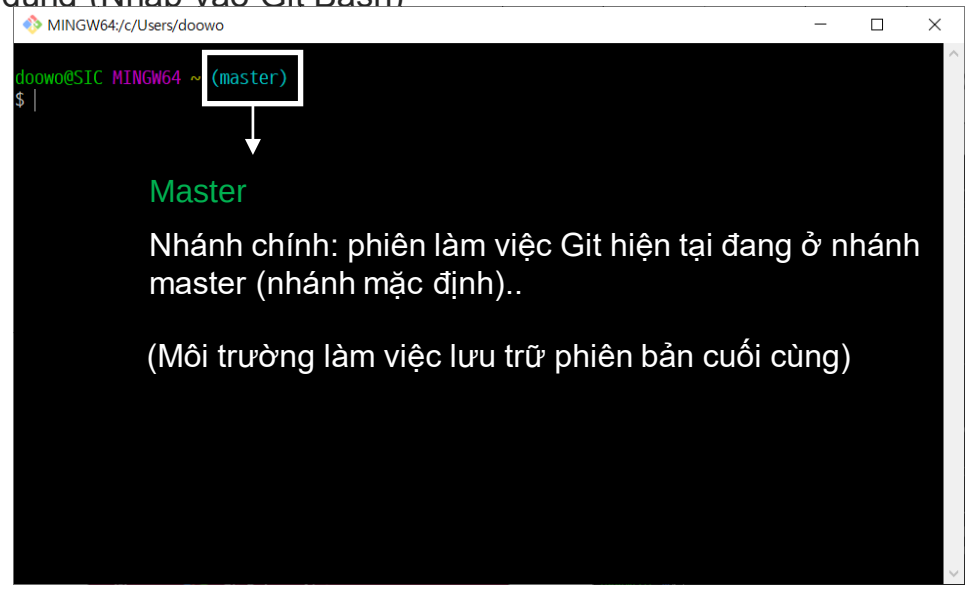


Cài đặt (Windows 11)

Thiết lập ban đầu Git

- Thiết lập ban đầu – thiết lập thông tin người dùng (Nhấp vào Git Bash)





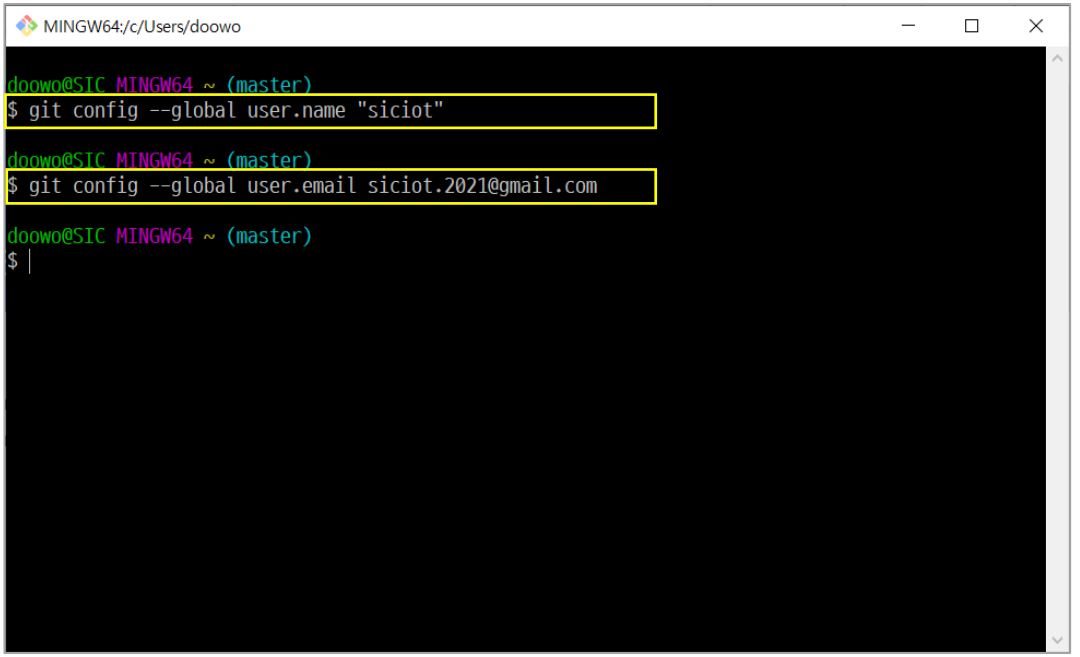
- Branch (master) là gì?
  - Một nhánh (branch) trong Git là một dòng phát triển riêng biệt, được sử dụng để quản lý các phiên bản hoặc tính năng khác nhau của dự án.
  - Nhánh master là nhánh chính mặc định, nơi thường lưu trữ phiên bản cuối cùng, ổn định của dự án.

Cài đặt (Windows 11)

Thiết lập ban đầu Git

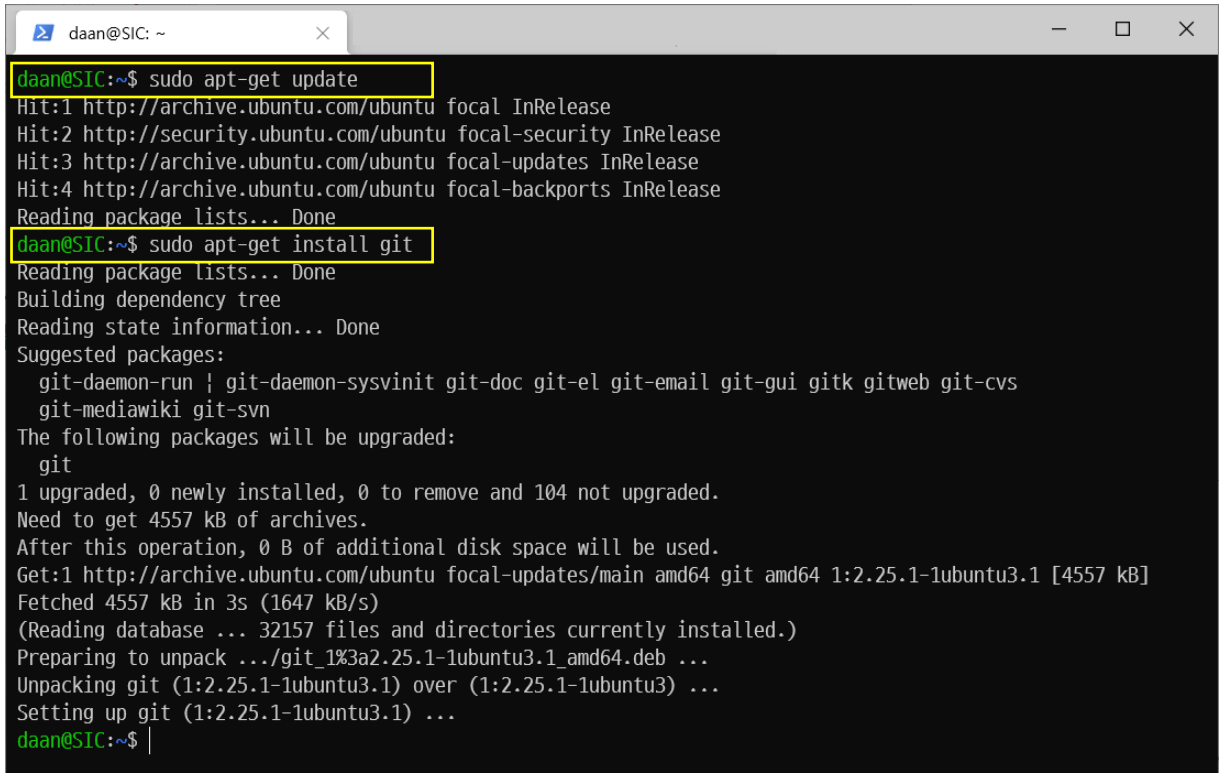
- Thiết lập thông tin người dùng
- Lưu ý: Tên người dùng “siciot” và email “siciot.2021@gmail.com” trong các lệnh chỉ là ví dụ. Vui lòng **thay thế bằng tên người dùng GitHub và địa chỉ email của bạn.**

```
$ git config --global user.name “siciot”
$ git config --global user.email siciot.2021@gmail.com
```



Cài đặt (ubuntu Raspberry Pi OS)

Hướng dẫn cài đặt Git



- Cài đặt bằng lệnh trong Terminal

```
$ sudo apt-get update
$ sudo apt-get install git
```

Cài đặt (ubuntu Raspberry Pi OS)

Hướng dẫn cài đặt Git



- Kiểm tra phiên bản Git

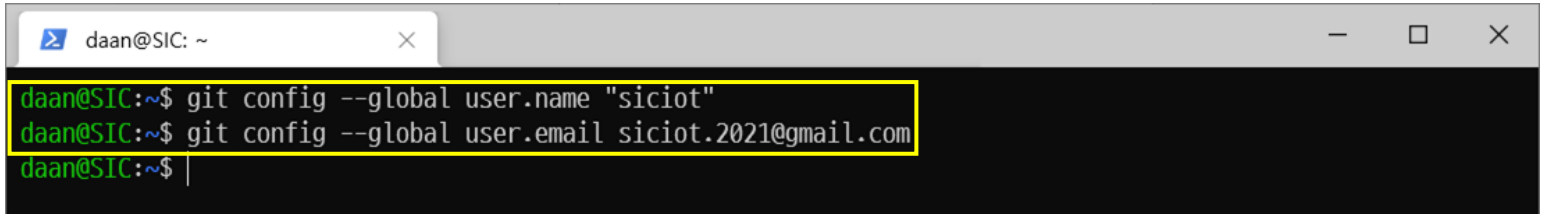
```
$ git --version
```

Cài đặt (ubuntu Raspberry Pi OS)

Hướng dẫn cài đặt Git

- Thiết lập ban đầu – thiết lập thông tin người dùng
- user.name và user.email là tên và địa chỉ email sẽ được hiển thị trong Git.
- Lưu ý: Tên người dùng “siciot” và email “siciot.2021@gmail.com” trong các lệnh chỉ là ví dụ. Vui lòng **thay thế bằng tên người dùng GitHub và địa chỉ email của bạn.**

```
$ git config --global user.name “siciot”
$ git config --global user.email siciot.2021@gmail.com
```



Bài 3.

Tổng quan về GitHub và các thuật ngữ

3.1. Tổng quan về GitHub

3.2. Thuật ngữ GitHub

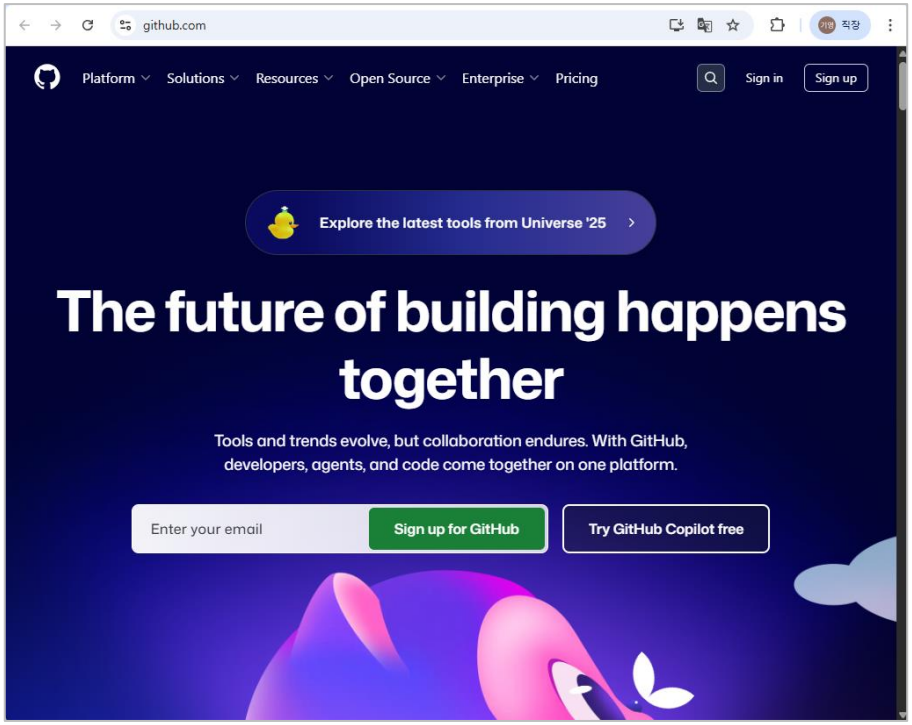
GitHub là gì?

GitHub là một nền tảng lưu trữ mã nguồn mở, cung cấp nhiều chức năng để lưu trữ và quản lý phiên bản mã nguồn trong phát triển phần mềm mã nguồn mở, đồng thời hỗ trợ cộng tác và trao đổi trực tuyến giữa cộng đồng lập trình viên.

Người dùng có thể duyệt các kho lưu trữ trên GitHub và tải xuống mã nguồn. Ngoài ra, có thể sử dụng các chức năng như thảo luận, quản lý kho lưu trữ, cộng tác với các kho khác và xem xét các thay đổi mã nguồn

GitHub

Nguồn: https://guides.github.com/activities/hello-world/



https://github.com/

GitHub là máy chủ lớn nhất phục vụ kho lưu trữ Git

Phần lớn các kho lưu trữ Git được lưu trữ trên GitHub. Nhiều dự án mã nguồn mở sử dụng GitHub để lưu trữ Git, theo dõi vấn đề (issue), xem xét mã nguồn (code review), và nhiều chức năng khác.

Mặc dù GitHub không phải là một phần trực tiếp của dự án mã nguồn mở Git, nhưng trong quá trình sử dụng Git một cách chuyên nghiệp, bạn sẽ cần hoặc muốn tương tác với GitHub vào một thời điểm nào đó.



Quản lý phiên bản



Collaboration & Backup

Bài 3.

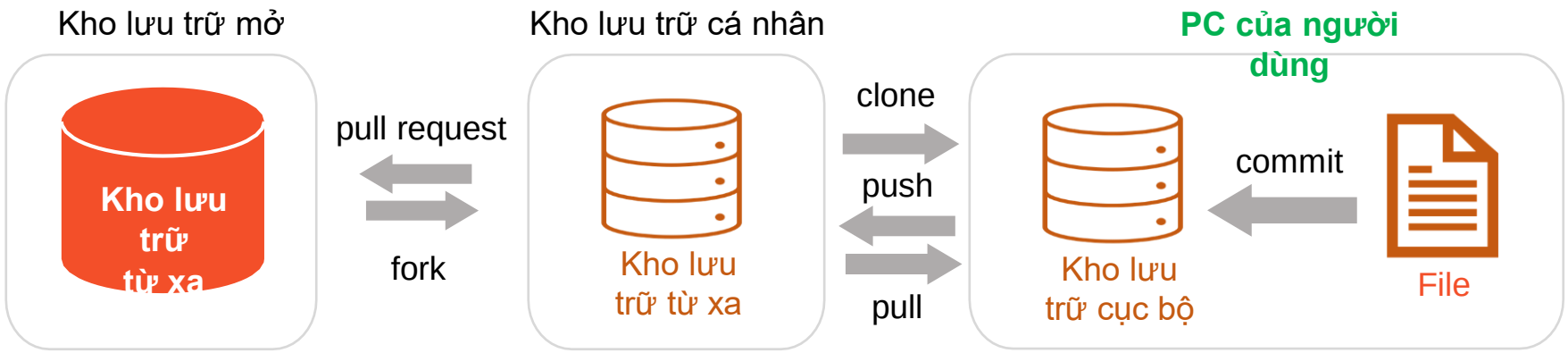
Tổng quan về GitHub và các thuật ngữ

3.1. Tổng quan về GitHub

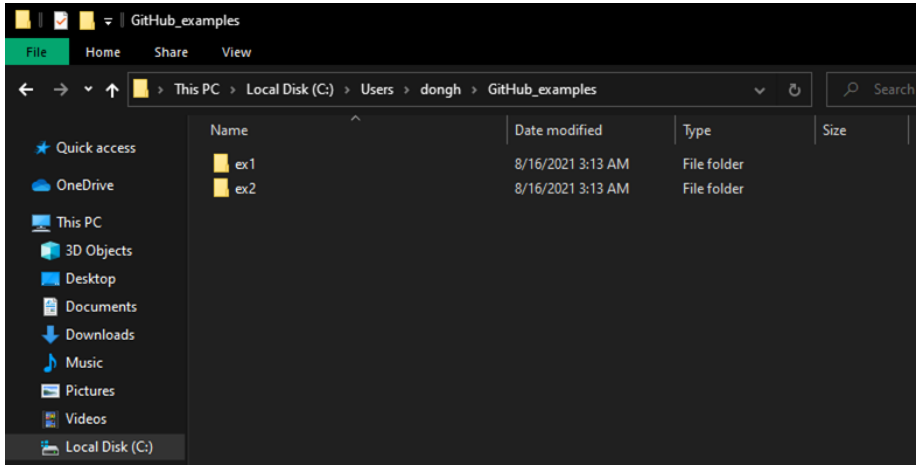
3.2. Thuật ngữ GitHub

GitHub là gì?

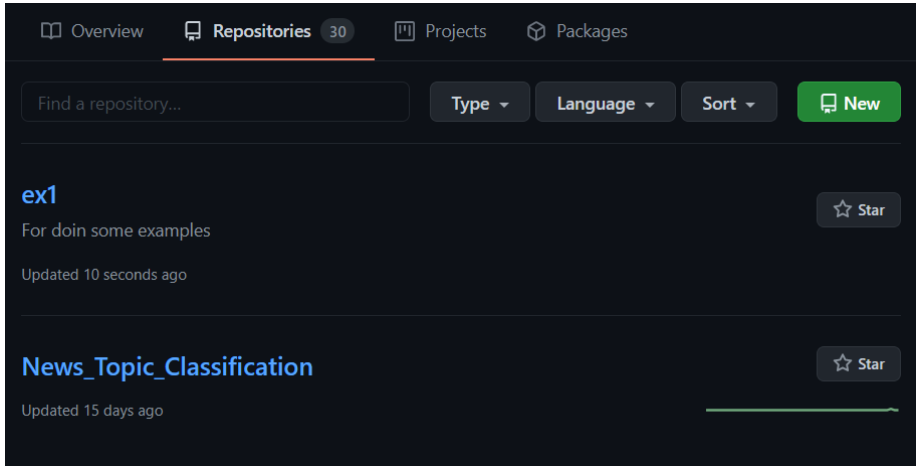
Tìm hiểu thuật ngữ GitHub



Tìm hiểu thuật ngữ GitHub



- Kho lưu trữ (Repository): Nơi lưu trữ file hoặc thư mục
- Kho lưu trữ cục bộ (Local Repository): Kho lưu trữ trên PC cục bộ



- Kho lưu trữ từ xa (Remote Repository): Kho lưu trữ trên Internet hoặc trực tuyến như GitHub

Tóm tắt các câu lệnh

- git init : Lệnh này tạo ra một kho lưu trữ Git mới. Nó có thể được sử dụng để chuyển đổi một dự án hiện có, không được đánh dấu phiên bản thành kho lưu trữ Git hoặc khởi tạo một kho lưu trữ trống mới. Hầu hết các lệnh Git khác không có sẵn bên ngoài kho lưu trữ được khởi tạo, vì vậy đây thường là lệnh đầu tiên bạn sẽ chạy trong một dự án mới.

```
$ git init
```

- git config : Lệnh này kiểm tra và thay đổi cài đặt. Bạn có thể đặt tên người dùng hoặc địa chỉ e-mail Khi nó được sử dụng để thiết lập môi trường git, bạn có thể kiểm tra cài đặt bằng cách sử dụng lệnh git config --list.

```
$ git config
```

- Ex $ git config --global user.email johndoe@example.com
- git status : Lệnh này kiểm tra trạng thái kho lưu trữ. Bạn có thể xem nội dung nào trong kho lưu trữ, những thay đổi nào cần được thực hiện và nhánh nào của kho lưu trữ bạn hiện đang làm việc.

```
$ git status
```

```
siciot2021@siciot2021-VirtualBox:~/example1$ git status
On branch main
Your branch is up to date with 'origin/main'.

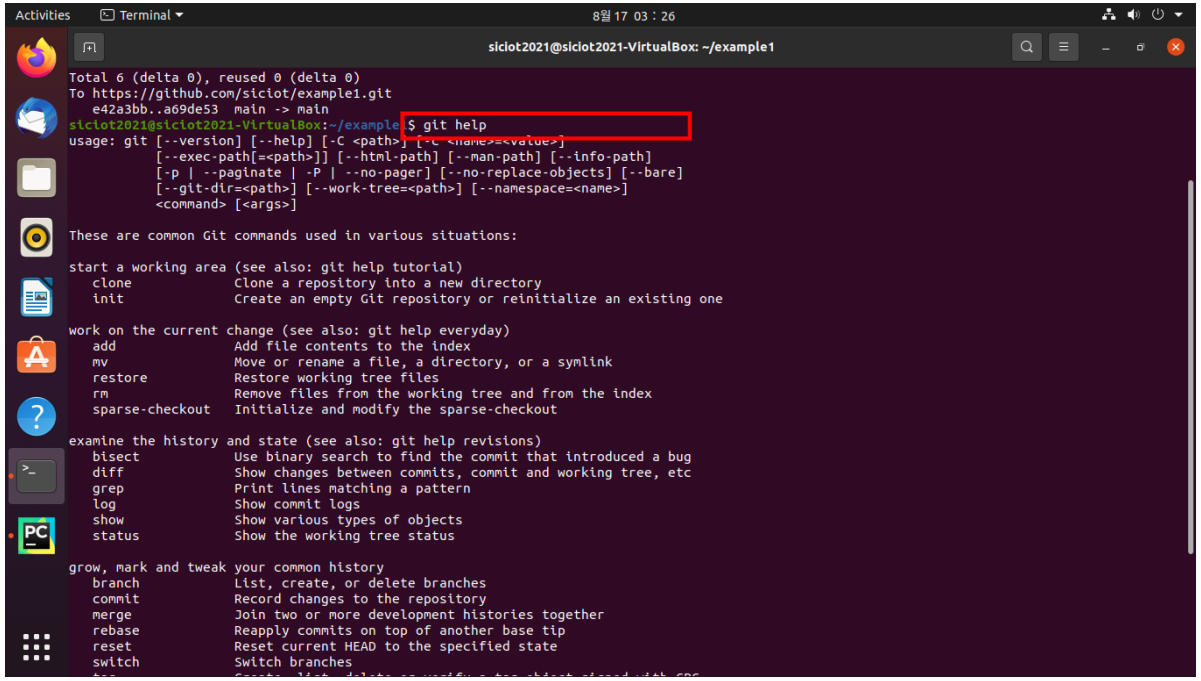
nothing to commit, working tree clean
```

Tóm tắt các câu lệnh

- git help : Lệnh này kiểm tra các lệnh Git khi bạn quên một lệnh để sử dụng
- git help init : Lệnh này cũng kiểm tra các lệnh Git cụ thể

```
$ git help
```

```
$ git help init
```



Tóm tắt các câu lệnh

- git add : Lệnh này không thêm file mới vào kho lưu trữ. Thay vào đó, nó cho phép git xem các file mới. Khi một file được thêm vào, nó sẽ được đưa vào kho lưu trữ "ảnh chụp nhanh" trong Git và được đưa vào mục tiêu theo dõi.

```
$ git add
```

- git commit : Lệnh này cho phép chụp "ảnh chụp nhanh” (snapshot) của kho lưu trữ sau khi thực hiện bất kỳ thay đổi nào. Thông thường, nó được sử dụng dưới dạng "git commit -m "Message". -m là một tùy chọn có nghĩa là phần tiếp theo của lệnh là một thông báo.

```
$ git commit -m “Message”
```

- git branch : Lệnh này tạo một nhánh mới và tạo một dòng thời gian xác nhận về các thay đổi và bổ sung file của riêng bạn. Sử dụng biểu mẫu tương tự như git branch {new branch name}. Trong tình huống cộng tác, kho lưu trữ công khai có thể tạo một nhánh mới để công việc của tôi không bị ảnh hưởng bởi các nhân viên khác.

```
$ git branch {new branch name}
```

- git checkout : Lệnh này kiểm tra một kho lưu trữ hiện không được định vị. Nó là một lệnh điều hướng đưa bạn đến kho lưu trữ bạn muốn kiểm tra. Nếu bạn muốn xem nhánh chính, bạn có thể sử dụng git checkout master và git checkout {destination branch} để xem một nhánh khác. Các bạn cũng có thể sử dụng git log để xem commit id và quay lại commit trước đó.

```
$ git checkout
```

Tóm tắt các câu lệnh

- git merge : Lệnh này kết thúc công việc trên một nhánh và hợp nhất nó thành một nhánh chính mà tất cả các cộng tác viên có thể nhìn thấy. Lệnh Git Merge Cats thêm tất cả các thay đổi được thực hiện trong nhánh "Cats" vào Master.

```
$ git merge
```

- git push : Lệnh này đẩy các thay đổi sang GitHub khi bạn muốn áp dụng xác nhận vào Kho lưu trữ GitHub sau khi làm việc trên máy tính cục bộ của bạn.

```
$ git push
```

- git pull : Lệnh này tải xuống các thay đổi từ GitHub khi bạn đang làm việc trên máy cục bộ của mình và muốn có phiên bản mới nhất của kho lưu trữ mà bạn đang làm việc.

```
$ git pull
```

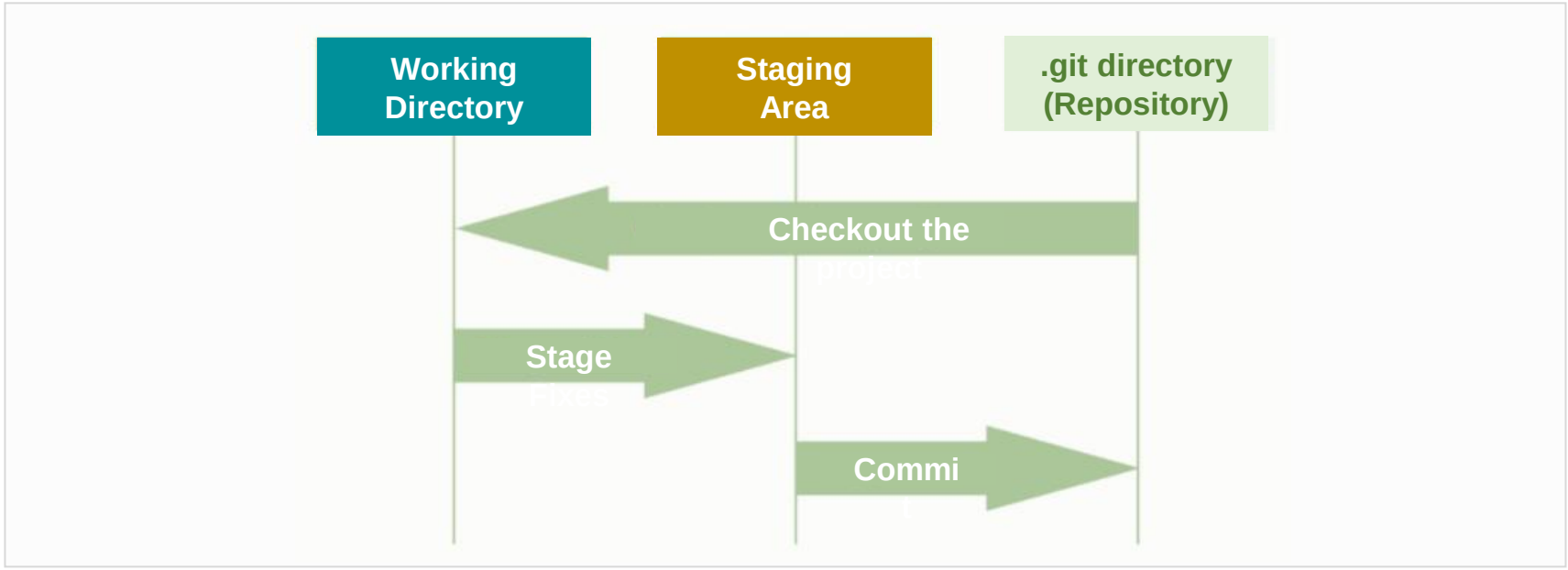
- git clone : Lệnh này nhập một dự án tồn tại trong kho lưu trữ GitHub và ngay lập tức bắt đầu làm việc trên dự án. Nó thực hiện chức năng tương tự như git init + git remote add origin { URL } + git pull origin main.

```
$ git clone
```

```
$ git init + git remote add origin { URL } + git pull origin main
```

3 trạng thái của File

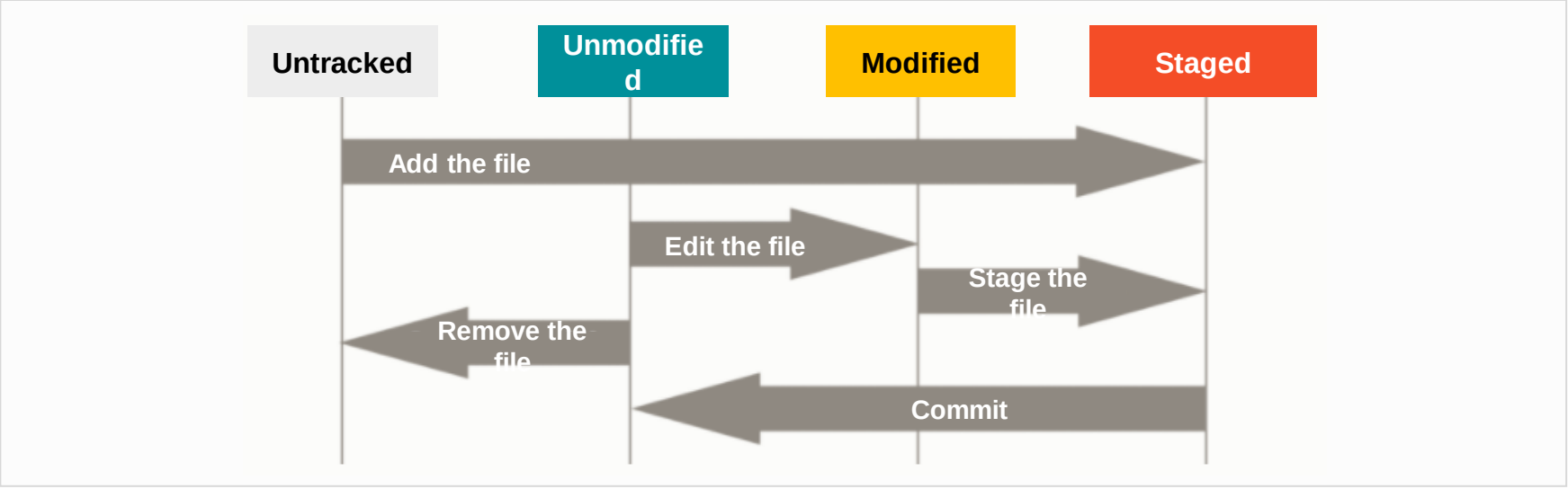
- Git quản lý các file ở ba trạng thái: Committed, Modified, and Staged.
  - Committed : Điều đó có nghĩa là dữ liệu được lưu trữ an toàn trong cơ sở dữ liệu cục bộ.
  - Modified : Nó có nghĩa là file đã được thay đổi nhưng nó chưa được xác nhận với cơ sở dữ liệu cục bộ.
  - Staged : Điều đó có nghĩa là bạn đã đánh dấu một file đã sửa đổi trong phiên bản hiện tại của nó để chuyển sang ảnh chụp nhanh xác nhận tiếp theo của bạn.



https://paransilverlight.tistory.com/242

Vòng đời của File

- Tất cả các file được chia thành 2 loại Theo dõi và Không theo dõi. Các file được theo dõi đã được bao gồm trong ảnh chụp nhanh. Các file được theo dõi cũng là một trong những file Chưa sửa đổi, Sửa đổi và Dàn dựng (được xác nhận). Nói một cách đơn giản, các file được theo dõi là các file mà Git biết. Tất cả các file không phải là file được theo dõi đều là file không được theo dõi. Các file không được theo dõi không được bao gồm trong ảnh chụp nhanh cũng như trong khu vực dàn dựng. Khi bạn sao chép kho lưu trữ lần đầu tiên, nó ở trạng thái Chưa sửa đổi được theo dõi vì không có gì được sửa đổi sau khi thanh toán. Nếu một file được sửa đổi sau xác nhận cuối cùng trong khi chưa có gì được sửa đổi, Git sẽ nhận ra file đó là đã được sửa đổi. Để xác nhận, file đã sửa đổi được đưa vào trạng thái theo giai đoạn. Vòng đời xác nhận file trạng thái theo giai đoạn này được lặp lại.



Nhánh là gì?

- Hầu hết các hệ thống quản lý phiên bản đều hỗ trợ chức năng phân nhánh.
  - Trong quá trình phát triển phần mềm, cần phải sao chép nhiều mã nguồn. Bạn có thể tạo toàn bộ các mã cùng nhau, nhưng thường xảy ra trường hợp bạn có thể chia toàn bộ mã thành từng phần để lập trình riêng biệt. Sau khi sao chép mã, bạn có thể phát triển nó một cách độc lập với mã gốc. Nói cách khác, chức năng cho phép bạn phát triển độc lập là Nhánh (Branch).

    → Nhánh theo nghĩa đen giống như một nhánh cây; nó cho phép bạn làm việc riêng lẻ độc lập với toàn bộ dự án. Bạn có thể tạo một không gian làm việc duy nhất và bắt đầu làm việc. Nếu kết quả tốt, bạn có thể áp dụng chúng cho nhánh chính có kết quả ổn định. Chức năng này cải thiện hiệu quả và tính ổn định của dự án.

Nhánh là gì?

- Nhánh của Git rất nhẹ và nhanh chóng để chuyển đổi, vì vậy có thể tạo một nhánh hoặc di chuyển giữa nhiều nhánh trong thời gian ngắn.
  - Không giống như các hệ thống quản lý phiên bản khác, Git khuyên bạn nên tạo các nhánh và hợp nhất chúng sau.

  → Tạo một branch là một công việc rất đơn giản như tạo một tập tin 41-byte duy nhất, nhưng nó hỗ trợ Quản lý phiên bản rất hiệu quả.
- Toàn bộ bản sao mất hàng chục giây đến hàng chục phút tùy thuộc vào quy mô của dự án, nhưng Git rất nhanh. Ngoài ra, vì thông tin của xác nhận trước đó được lưu mỗi khi bạn xác nhận, bạn có thể biết bắt đầu hợp nhất từ đâu (Merge Base).

  → Tính năng này là một tính năng mạnh mẽ để các nhà phát triển thường tạo và sử dụng các nhánh.

Nếu chúng ta gặp một số vấn đề phát sinh,

- Trong quá trình phát triển thực tế, chúng ta có thể sử dụng Branch như dưới đây:
  - Chúng tôi viết mã và làm việc trên dự án.
    Chúng tôi tạo một nhánh mới (issue_1) để xử lý các vấn đề phát sinh.
    Chúng tôi bắt đầu làm việc trên branch mới được tạo.

- Trong khi giải quyết vấn đề này, nếu bạn thấy một vấn đề quan trọng khác và bạn cần khắc phục ngay lập tức (Hotfix) :
  - Đến nhánh Production (không có vấn đề gì) trước khi xử lý các vấn đề mới.
    Tạo một nhánh hotfix mới.
    Sau khi kiểm tra Hotfix đã sửa, hợp nhất nhánh Hotfix vào nhánh Production.
    Đến nhánh (issue_1) bạn đang thực hiện và tiến hành các nhiệm vụ còn lại.

Bài 4.

Cách sử dụng GitHub

4.1. Bài luyện tập GitHub đơn giản

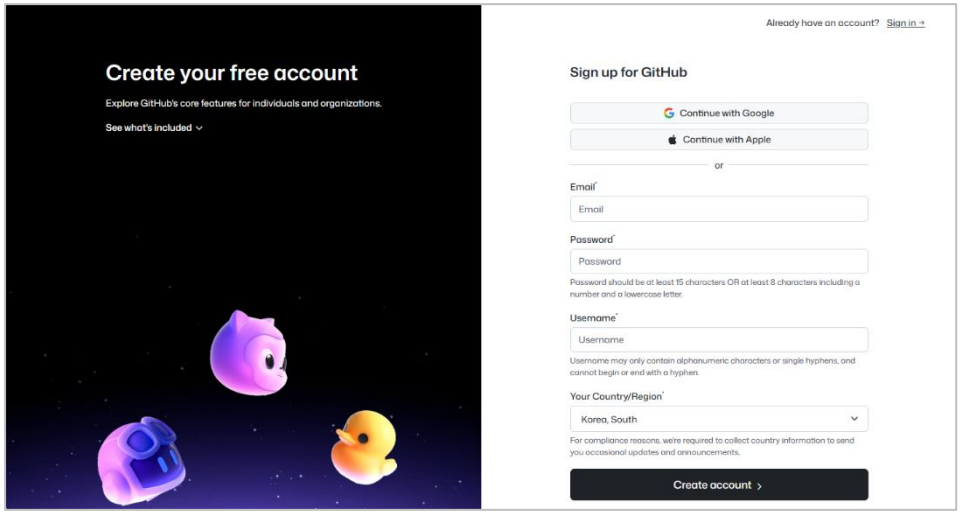
4.2. Cách sử dụng GitHub khi làm việc theo cá nhân

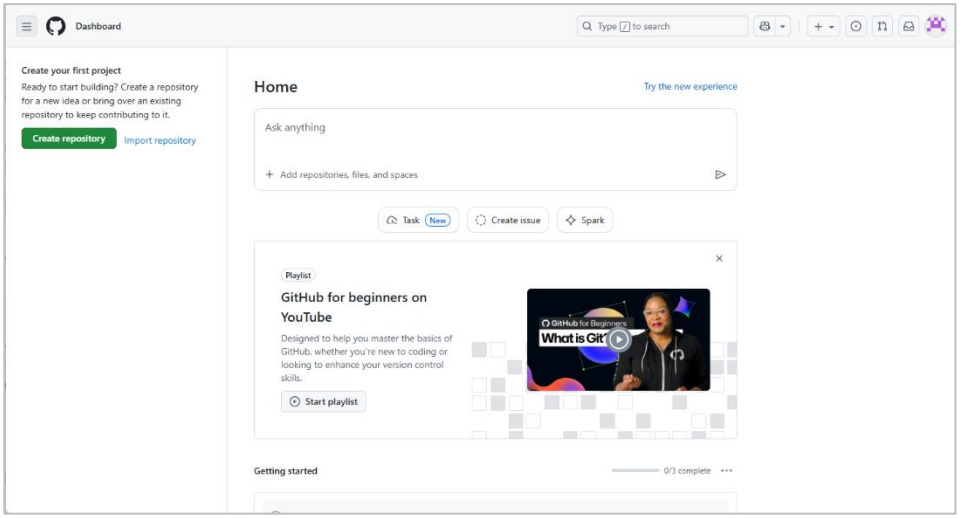
4.3. Cách sử dụng GitHub thông qua môi trường GUI

Cài đặt và Cấu hình tài khoản

Tạo một tài khoản

- Truy cập https://github.com
- Điền đầy đủ thông tin và nhấp vào Sign up..
- Tiến hành xác thực người dùng bằng email đã đăng ký.

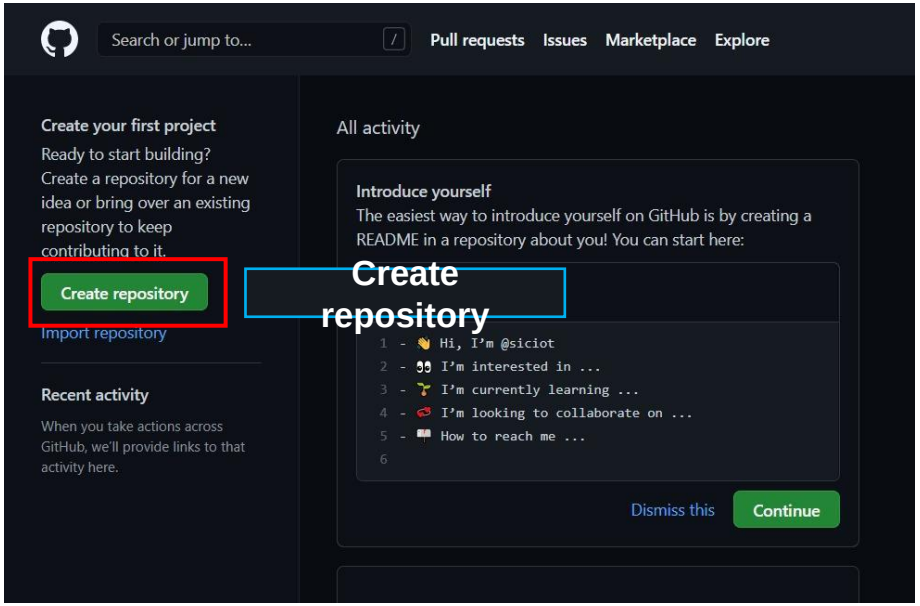


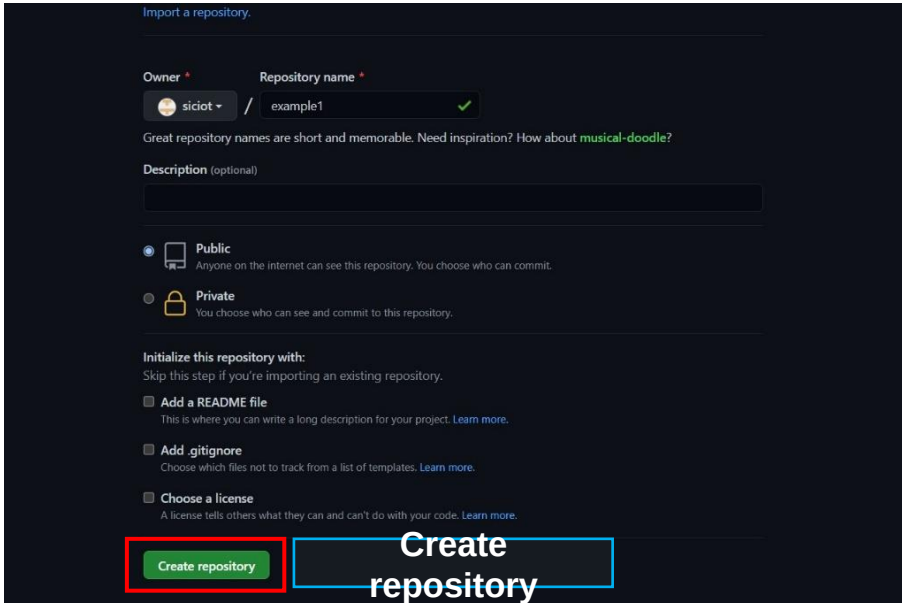


Tạo Kho lưu trữ cá nhân

Tạo kho lưu trữ cá nhân

- Sau khi đăng nhập, nhấp vào nút **Create repository** ở bên trái.
- Đặt tên Kho lưu trữ và nhấp vào nút **Create repository** ở dưới cùng.

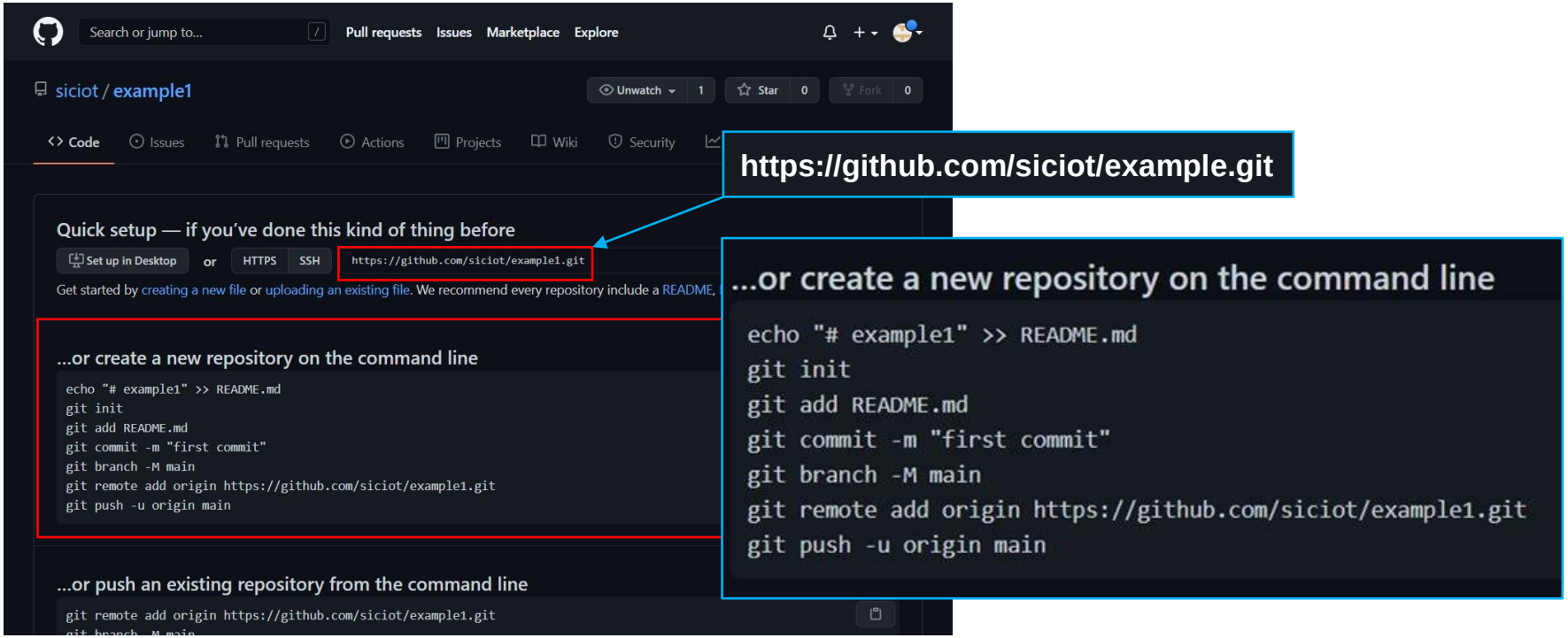




Tạo Kho lưu trữ cá nhân

Tạo Kho lưu trữ cá nhân

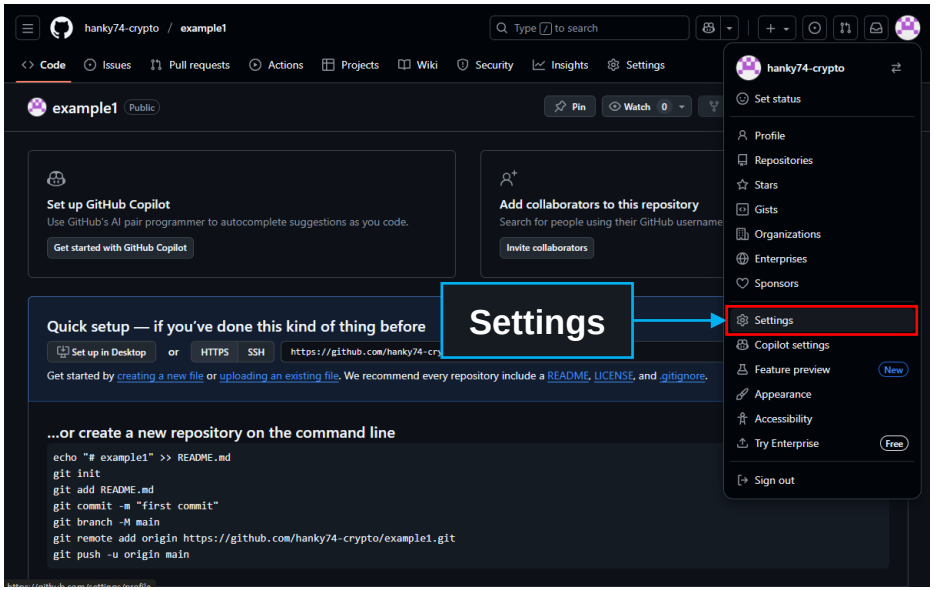
- Xác nhận rằng Kho lưu trữ từ xa có tên **example1** đã được tạo trong tài khoản siciot.
- Tiến hành như được mô tả trong .**[.. hoặc tạo một kho lưu trữ mới trên dòng lệnh.]**

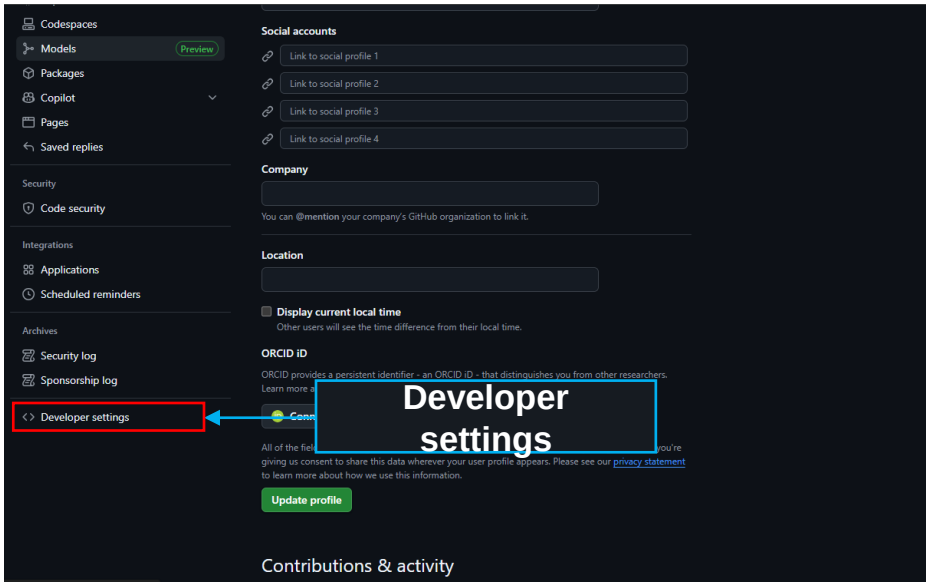


Thao tác Push My Document

Mã truy cập cá nhân

- Do thay đổi gần đây trong phương thức xác thực của GitHub, cần sử dụng mã truy cập cá nhân để xác thực..
- Nhấp vào mũi tên xuống tại hồ sơ của bạn → Settings → Developer settings.

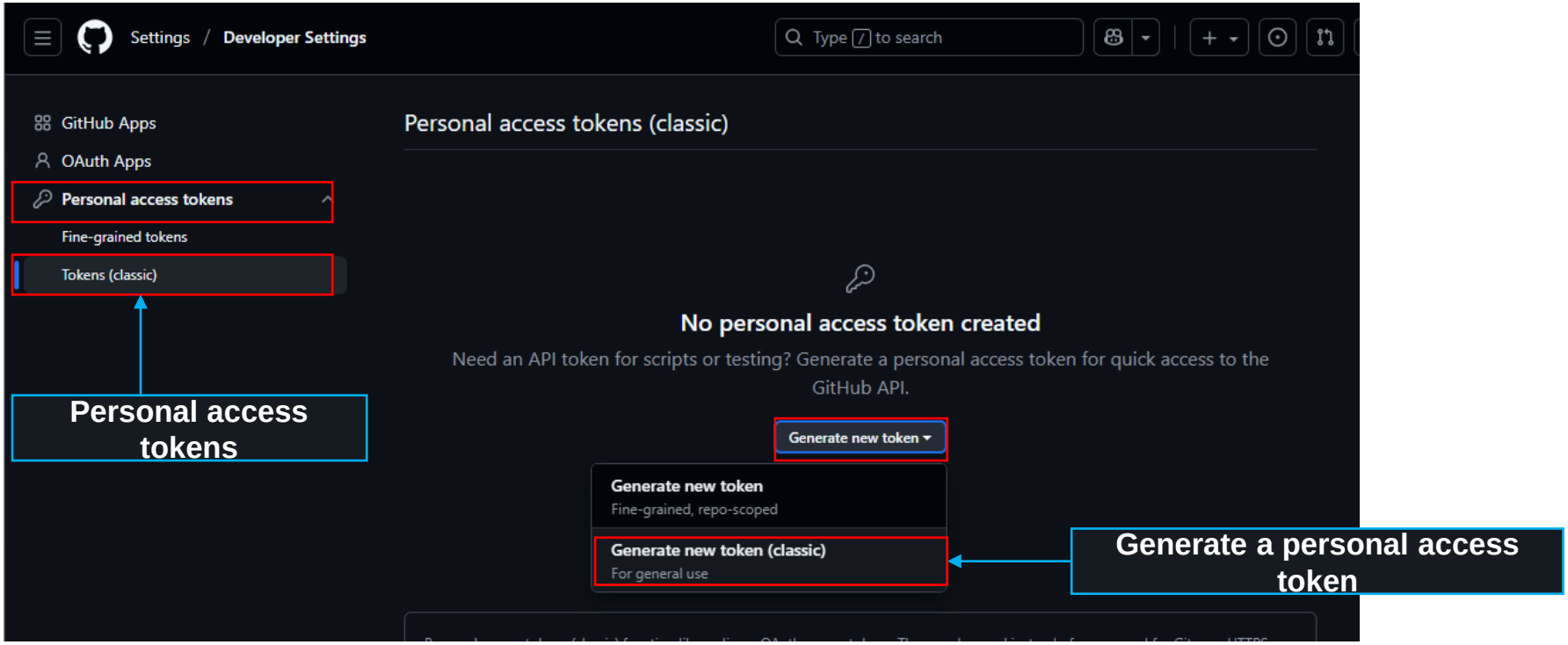




Thao tác Push My Document

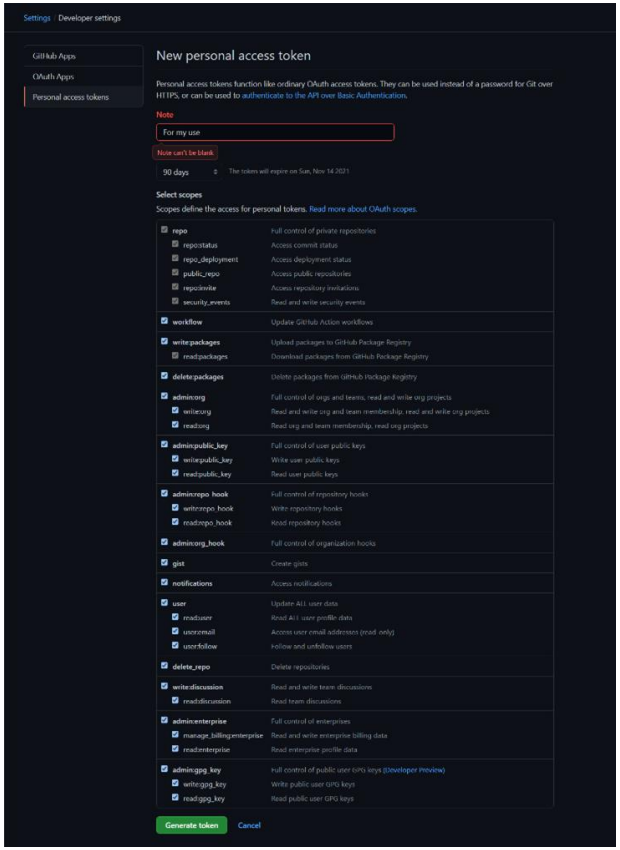
Mã truy cập cá nhân

- Nhấp vào Personal access tokens > Tokens (classic)
- Nhấp vào Generate new token > Generate new token (classic)



Thao tác Push My Document

Mã token truy cập cá nhân

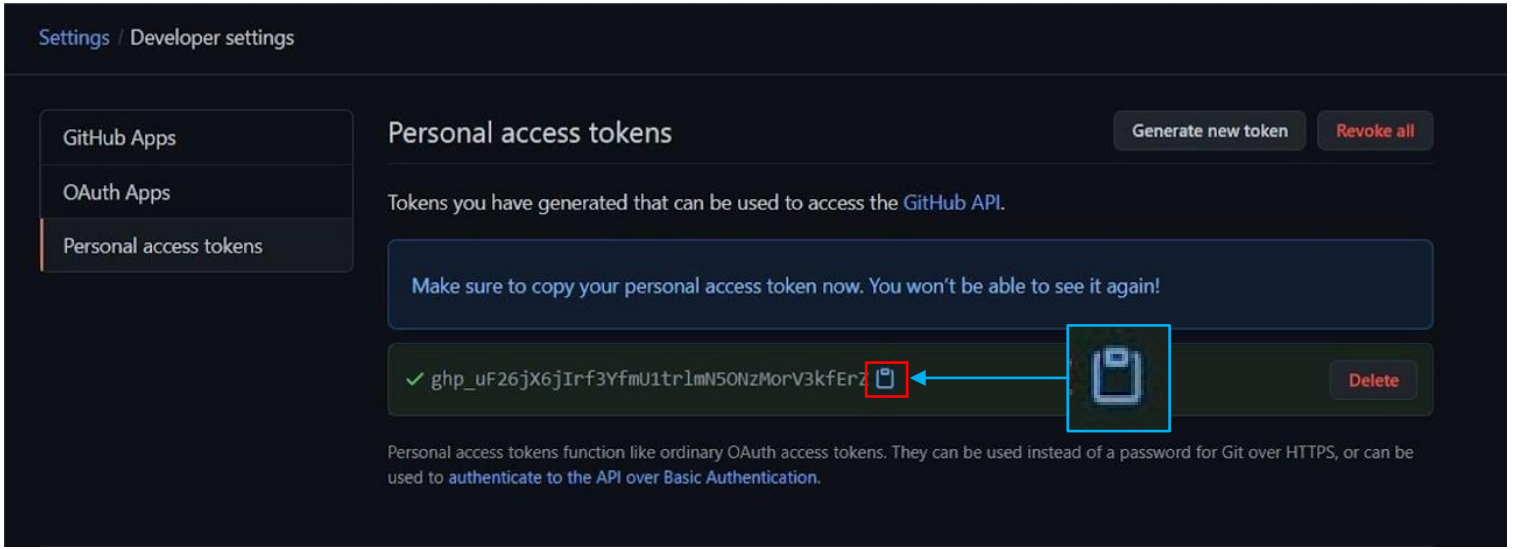


- Nếu bạn chưa biết chi tiết cách sử dụng GitHub, hãy chọn tất cả các checkbox để thoải mái sử dụng token cho bất kỳ mục đích nào.
- Trường Note được sử dụng như một ghi chú ngắn gọn về mục đích phát hành token.
- Đặt ngày hết hạn trong khoảng thời gian thuận tiện cho bạn.
- Sau khi hoàn thành, hãy nhấp vào nút **Generate token** ở dưới cùng.

Thao tác Push My Document

Mã token truy cập cá nhân

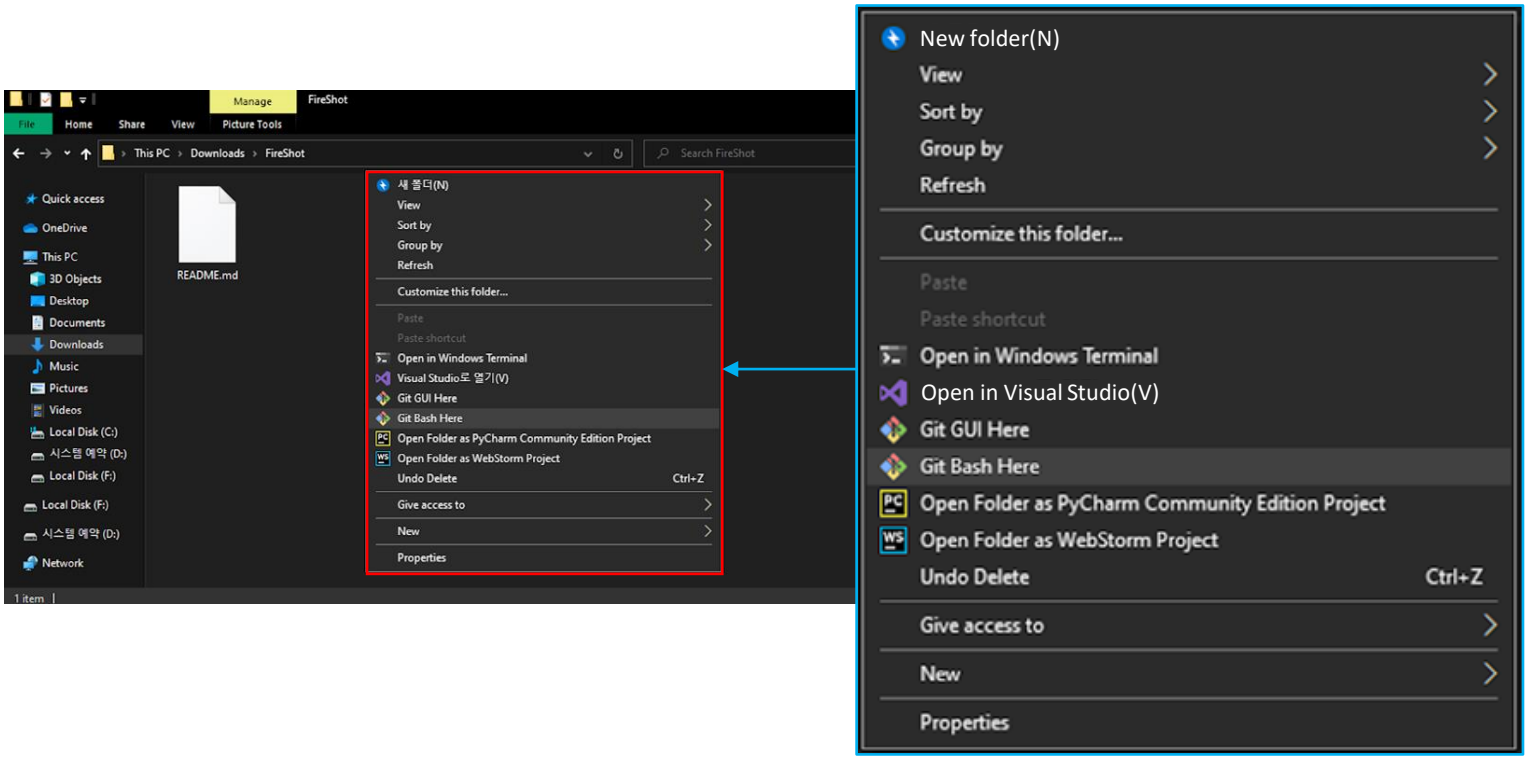
- Kiểm tra và sao chép token của bạn.
- Nhấp vào **biểu tượng đĩa mềm màu xanh** phía bên phải để sao chép.



Thao tác Push My Document

Tạo kho lưu trữ cá nhân

- Trên PC, hãy chuyển đến thư mục bạn muốn kết nối với Kho lưu trữ từ xa GitHub → Nhấp chuột phải → Nhấp vào Git Bash Here.



Thao tác Push My Document

Tạo kho lưu trữ cá nhân

- Lưu văn bản giữa dấu ngoặc kép dưới dạng file có tên README.md trong thư mục hiện tại thông qua lệnh echo. Tạo một thư mục con có tên .git thông qua git init để chứa các file khung xương (Skeleton) cần thiết trong kho lưu trữ.

```
$ git init
```

- Lệnh này tạo một thư mục con mới có tên .git chứa tất cả các file tạo kho lưu trữ cần thiết của bạn - một khung xương kho lưu trữ Git.
  Thêm file README.md vào mục tiêu theo dõi của Git.
  xác nhận các file được theo dõi bằng cách sử dụng văn bản giữa dấu ngoặc kép làm thông báo xác nhận.



Thao tác Push My Document

Tạo kho lưu trữ cá nhân

- Ghi đè tên nhánh bằng main.
  Nó liên kết với Kho lưu trữ từ xa có tên origin và địa chỉ nằm sau origin. Lưu ý rằng địa chỉ tại thời điểm này là file git của Kho lưu trữ GitHub mà bạn sử dụng. Ngoài ra, nếu bạn sao chép và dán địa chỉ trong git bash, lỗi giao thức https không được hỗ trợ có thể xảy ra. Do đó, cần thêm https vào địa chỉ.
  Đẩy từ nhánh main sang origin (Remote Repository).



Thao tác Push My Document

Tạo kho lưu trữ cá nhân

- Kiểm tra các tin nhắn theo thứ tự.
  Thực thi lệnh git push –u origin main. Bạn có thể thấy một cửa sổ yêu cầu bạn đăng nhập vào GitHub, sau đó đăng nhập bằng tài khoản của bạn.

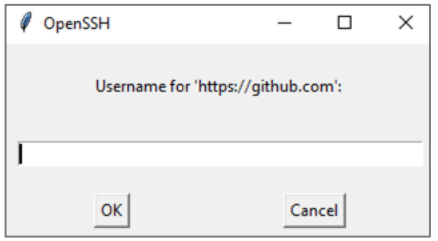
```
$ git push –u origin main
```

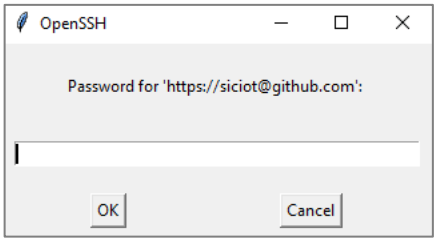


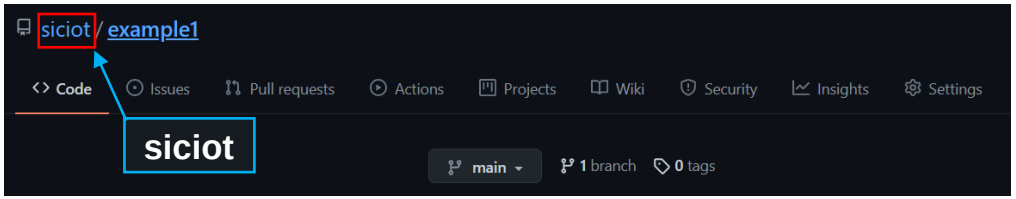
Thao tác Push My Document

Tạo kho lưu trữ cá nhân

- Sau khi đăng nhập, một cửa sổ mới hiện lên với hai câu hỏi. Khi được yêu cầu nhập tên người dùng của bạn, bạn có thể tìm thấy tên người dùng của mình được đánh dấu bằng hộp màu đỏ trong Kho lưu trữ từ xa GitHub của bạn.
- Sau đó, bạn sẽ được yêu cầu nhập mật khẩu. Dán mã thông báo đã phát hành trước đó và nhập mã thông báo đó.
- Nếu push thành công, bạn có thể check nó trong git bash.







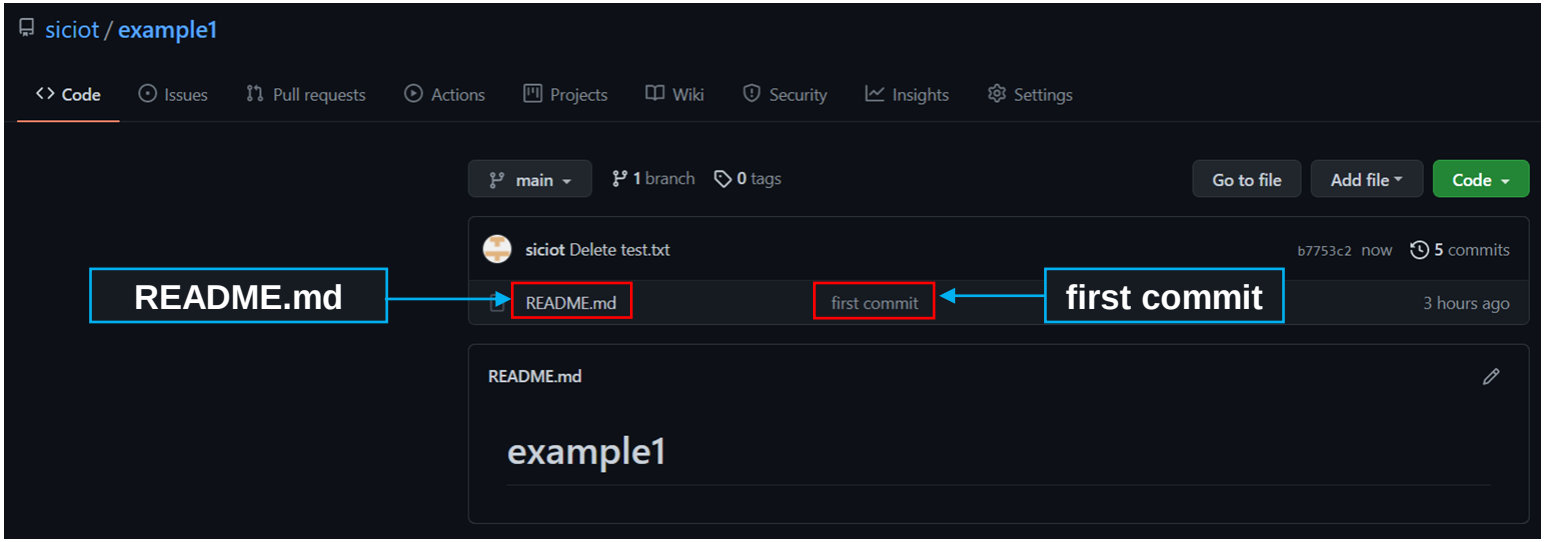

```
donqh@DESKTOP-K0OCA2A MINGW64 ~/Downloads/FireShot (main)
$ git push -u origin main
Logon failed, use ctrl+c to cancel basic credential prompt.
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 227 bytes | 227.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To https://github.com/siciot/example1.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

$ git push –u origin main

Thao tác Push My Document

Tạo kho lưu trữ cá nhân

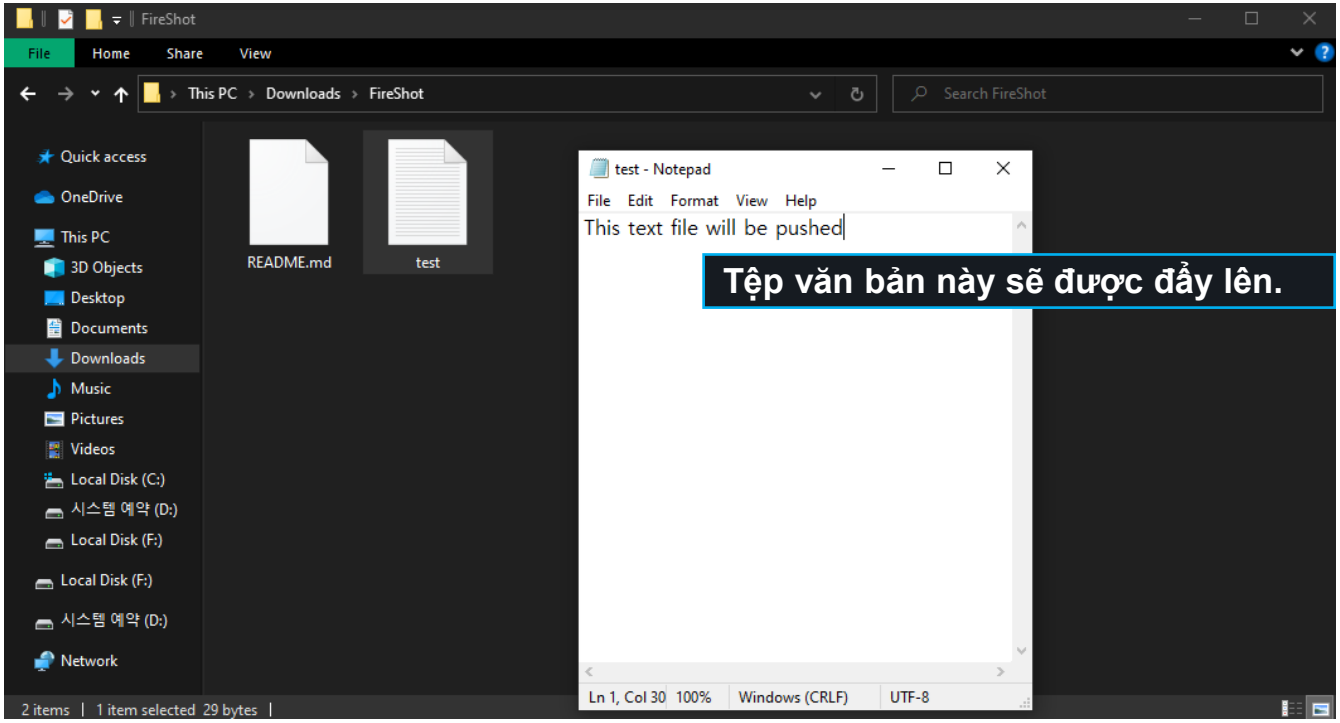
- Hãy truy cập vào kho lưu trữ GitHub để kiểm tra xem thao tác push đã được thực hiện thành công hay chưa. Nếu thành công, bạn có thể kiểm tra các tệp đã được tải lên bằng lệnh git add từ kho cục bộ (Local Repository) lên kho từ xa (Remote Repository) của mình.
- Bạn cũng có thể kiểm tra thông điệp commit mới nhất của tệp, cũng như các tệp hoặc thư mục trong kho từ xa.



Nếu thêm file

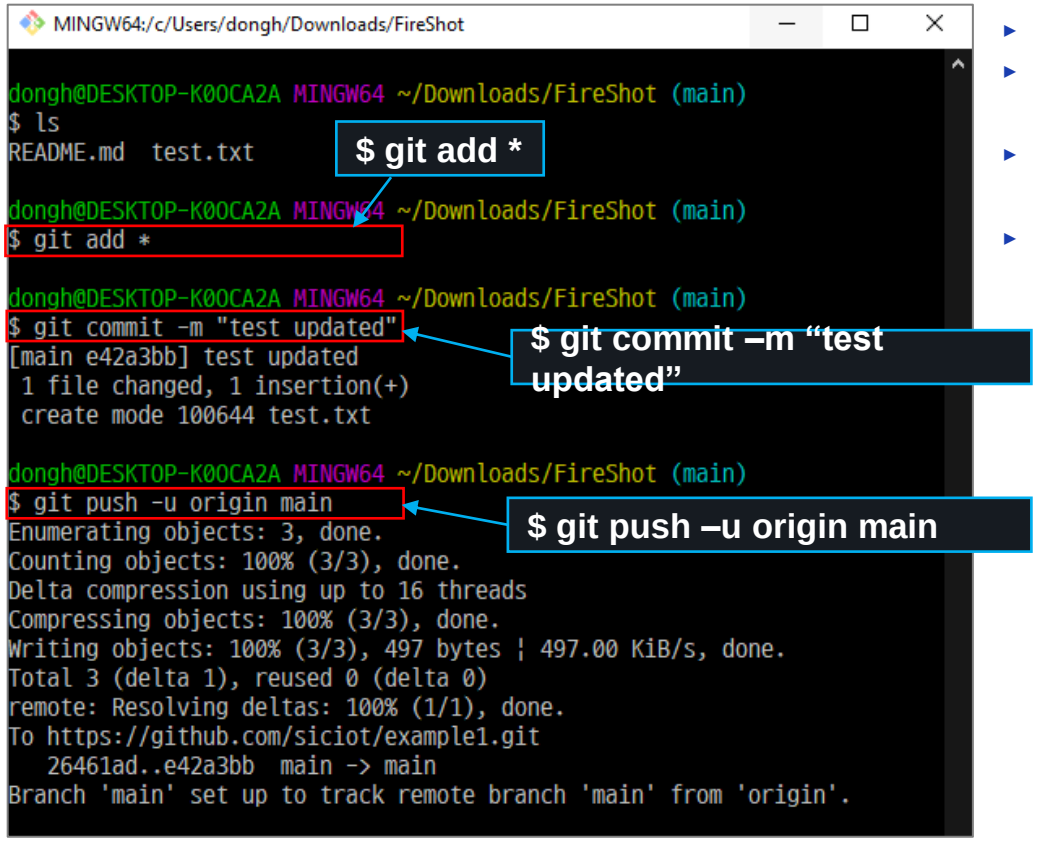
Lưu kết quả đã chỉnh sửa

- Tạo test.txt ở cùng vị trí với README.md trong thư mục làm việc.



Nếu thêm file

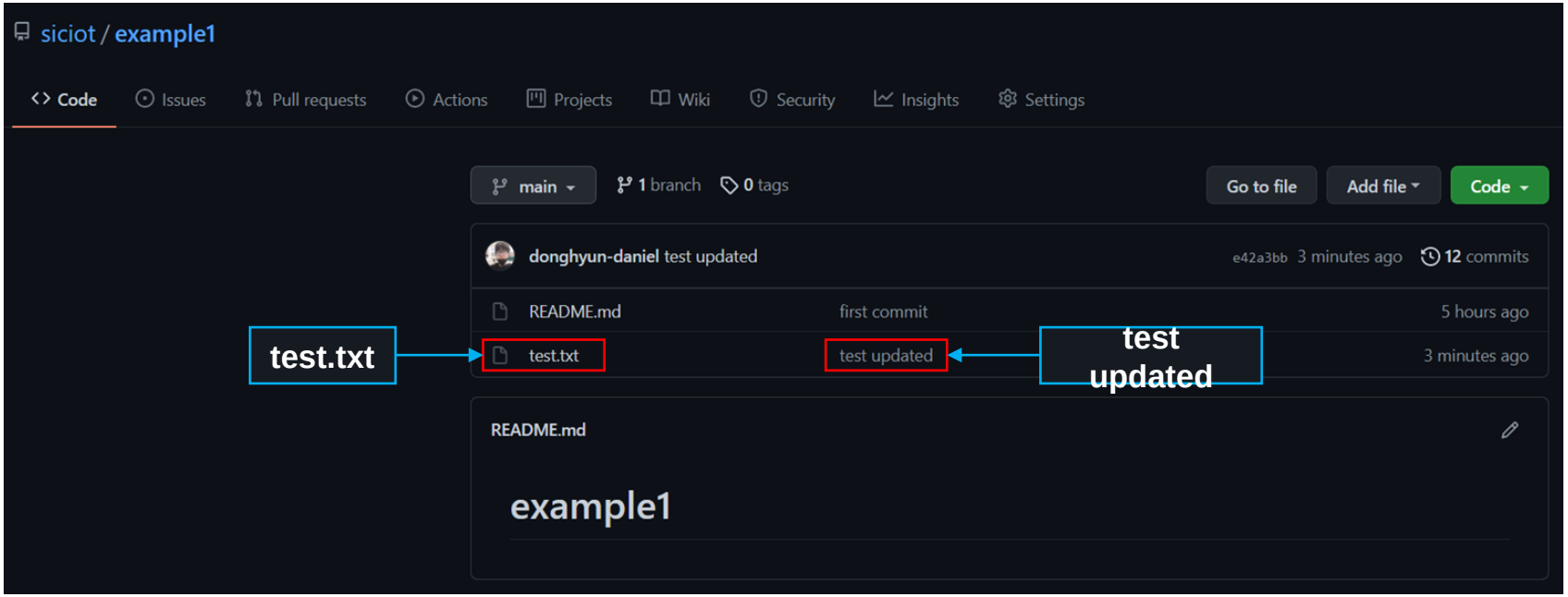
Lưu kết quả đã chỉnh sửa



- Kiểm tra lại file đã tạo.
- Thêm (*) vào tất cả các file (test.txt) trong thư mục hiện tại vào git cần theo dõi.
- Xác nhận các mục tiêu theo dõi với thông báo xác nhận " test updated ".
- Đẩy đến nhánh chính của origin.

Nếu thêm file

- Lưu kết quả đã chỉnh sửa
  - Kiểm tra văn bản đã thêm.txt trong Remote Repository (thử bấm F5 nếu bạn không thấy nó).



Bài 4.

Cách sử dụng GitHub

4.1. Bài luyện tập GitHub đơn giản

4.2. Cách sử dụng GitHub khi làm việc theo cá nhân

4.3. Cách sử dụng GitHub thông qua môi trường GUI

Nếu làm việc trong hệ điều hành khác

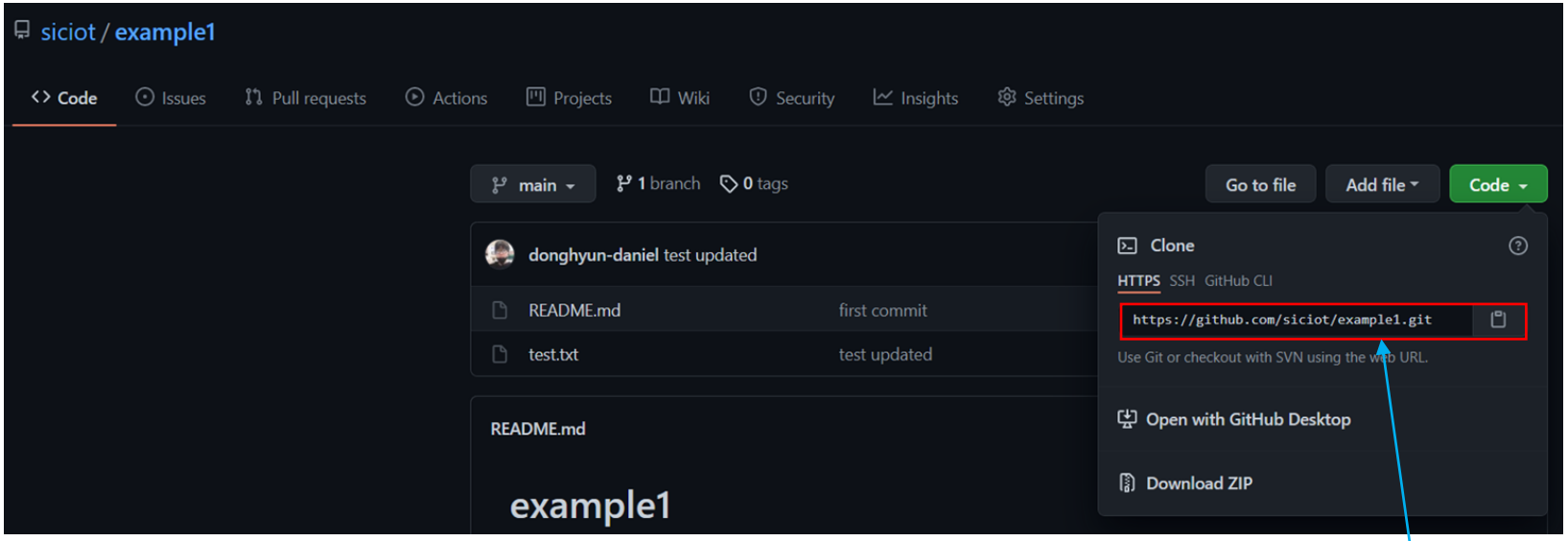
Sử dụng GitHub trong Linux

- Bạn đã tìm hiểu về cách tạo Kho lưu trữ trên GitHub bằng cách sử dụng Sửa đổi Window 10, Commit, Push và Push.
- GitHub cũng có thể được sử dụng trong Linux (Raspbian OS) theo cách tương tự như Windows.
- Bạn sẽ tìm hiểu trường hợp bạn đẩy nội dung bạn đã làm việc trên Windows vào Kho lưu trữ GitHub và bạn làm việc với nội dung trên PC thứ hai trong Linux.

Nếu cần nhân bản dự án (1/3)

Sử dụng GitHub trong Linux

- Lấy địa chỉ của file git trong Kho lưu trữ GitHub chứa dự án hiện có của tôi sẽ được nhập.

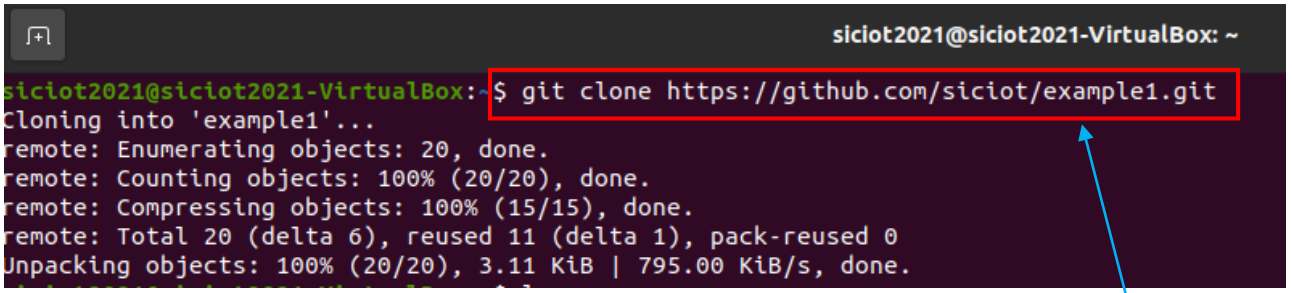


https://github.com/siciot/example.git

Nếu cần nhân bản dự án (2/3)

Sử dụng GitHub trong Linux

- Nhập một dự án hiện có bằng cách sử dụng lệnh git clone trong Linux Terminal. Đặt địa chỉ trước đó sau khi sao chép git.

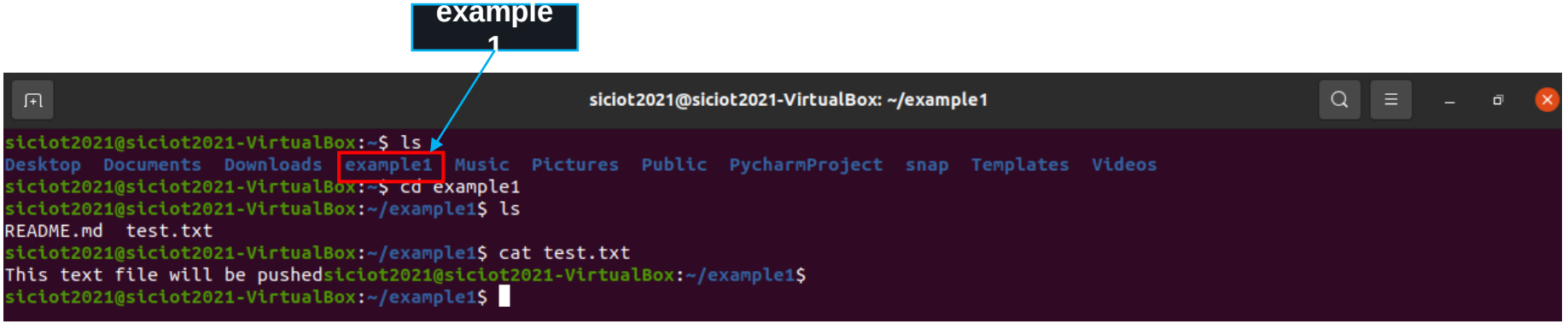


~$ git clone https://github.com/siciot/example1.git

Nếu cần nhân bản dự án (3/3)

Sử dụng GitHub trong Linux

- Kiểm tra danh sách bằng ls. Bạn có thể thấy rằng thư mục có tên example1 đã được tạo.
- Di chuyển và kiểm tra nội dung. Bạn có thể thấy rằng ngay cả nội dung của test.txt được tạo trước đó cũng giống nhau.
- Khi dự án hiện có được lưu trữ trong Kho lưu trữ GitHub, bạn có thể dễ dàng nhập nó bất kể hệ điều hành nào.



Nếu cần chỉnh sửa file trên Linux (1/6)

Sử dụng GitHub trong Linux

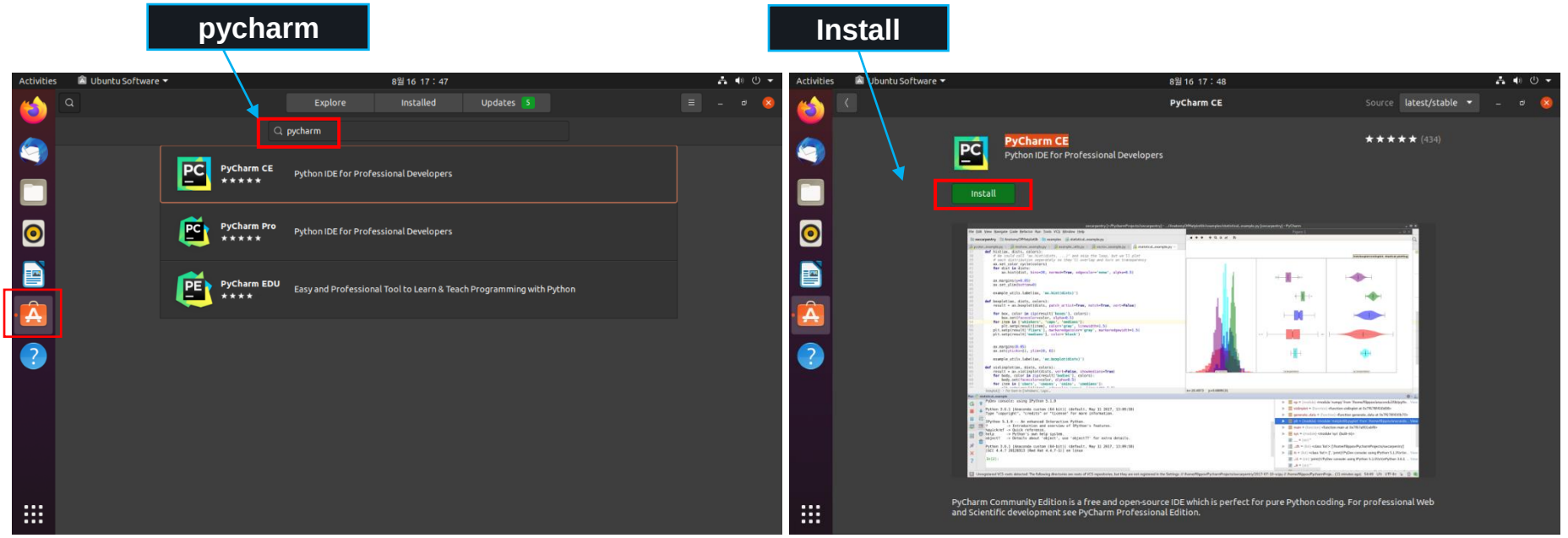
- Sử dụng Pycharm nếu bạn không quen thuộc với trình soạn thảo Linux cơ bản.
- Pycharm là một IDE rất phổ biến có thể chỉnh sửa và kiểm soát các file thuộc nhiều loại khác nhau cũng như Python.
- Trong Pycharm, có thể chọn phiên bản Python cho mỗi mã và nó hỗ trợ cài đặt gói dễ dàng.
- Trong Pycharm, việc thực thi mã rất đơn giản, quản lý hàm dễ dàng và các chức năng cho môi trường ảo được hỗ trợ như gốc.
- Pycharm luôn là một IDE được xếp hạng cao vì nhiều ưu điểm của nó, bao gồm nhiều plugin khác nhau.



Nếu cần chỉnh sửa file trên Linux (2/6)

Sử dụng GitHub trong Linux

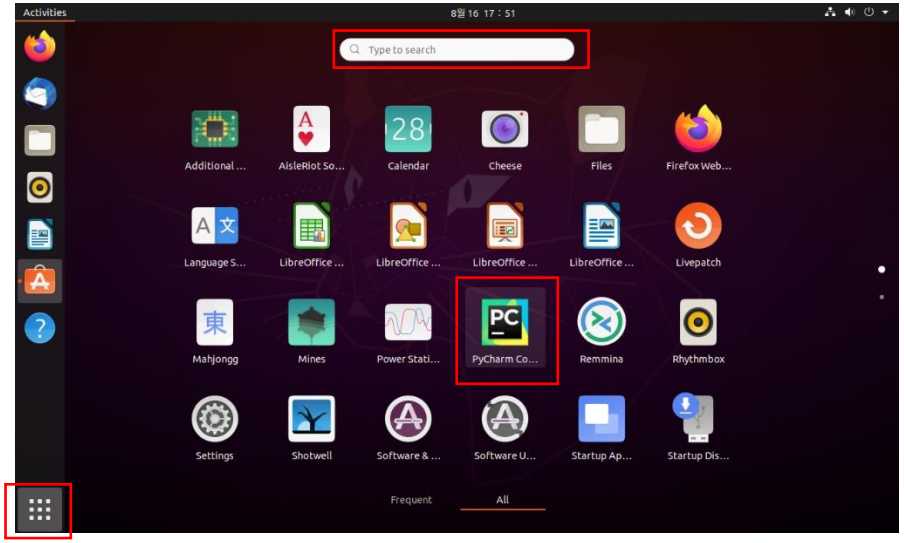
- Cài đặt Pycharm rất đơn giản.
- Chạy Ubuntu Software, tìm kiếm Pycharm và nhấp vào nút Install để cài đặt nó.



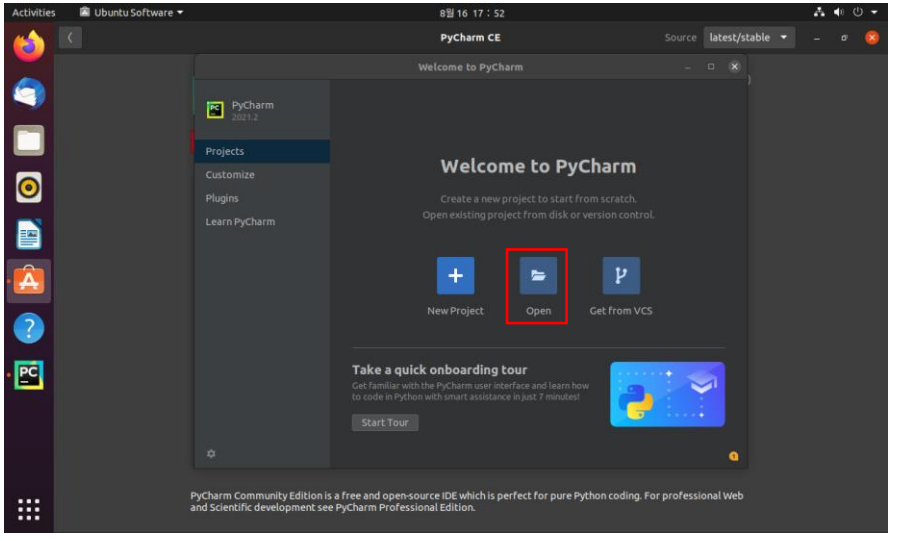
Nếu cần chỉnh sửa file trên Linux (3/6)

Sử dụng GitHub trong Linux

- Nhấp vào Show Applications in góc dưới bên trái và nhấp vào Pycharm. Nếu bạn không thấy biểu tượng này, hãy tìm kiếm Pycharm.



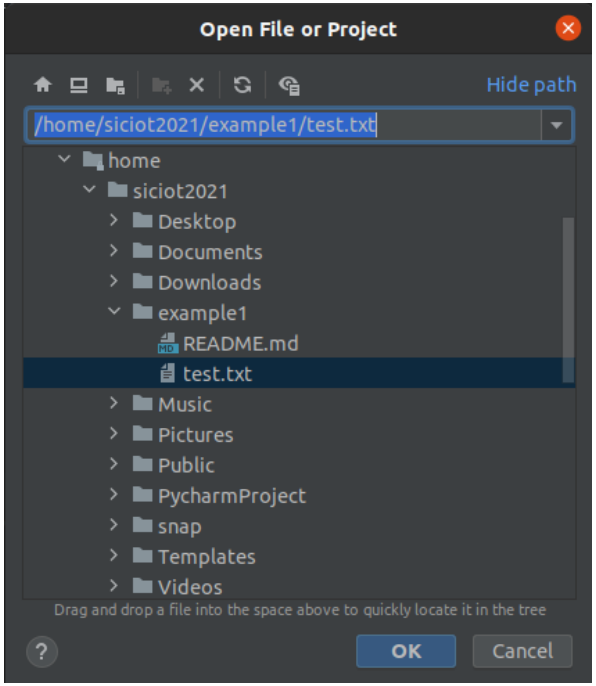
- Nhấp vào Open để duyệt file.

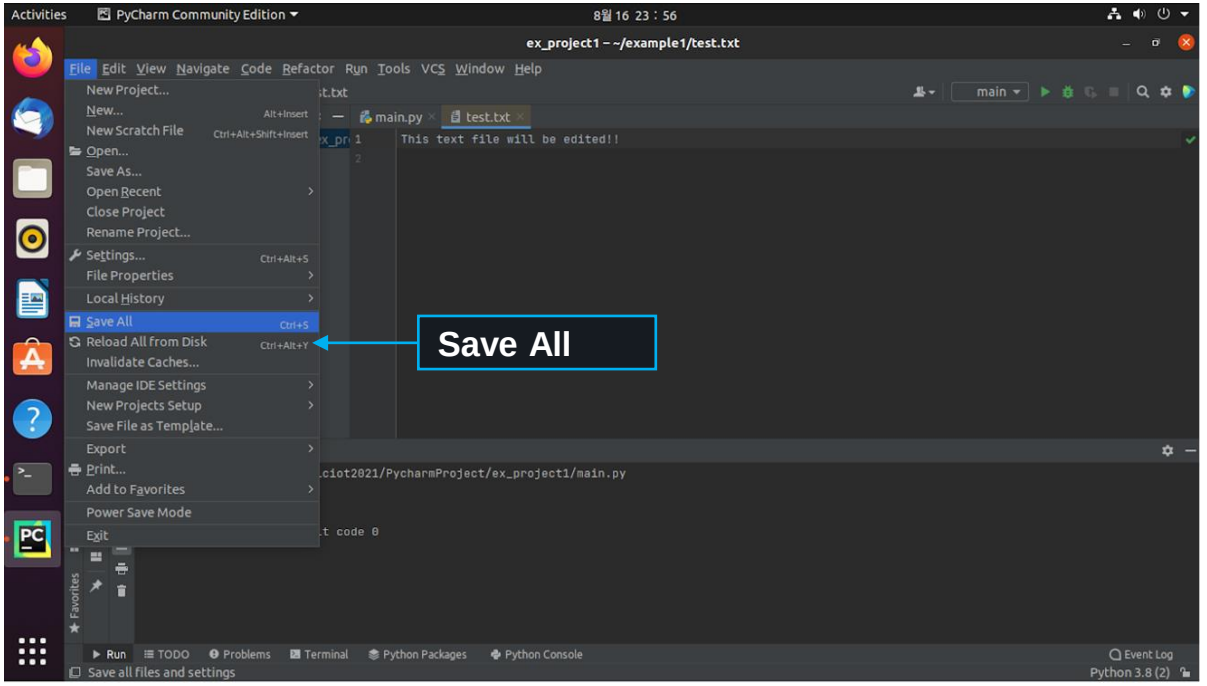


Nếu cần chỉnh sửa file trên Linux (4/6)

Sử dụng GitHub trong Linux

- Tiến hành chỉnh sửa nội dung của file test.txt thuộc example1 được nhân bản trước đó.
- Sau khi chỉnh sửa nội dung, nhấp vào Save All để lưu vào thoát chương trình.

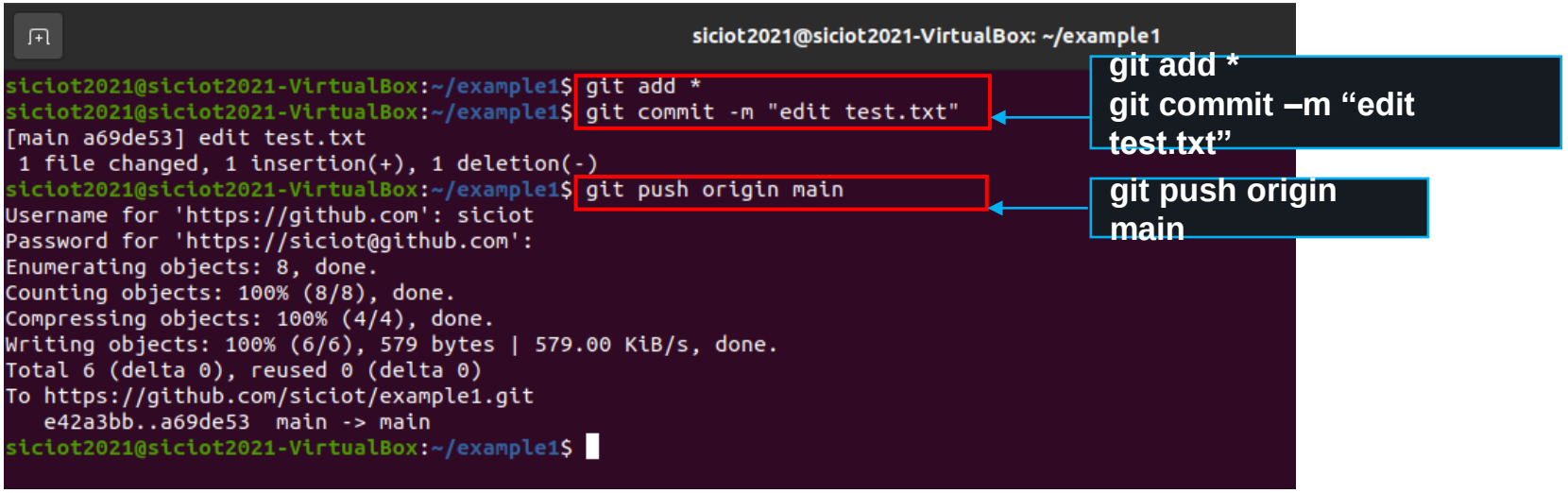




Nếu cần chỉnh sửa file trên Linux (5/6)

Sử dụng GitHub trong Linux

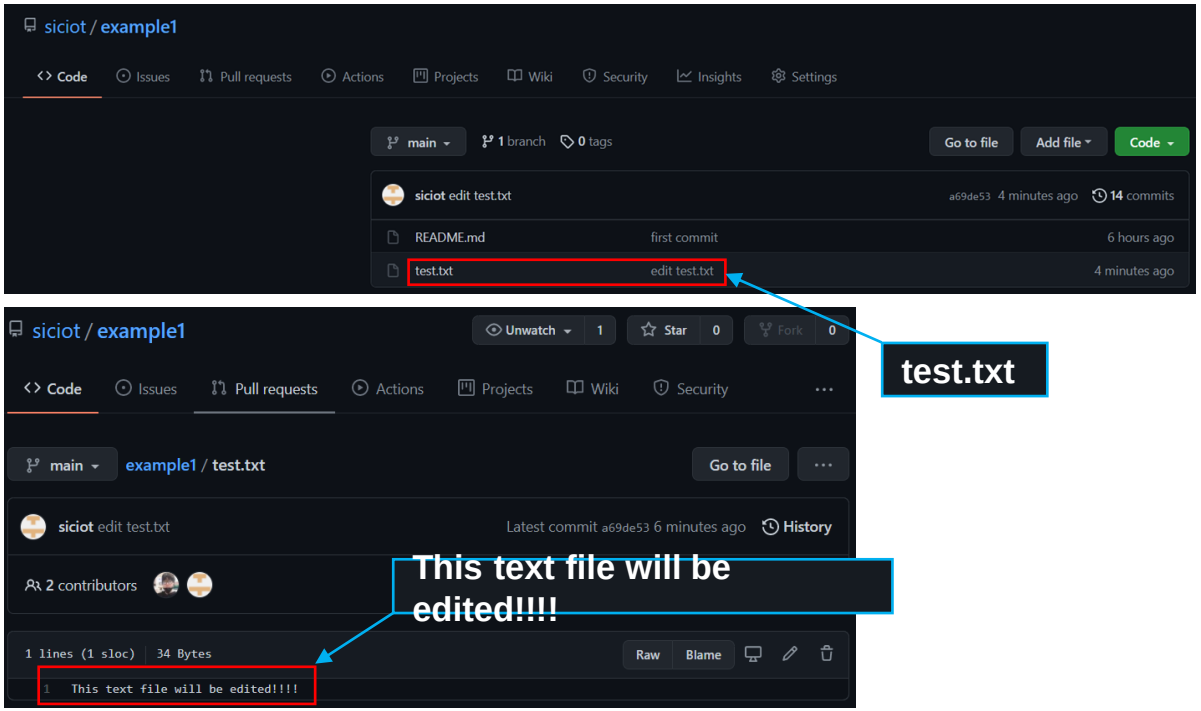
- Thêm vào mục tiêu cần theo dõi trong cùng một luồng xử lý tương tự trong Window.
- Thực hiện commit với nội dung mô tả
- Thực hiện thao tác Push tới nhánh main của origin.



Nếu cần chỉnh sửa file trên Linux (6/6)

Sử dụng GitHub trong Linux

- Kiểm tra kho lưu trữ từ xa GitHub. Xác nhận rằng tệp test.txt đã được chỉnh sửa và ánh xạ lại cùng với một thông báo xác nhận kiểm tra chỉnh sửa.txt.



Tổng kết

Nếu cần lưu một dự án hiện có trên PC cục bộ của bạn vào Kho lưu trữ GitHub, hãy tiến hành như sau

- Chuẩn bị thư mục project để sử dụng git → git init

```
$ git init
```

- Gộp nội dung của thư mục dự án trong đối tượng theo dõi của git → git add * (Nếu bạn chỉ muốn gộp một số tệp vào trong đối theo dõi, hãy nhập tên cụ thể thay vì *)

```
$ git add *
```

- Thực hiện lệnh commit để ghi lại các thay đổi trong đối tượng theo dõi → git commit.

```
$ git commit
```

- Thực hiện lệnh Push để ánh xạ các thay đổi với Kho lưu trữ từ xa được ghi lại → git push.

```
$ git push
```

Tổng kết

- Nếu muốn gọi tới và thao tác với project nằm trong GitHub Repository về PC cục bộ, thực hiện thao tác nhập (import) sử dụng địa chỉ .git của GitHub Repository → git clone.

```
$ git clone
```

- Mặc dù phương pháp xử lý file có điểm khác nhau, hai HĐH Linux và Windows không có nhiều điểm khác nhau nhiều khi đề cập tới quy trình sử dụng Git.

Bài 4.

Cách sử dụng GitHub

4.1. Bài luyện tập GitHub đơn giản

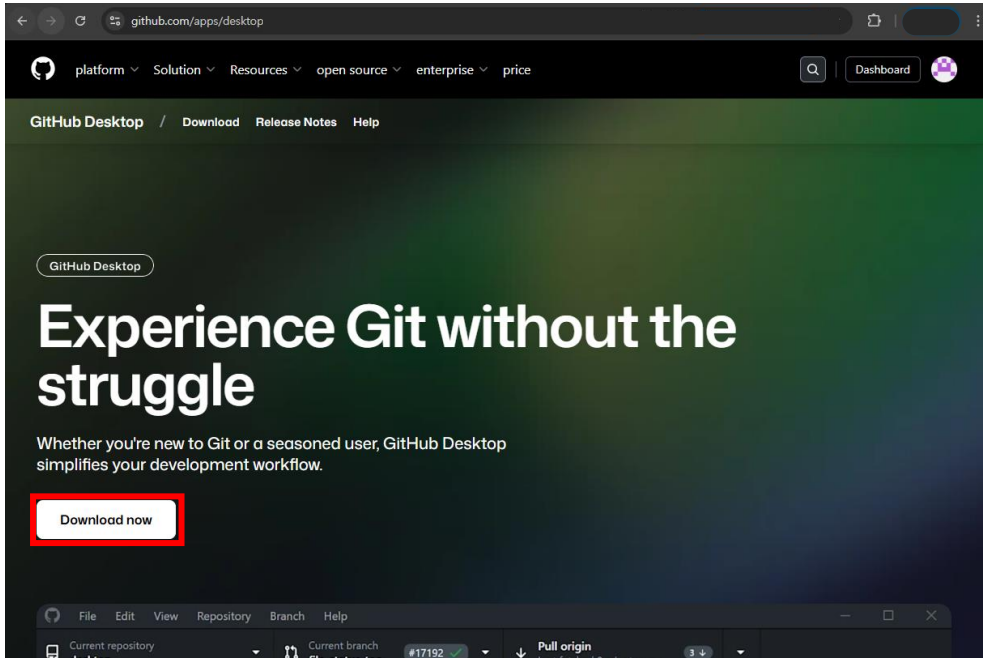
4.2. Cách sử dụng GitHub khi làm việc theo cá nhân

4.3. Cách sử dụng GitHub thông qua môi trường GUI

Hướng dẫn sử dụng GUI

Phần mềm mã nguồn mở hỗ trợ GUI sử dụng GitHub

- Giới thiệu GitHub Desktop – phần mềm mã nguồn mở cung cấp môi trường giao diện đồ họa (GUI) dành cho những người chưa quen với môi trường dòng lệnh (CLI – Command Line Interface)..
- GitHub Desktop cung cấp môi trường GUI dễ sử dụng và trực quan hơn so với môi trường CLI.

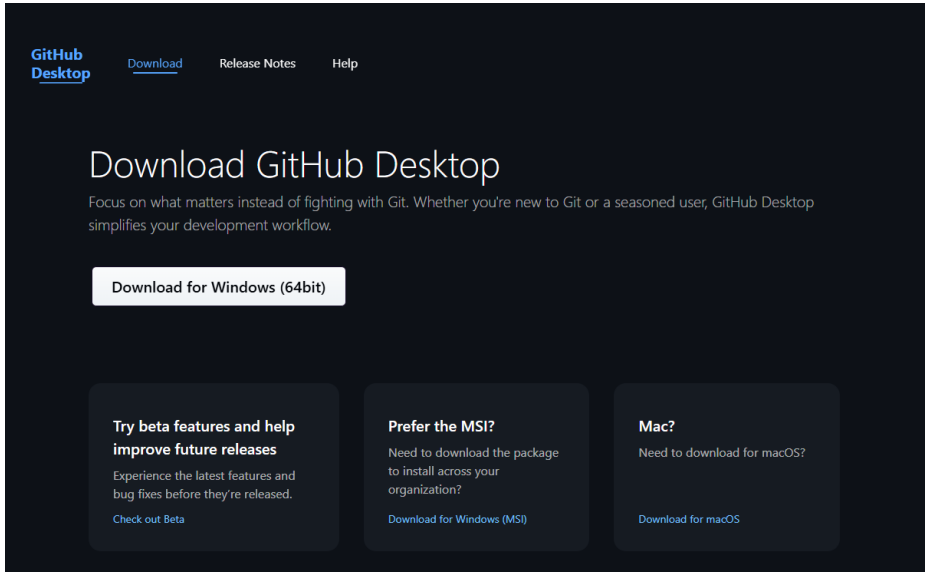


https://desktop.github.com/

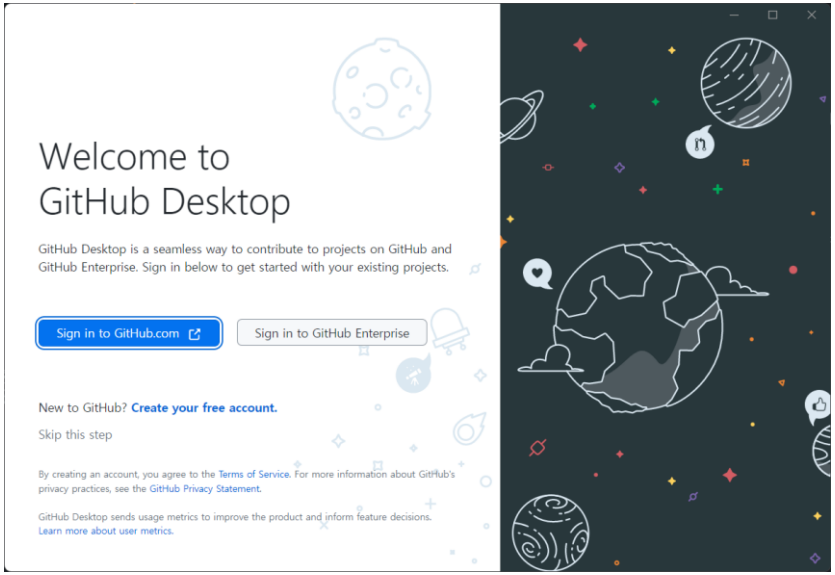
GitHub Desktop (1/11)

Cài đặt GitHub Desktop

- Sau khi tải xuống, hãy chạy tệp .exe phù hợp với hệ điều hành và môi trường của bạn.



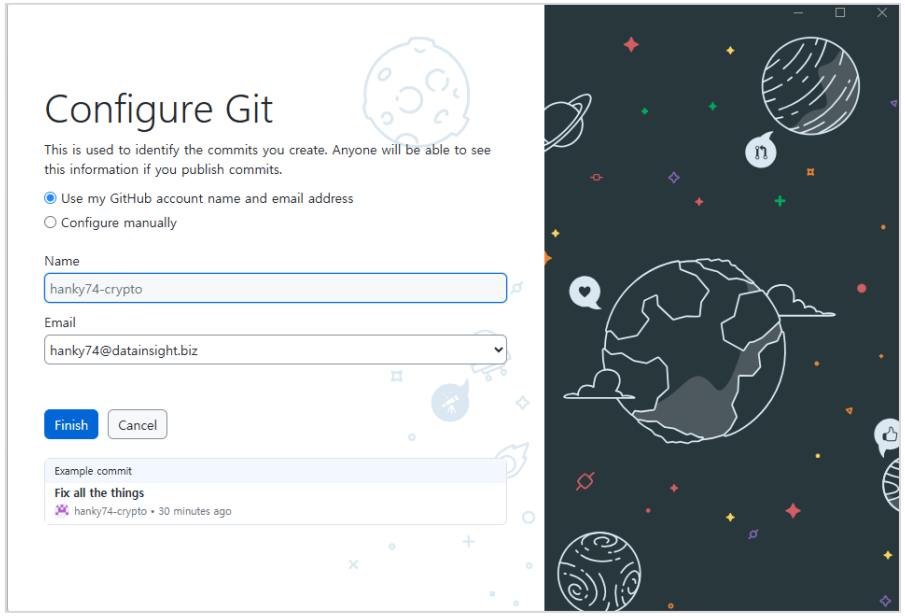
- Đăng nhập bằng tài khoản GitHub của bạn.



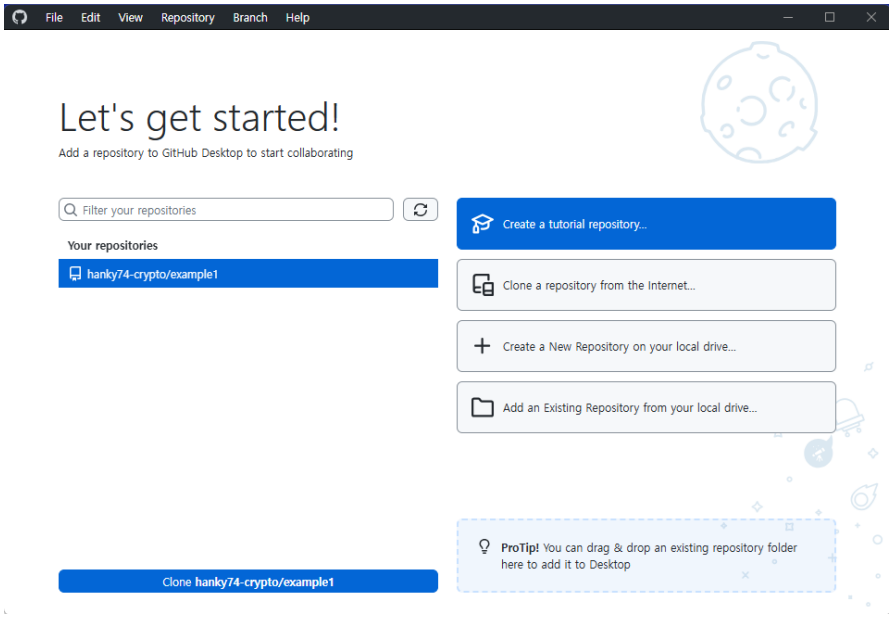
GitHub Desktop (2/11)

Cài đặt GitHub Desktop

- Đặt tên và địa chỉ e-mail cho commit và nhấn nút Continue (chức năng tương tự như git config... trong môi trường CLI).

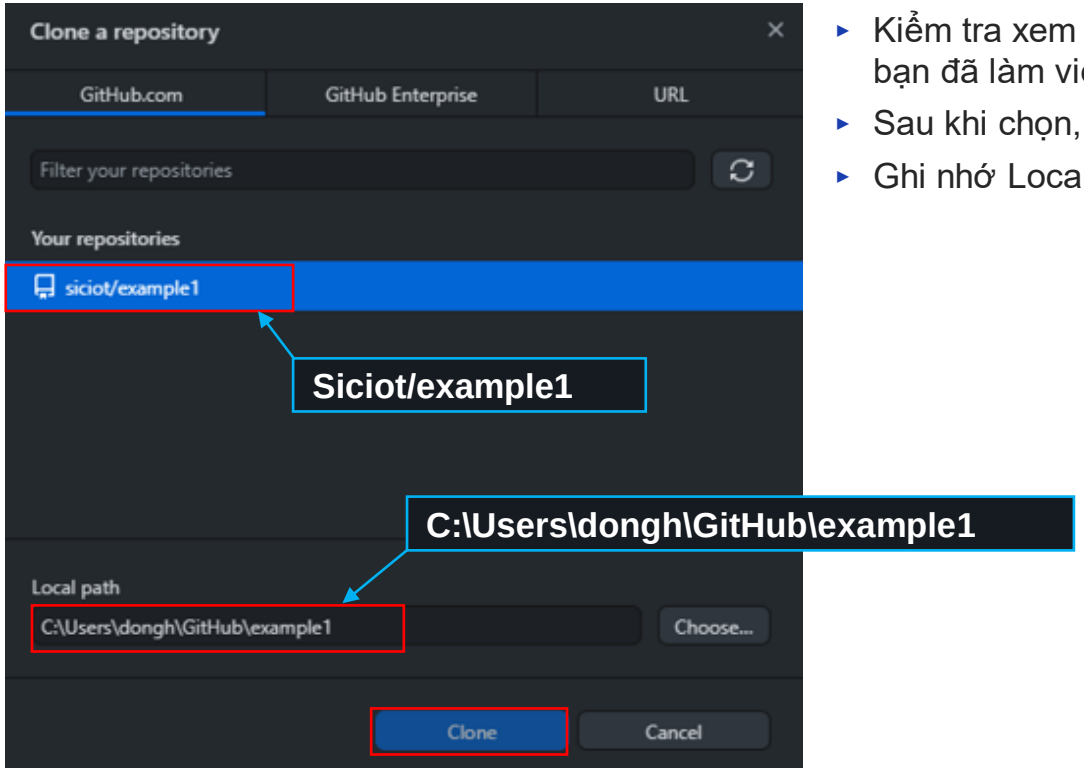


- Sau quá trình xác thực, hãy nhấp vào **clone a repository from the Internet…**



GitHub Desktop (3/11)

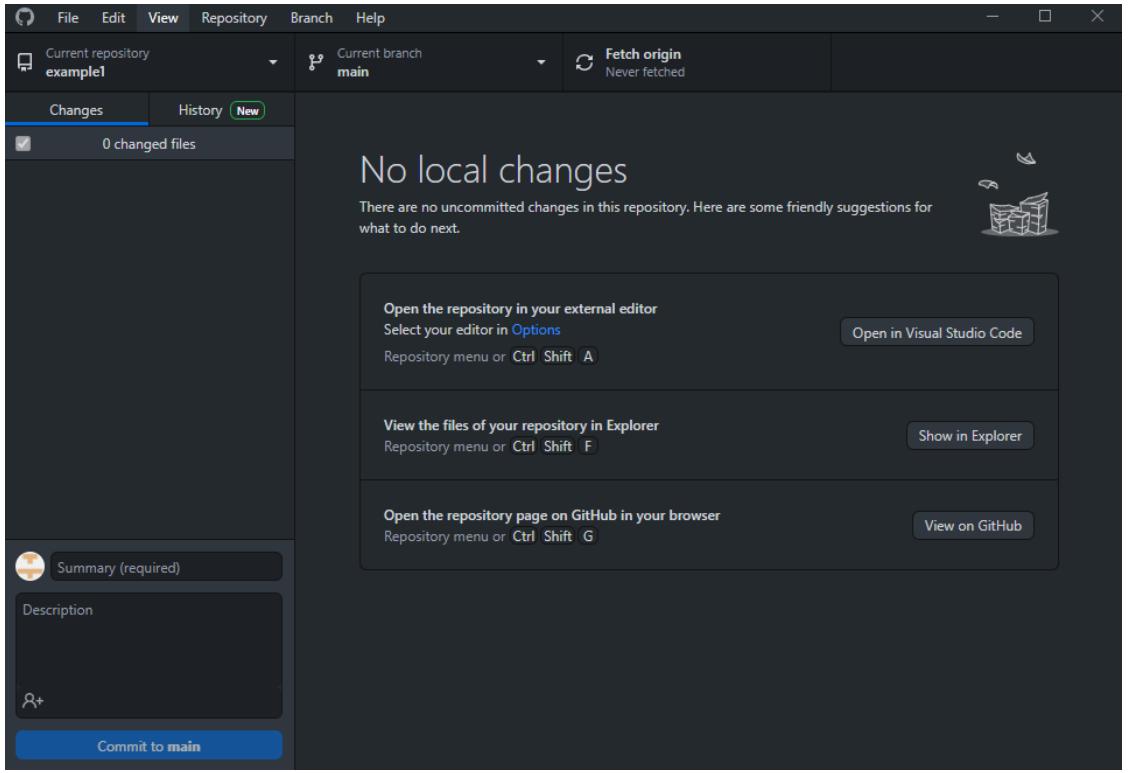
Sử dụng GitHub Desktop



- Kiểm tra xem bạn có thể tìm thấy Kho lưu trữ example1 mà bạn đã làm việc trước đó không.
- Sau khi chọn, hãy nhấp vào **Clone** ở dưới cùng.
- Ghi nhớ Local Path (đường dẫn tới thư mục chứa).

GitHub Desktop (4/11)

Sử dụng GitHub Desktop

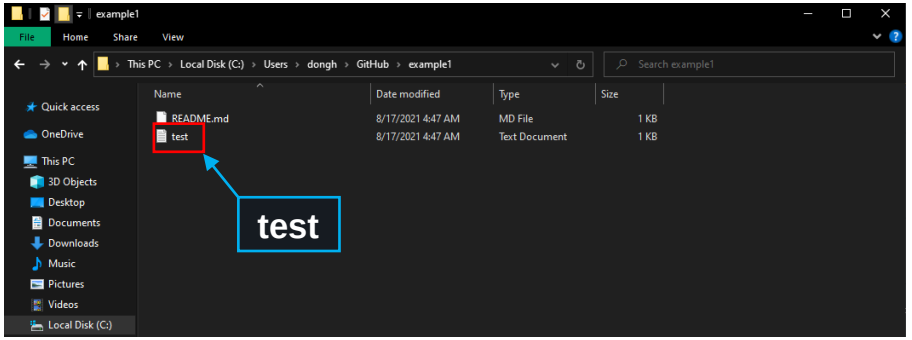


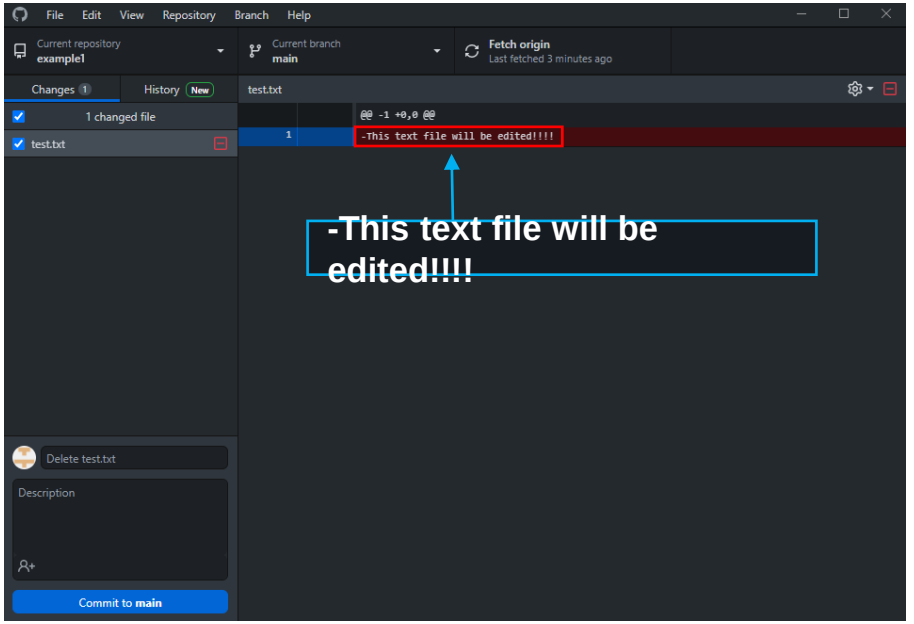
- Truy cập vào bản sao của example1.
- Chỉnh sửa kho lưu trữ cục bộ trong môi trường Windows để kiểm tra xem nó có được quản lý tốt ngay cả khi không có dòng lệnh cho git hay không.

GitHub Desktop (5/11)

Sử dụng GitHub Desktop

- Xóa tệp test.txt trong thư mục example1 ở đường dẫn cục bộ
- Khi quay lại GitHub Desktop, bạn sẽ thấy hệ thống đã nhận diện được việc thay đổi (xóa file test.txt). Đồng thời, nội dung commit message bạn nhập cũng sẽ được hiển thị.

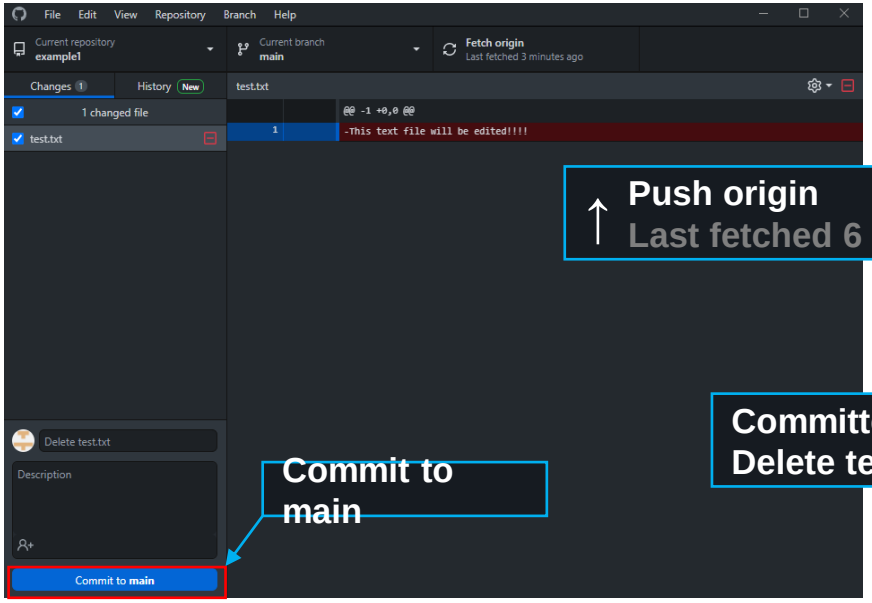




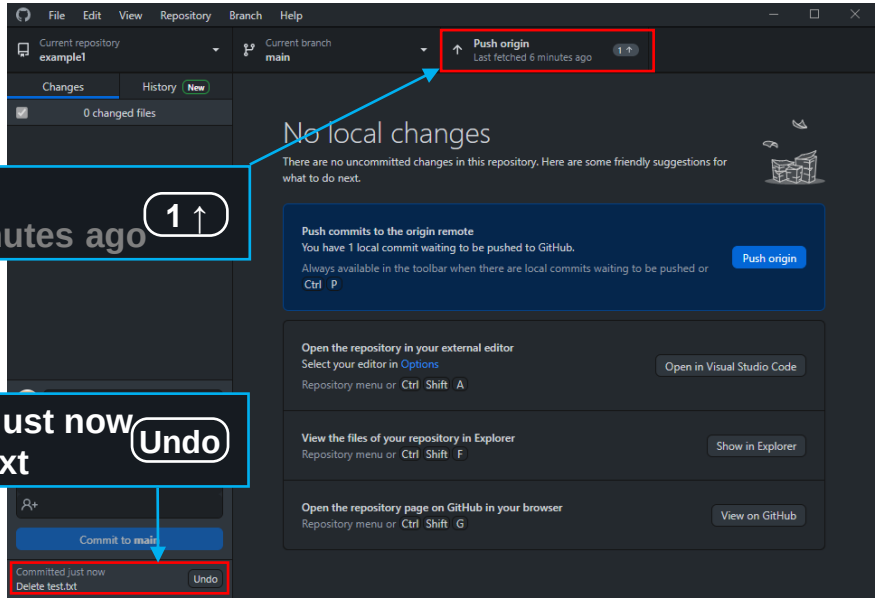
GitHub Desktop (6/11)

Sử dụng GitHub Desktop

- Nếu bạn muốn đẩy trạng thái đã xóa này vào Kho lưu trữ GitHub, hãy nhấp vào **Commit to main** vì bạn cần commit trước.



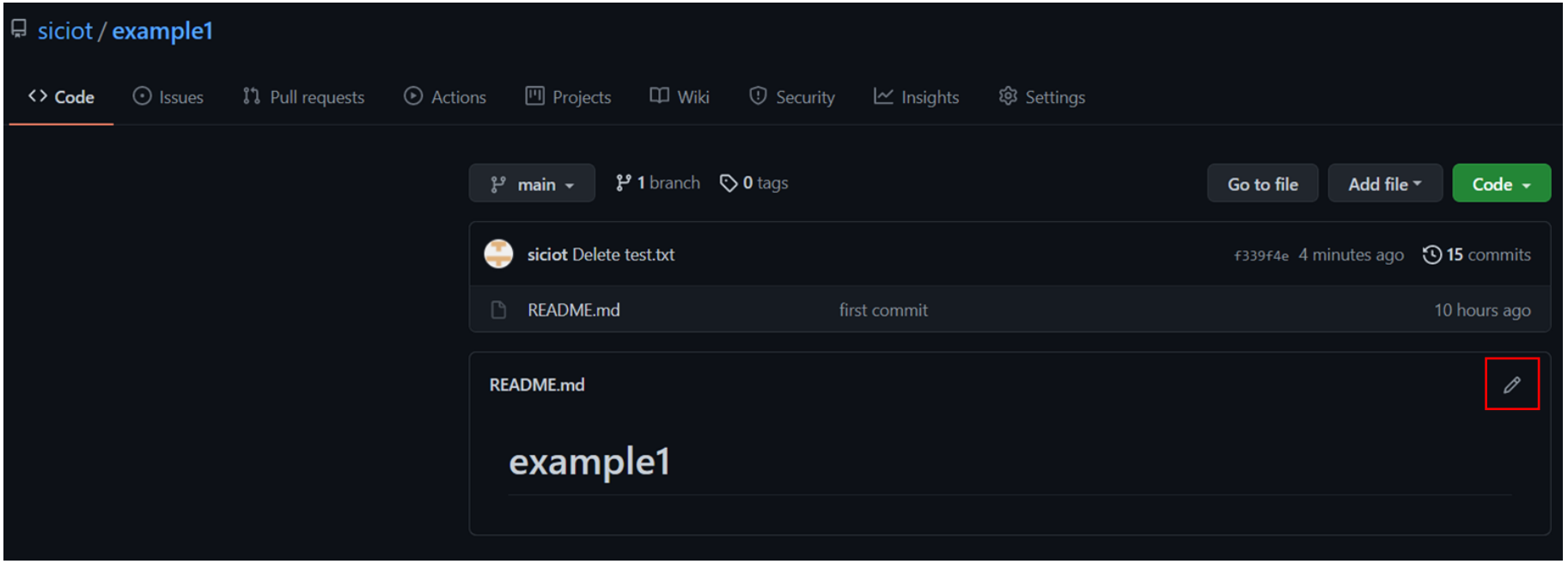
- Kiểm tra nút ở trên đã thay đổi thành **Push origin**, và thông báo Committed lúc này ở dưới cùng bên trái.
- Nhấp vào biểu tượng **Push origin**.



GitHub Desktop (7/11)

Sử dụng GitHub Desktop

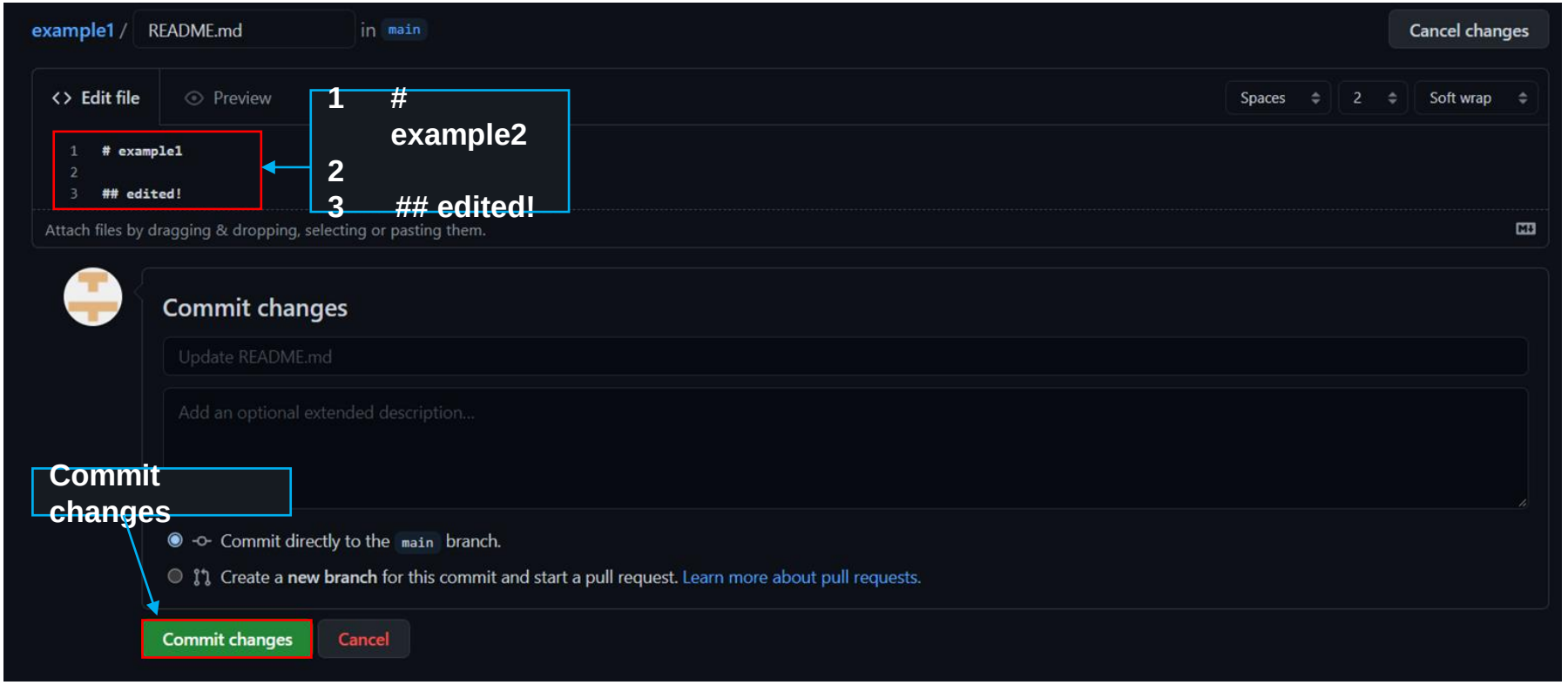
- Kiểm tra GitHub Repository để xem thao tác push có thành công không.
- Kiểm tra file test.txt đã bị xóa và ánh xạ sang chưa.
- Lần này thao tác ngược lại, nhấp vào **pen icon** để chỉnh sửa nội dung của Kho lưu trữ GitHub và kiểm tra trên máy cục bộ.



GitHub Desktop (8/11)

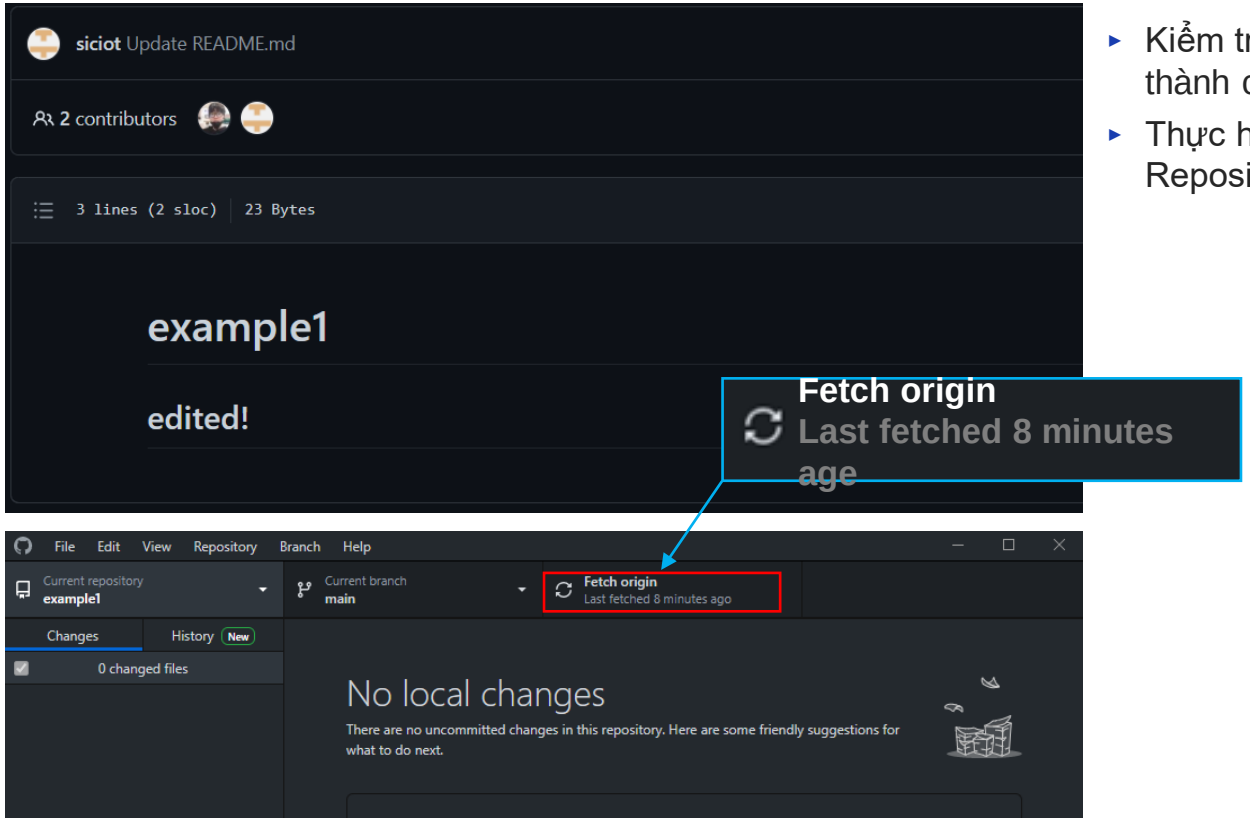
Sử dụng GitHub Desktop

- Chọn commit changes sau khi chỉnh sửa file README.md



GitHub Desktop (9/11)

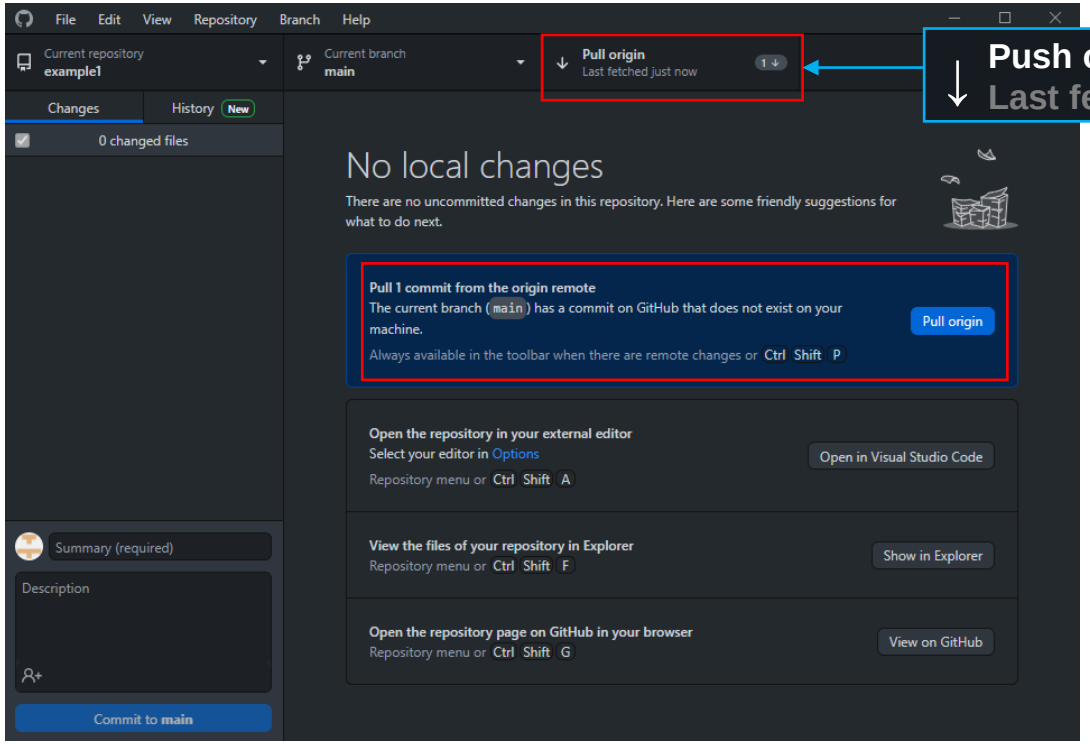
Sử dụng GitHub Desktop

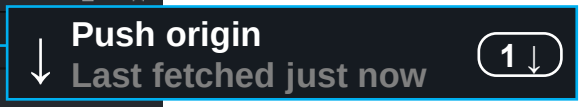


- Kiểm tra xem nội dung đã được cập nhật thành công trên GitHub Repository.
- Thực hiện fetch để kiểm tra thay đổi, vì Local Repository sẽ không tự động được cập nhật.

GitHub Desktop (10/11)

Sử dụng GitHub Desktop



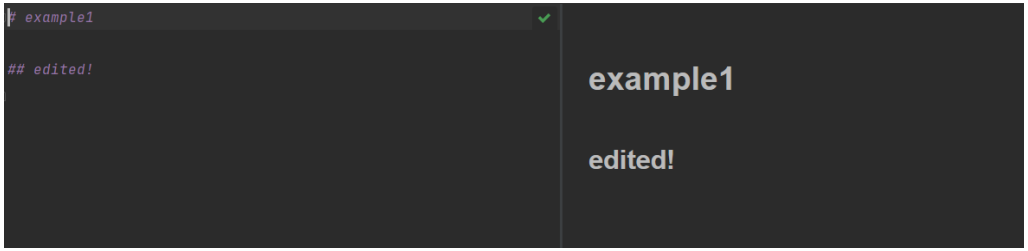


- Sau khi fetch, hệ thống sẽ thông báo rằng có thay đổi trên Origin.
- Bạn vẫn sẽ thấy thông báo No local changes. Để cập nhật file README.md đã thay đổi về Local Repository, bạn cần thực hiện Pull.
- Nhấp vào nút **Pull origin**.

GitHub Desktop (11/11)

Sử dụng GitHub Desktop

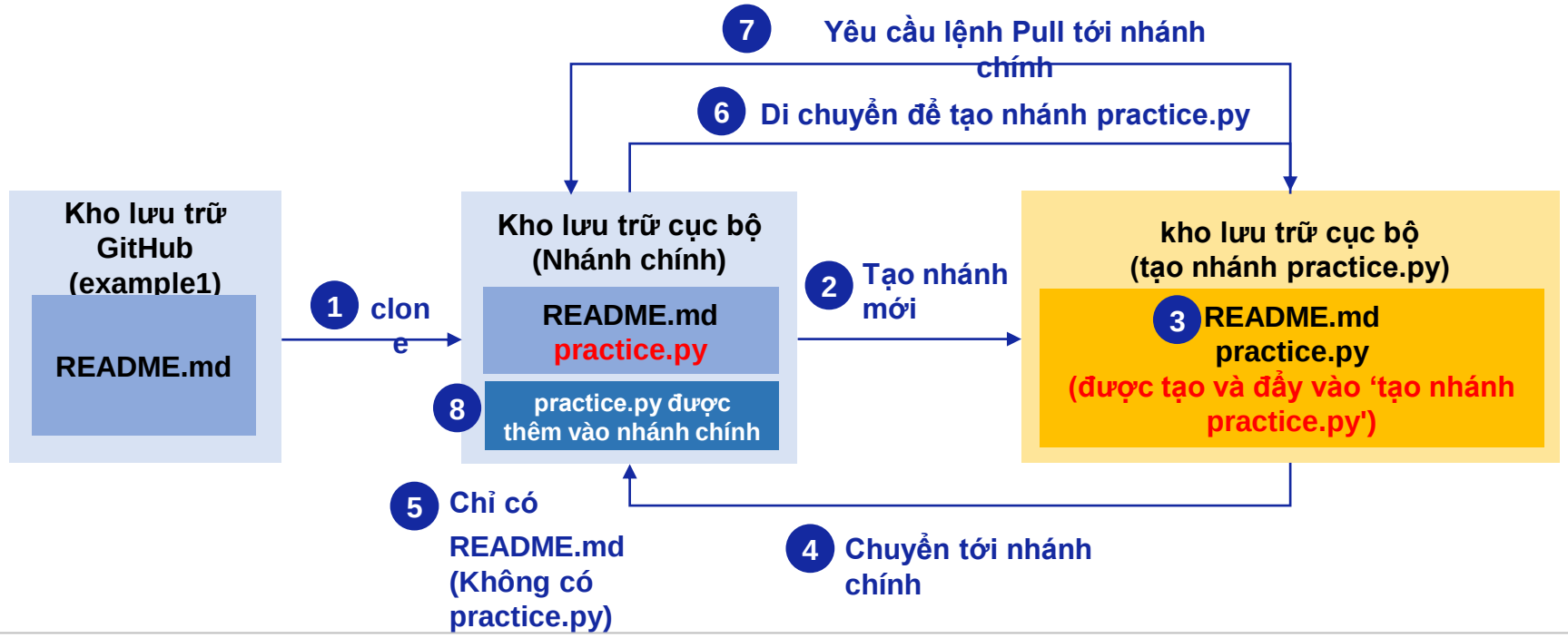
- Mở Local Repository để kiểm tra file markdown. Nếu pull thành công, bạn sẽ thấy các thay đổi đã được cập nhật đầy đủ.
- Bạn đã học cách sử dụng GitHub Desktop một cách trực quan, dễ dàng và trải nghiệm chức năng pull.



Sử dụng Nhánh trong GitHub Desktop (Linux)

Cài đặt GitHub Desktop trên Linux

- Sử dụng GitHub Desktop và Nhánh (Branch) trong môi trường Linux.
- Sử dụng Nhánh để làm việc độc lập và quy trình sáp nhập vào Nhánh chính.
- Sơ đồ bên dưới cho thấy luồng xử lý thực tế liên quan đến việc sử dụng Nhánh.



Sử dụng Nhánh trong GitHub Desktop (Linux)

Cài đặt GitHub Desktop trên Linux

- Truy cập https://github.com/shiftkey/desktop để cài đặt GitHub Desktop trong môi trường Linux.
- Trong dòng chữ " To setup the package repository, run these commands", hãy nhập 3 dòng lệnh vào Linux Terminal.




```
siciot2021@siciot2021-VirtualBox:~$ wget -qO - https://packagecloud.io/shiftkey/desktop/gpgkey | sudo tee /etc/apt/trusted.gpg.d/shiftkey-des
ktop.asc > /dev/null
siciot2021@siciot2021-VirtualBox:~$ sudo sh -c 'echo "deb [arch=amd64] https://packagecloud.io/shiftkey/desktop/any/ any main" > /etc/apt/sou
rces.list.d/packagecloud-shiftky-desktop.list'
siciot2021@siciot2021-VirtualBox:~$ sudo apt-get update
Hit:2 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:3 http://kr.archive.ubuntu.com/ubuntu focal InRelease
Get:4 http://kr.archive.ubuntu.com/ubuntu focal-updates InRelease [114 kB]
Hit:1 https://packagecloud.io/shiftkey/desktop/any any InRelease
Get:5 http://kr.archive.ubuntu.com/ubuntu focal-backports InRelease [101 kB]
Get:6 http://kr.archive.ubuntu.com/ubuntu focal-updates/main amd64 DEP-11 Metadata [282 kB]
Get:7 http://kr.archive.ubuntu.com/ubuntu focal-updates/universe amd64 DEP-11 Metadata [351 kB]
Get:8 http://kr.archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 DEP-11 Metadata [944 B]
Get:9 http://kr.archive.ubuntu.com/ubuntu focal-backports/universe amd64 DEP-11 Metadata [10.3 kB]
Fetched 859 kB in 5s (160 kB/s)
Reading package lists... Done
```

Sử dụng Nhánh trong GitHub Desktop (Linux)

Cài đặt GitHub Desktop trên Linux

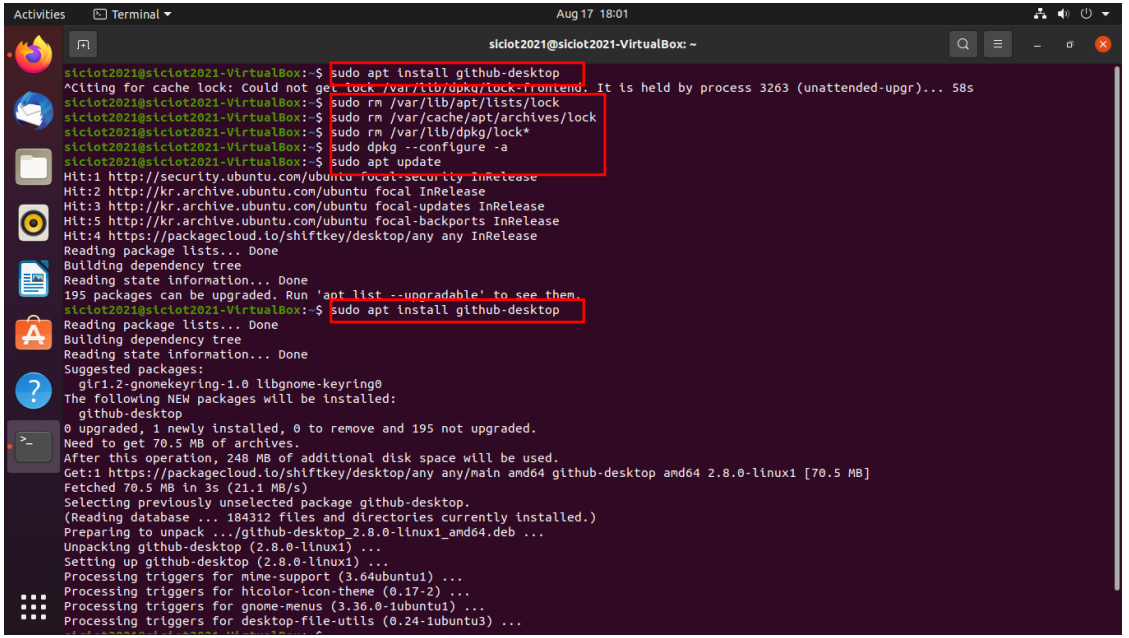
```
$ sudo apt install github-desktop
```

- Khi thực hiện lệnh trên, có thể sẽ không tiến hành được do sự cố liên quan đến bộ nhớ cache. Trong trường hợp này, nhấn (Ctrl + c) để dừng và nhập các lệnh bên dưới để khắc phục sự cố.

```
$ sudo rm /var/lib/apt/lists/lock
$ sudo rm /var/cache/apt/archives/lock
$ sudo rm /var/lib/dpkg/lock*

# Package Update
$ sudo dpkg --configure -a
$ sudo apt update

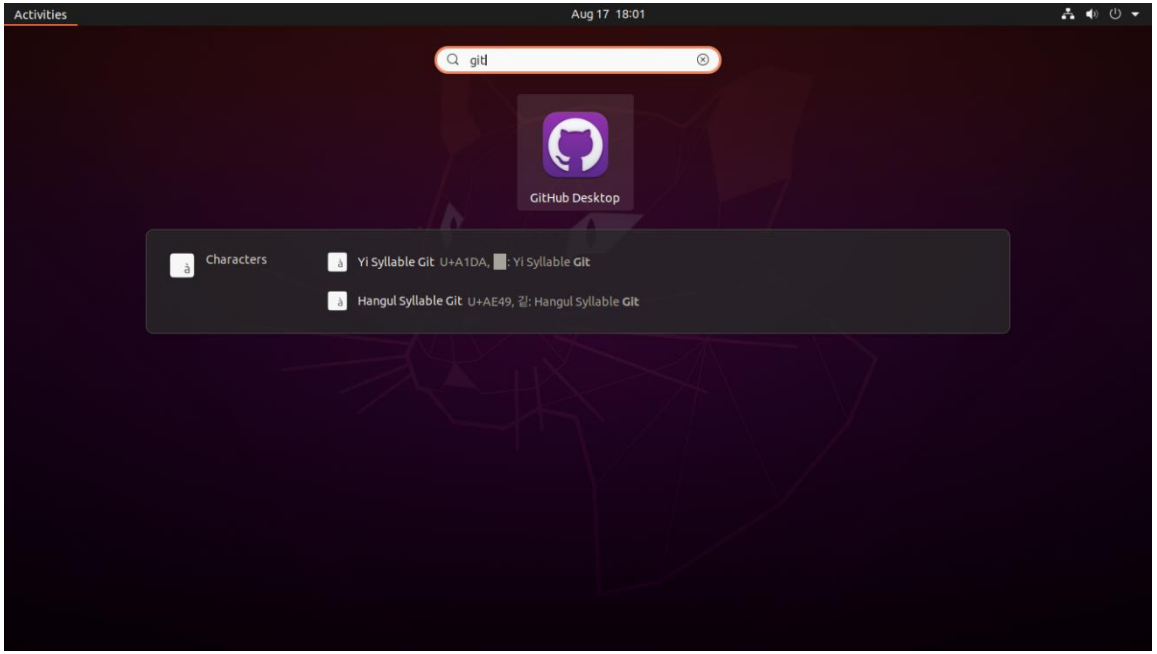
# Retry Install
$ sudo apt install github-desktop
```



Sử dụng Nhánh trong GitHub Desktop (Linux)

Cài đặt GitHub Desktop trên Linux

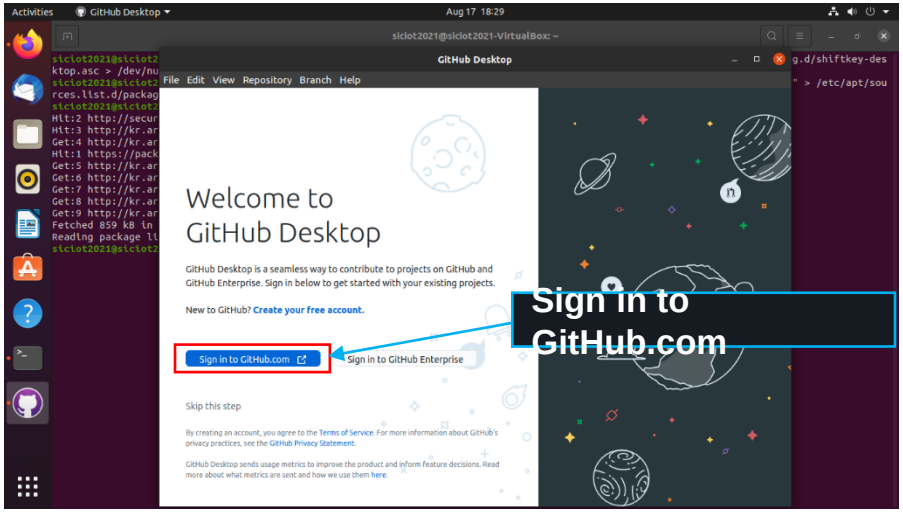
- Sau khi cài đặt thành công, bạn có thể tìm thấy GitHub Desktop. Sau khi chạy, cách sử dụng cũng tương tự với Windows.



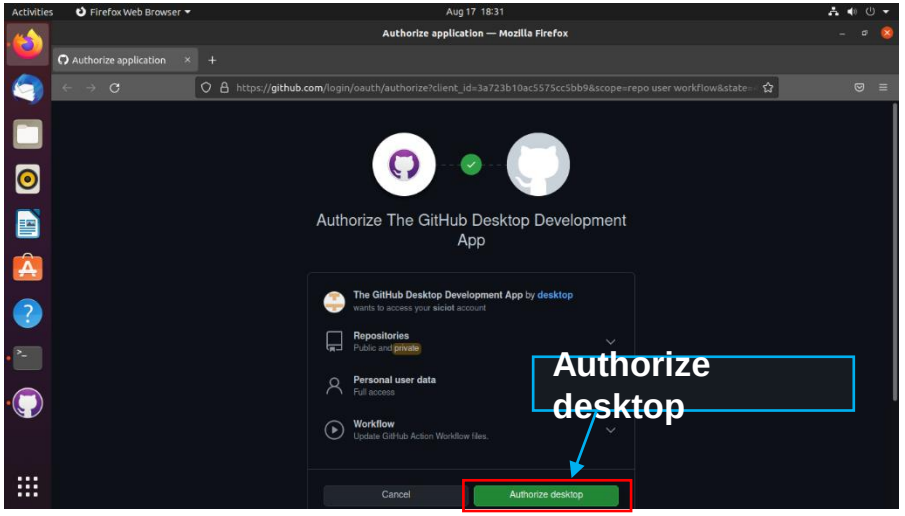
Sử dụng Nhánh trong GitHub Desktop (Linux)

Cài đặt GitHub Desktop trên Linux

- Nhấp vào biểu tượng **Sign in to GitHub.com** và đăng nhập bằng tài khoản GitHub của bạn.



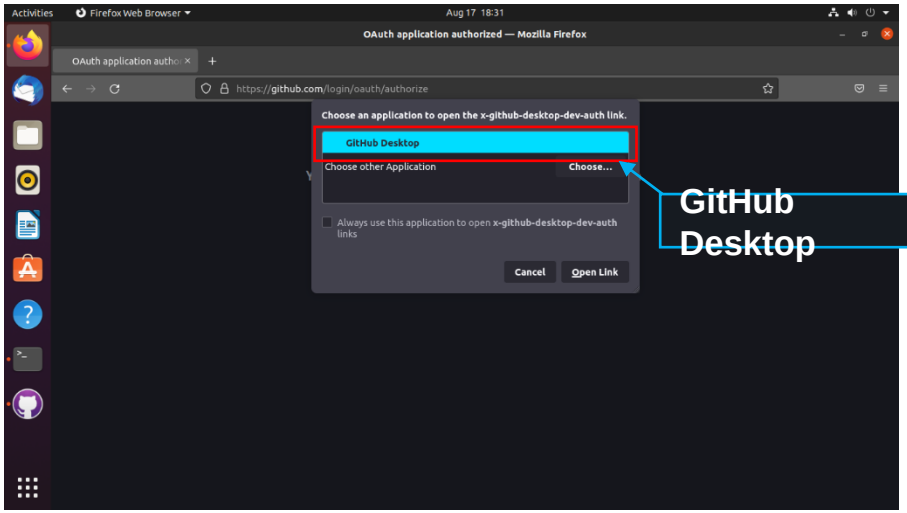
- Nhập thông tin đăng nhập → Nhấp vào nút **Authorize desktop**.



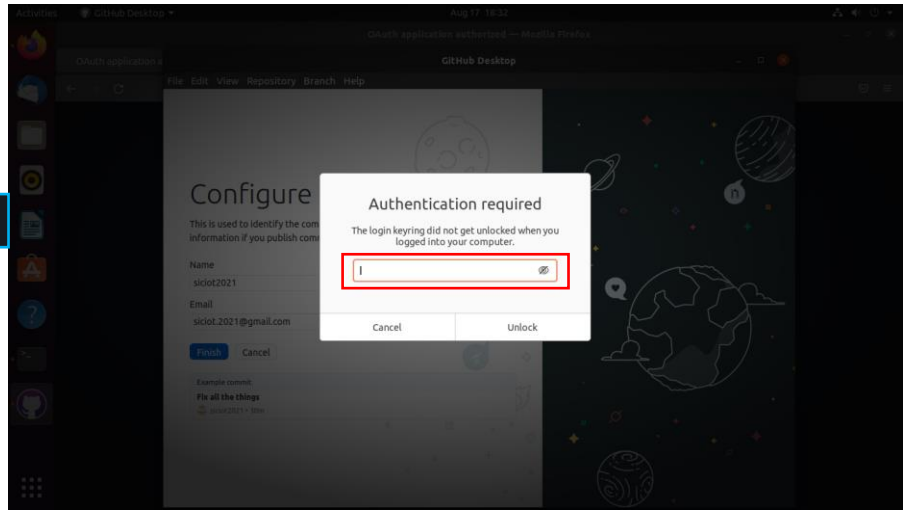
Sử dụng Nhánh trong GitHub Desktop (Linux)

Cài đặt GitHub Desktop trên Linux

- Nhấp đúp vào GitHub Desktop để chọn.



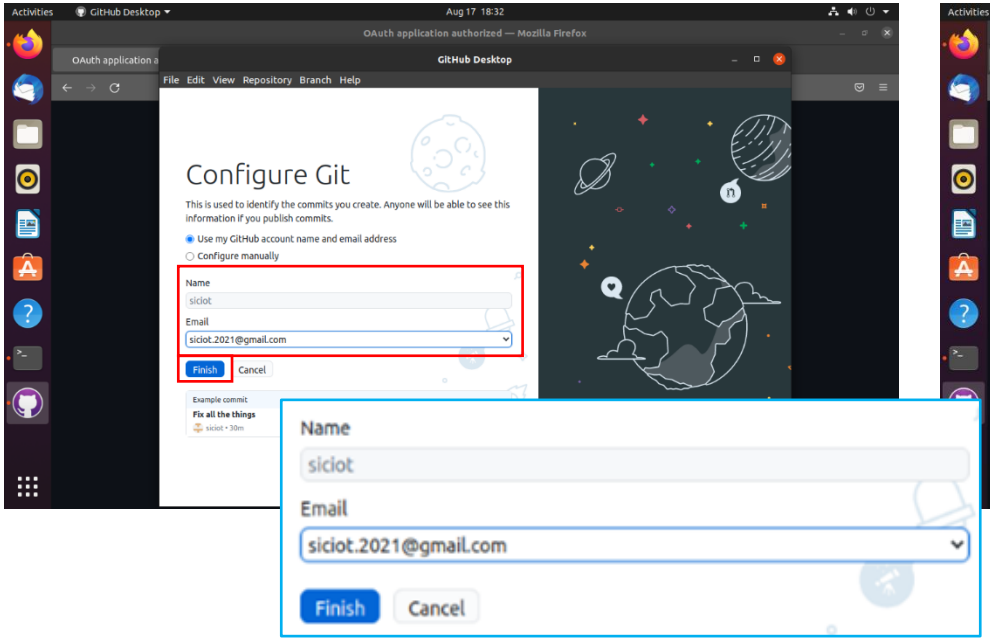
- Nhập mật khẩu của tài khoản Linux cho câu hỏi xác thực.



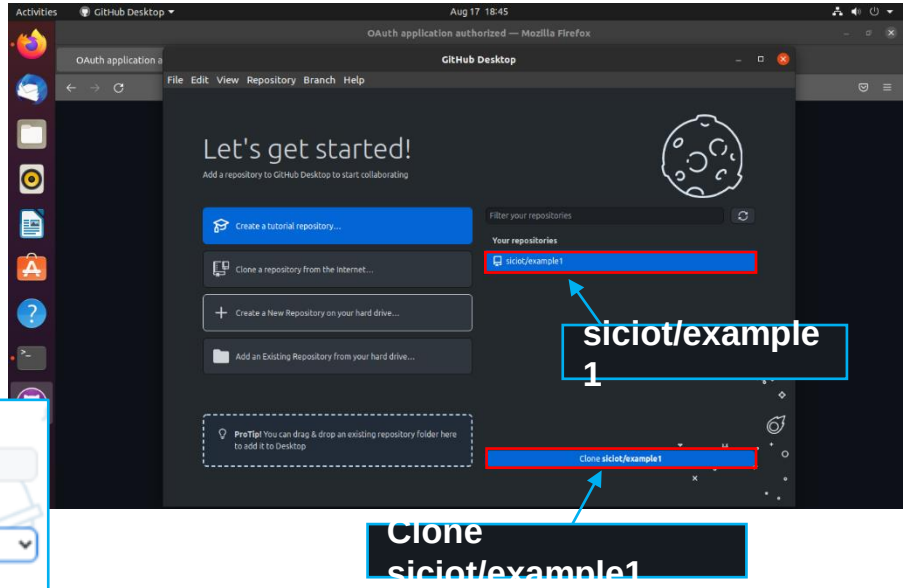
Sử dụng Nhánh trong GitHub Desktop (Linux)

Cài đặt GitHub Desktop trên Linux

- Đăng ký tên và địa chỉ e-mail cần đăng ký khi xác nhận. Sau khi cài đặt, bạn có thể thấy màn hình của GitHub Desktop trong môi trường Linux.



- Chọn kho lưu trữ example1 được tạo làm ví dụ, sao chép nó và nhập nó.

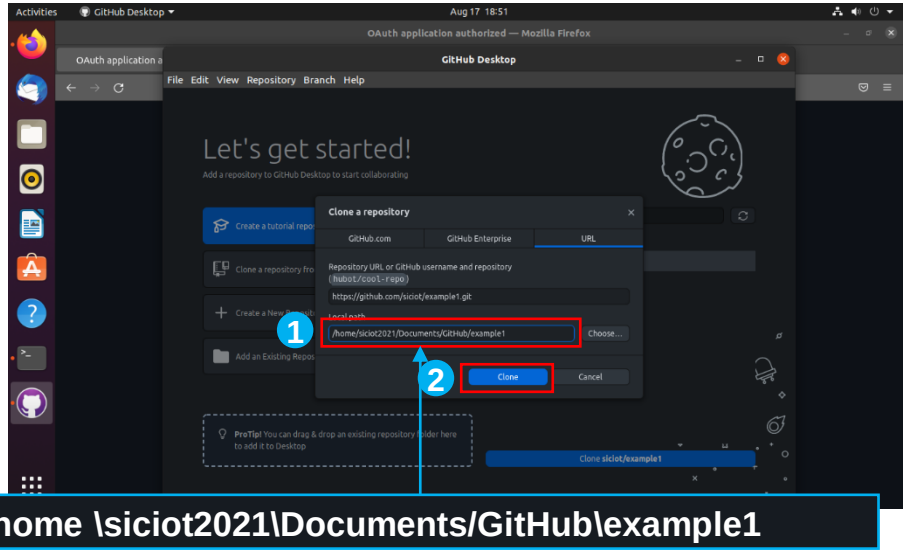


Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux

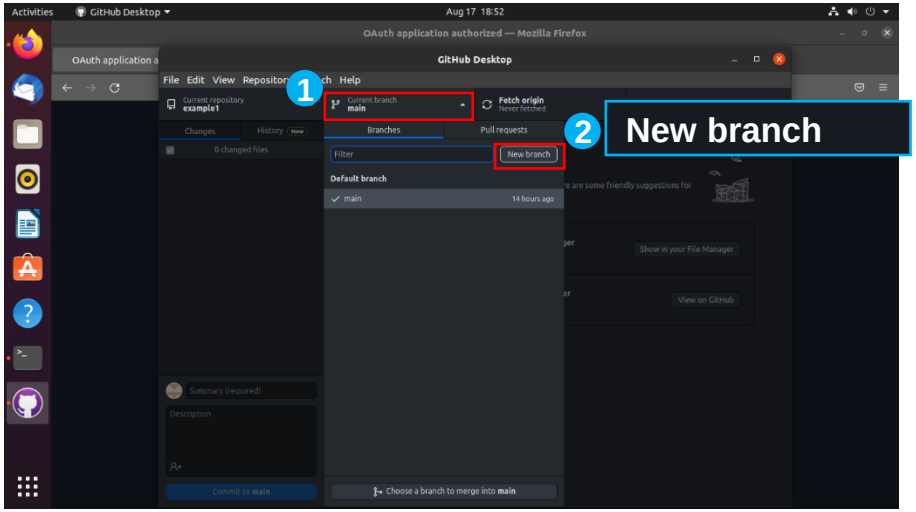
- Giả sử rằng trạng thái hiện tại của dự án Clone là nhánh hoạt động phục vụ cho một tác vụ vận hành ổn định. Nếu bạn muốn làm việc mà không ảnh hưởng đến dự án này, bạn có thể tạo một Nhánh mới để làm việc.

- Kiểm tra **Local Path** và nhấp vào biểu tượng **clone**.



\home \siciot2021\Documents/GitHub\example1

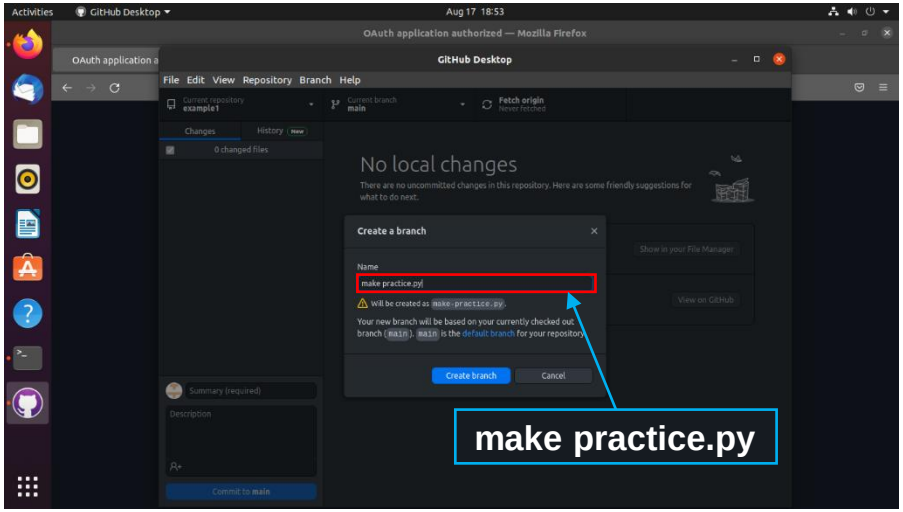
- Đi tới Nhánh hiện tại để rời khỏi Nhánh chính hiện tại và tạo Nhánh mới → Nhấp chọn **New branch**



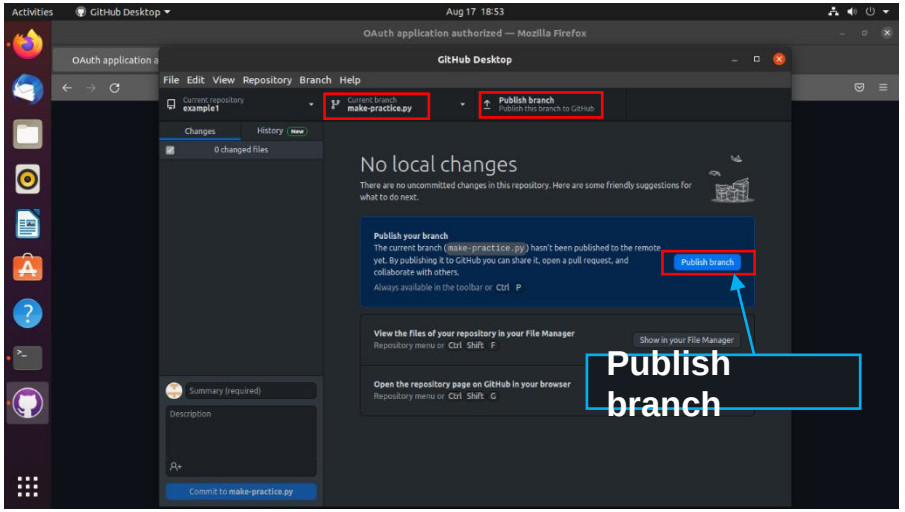
Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux

- Tạo một nhánh có tên là **make practice.py**. Nó là một nhánh để tạo một tệp Python mới có tên practice.py.



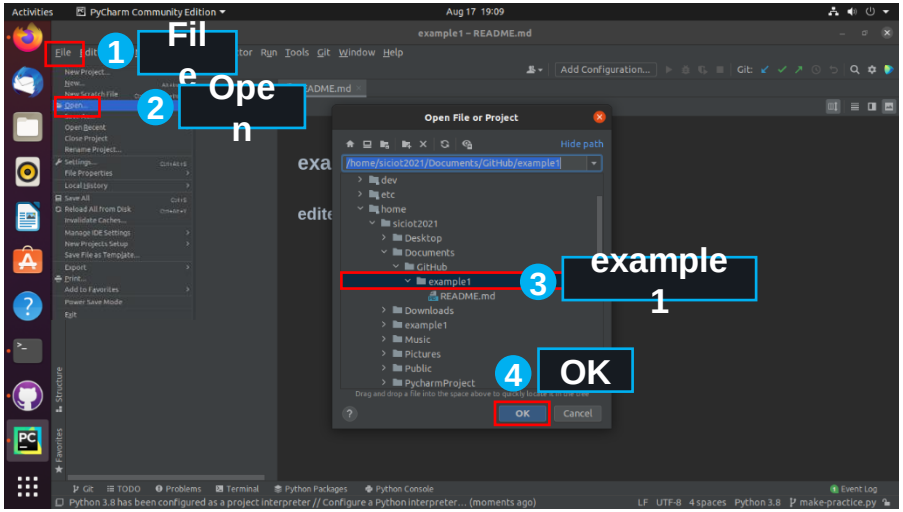
- Kiểm tra xem Nhánh hiện tại đã được thay đổi thành **make practice.py** chưa.
  Nhấp vào nút **Publish branch** để tạo nhánh do Kho lưu trữ từ xa chưa xác định branch mới tạo.



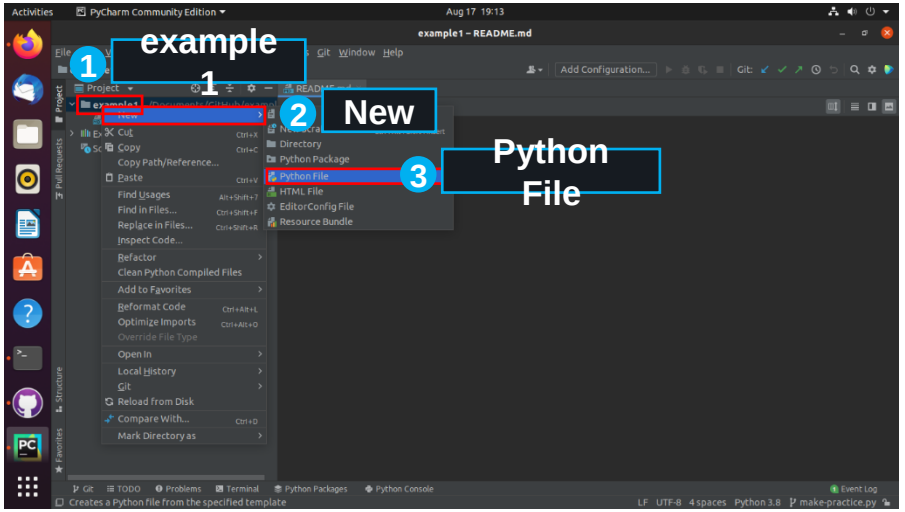
Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux

- Chạy Pycharm để tạo file practice.py trong Local Path.
- File → Open → Select directory example1 → OK



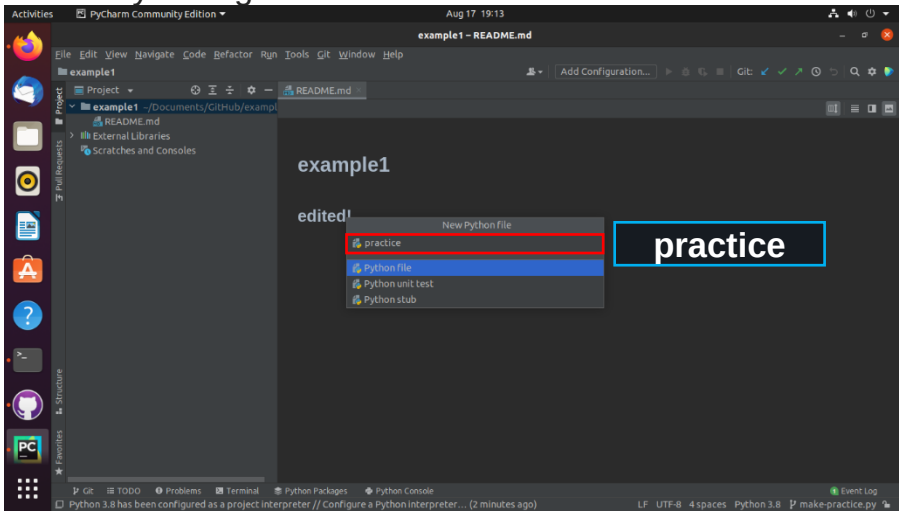
- Bấm chuột phải vào example1 → New→ Click Python File



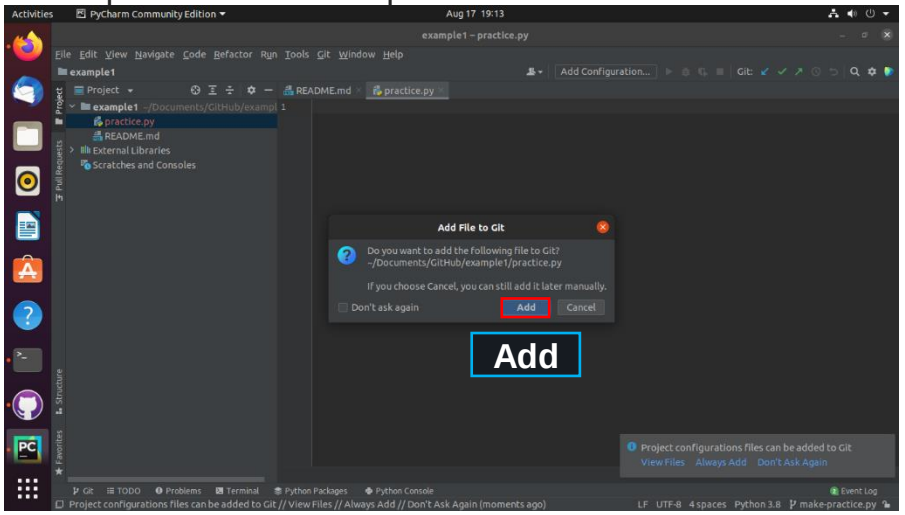
Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux

- Nhập pratice rồi bấm phím enter. Bạn có thể thấy rằng tệp practice.py đã được tạo thành công.
- Một cửa sổ bật lên hỏi bạn có muốn git theo dõi file này không.



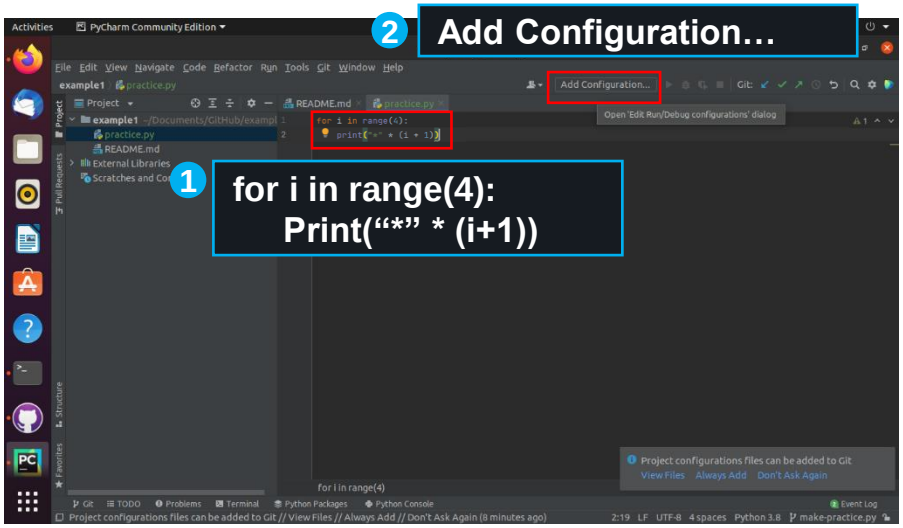
- Nhấp chọn Add để đưa file vào đối tượng theo dõi ngay lập tức (Không phải thực hiện git add practice.py).
- Nếu bạn không muốn theo dõi, hãy nhấp vào Cancel. Nhấp vào Add for this practice.



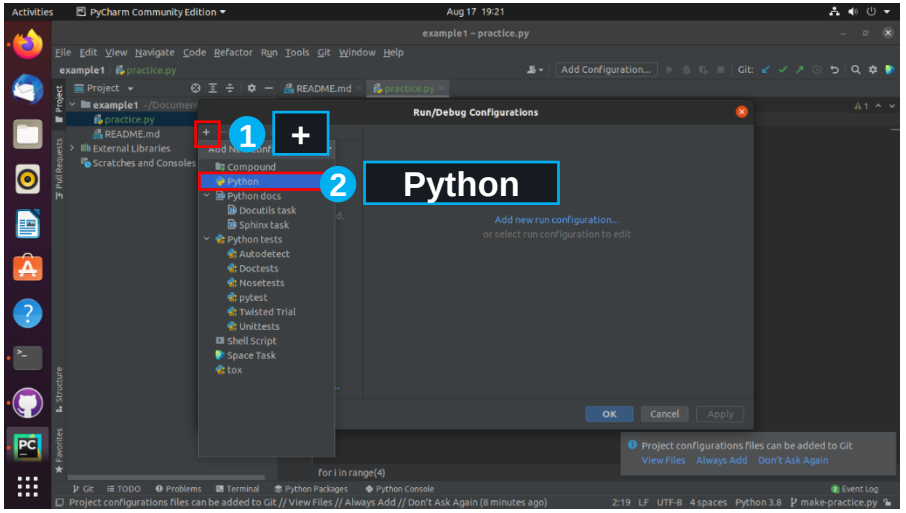
Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux

- Viết mã để tạo một chương trình đơn giản trong practice.py. Nó xuất ra "*" trên 4 dòng bằng cách tăng tuần tự.
  Nhấp chuột chọn Add Configuration vì chưa được cấu hình để chạy.



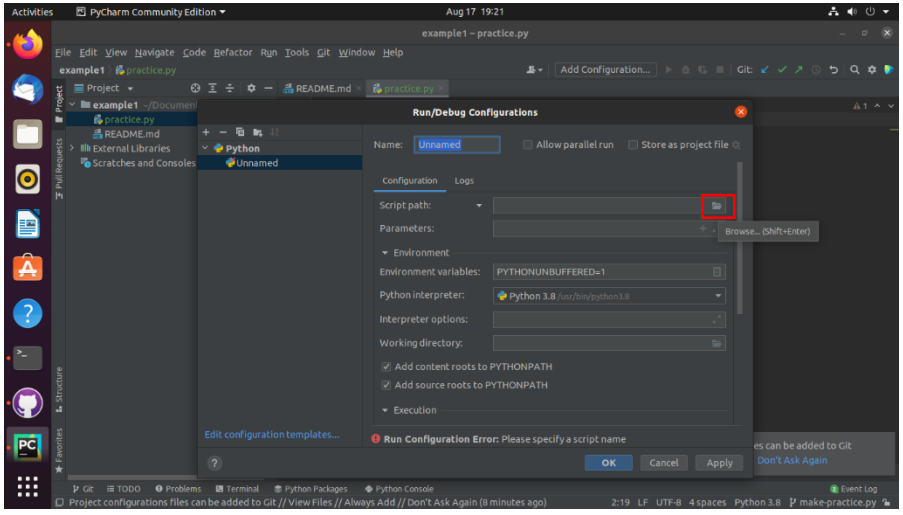
- Nhấp vào biểu tượng + → Chọn Python



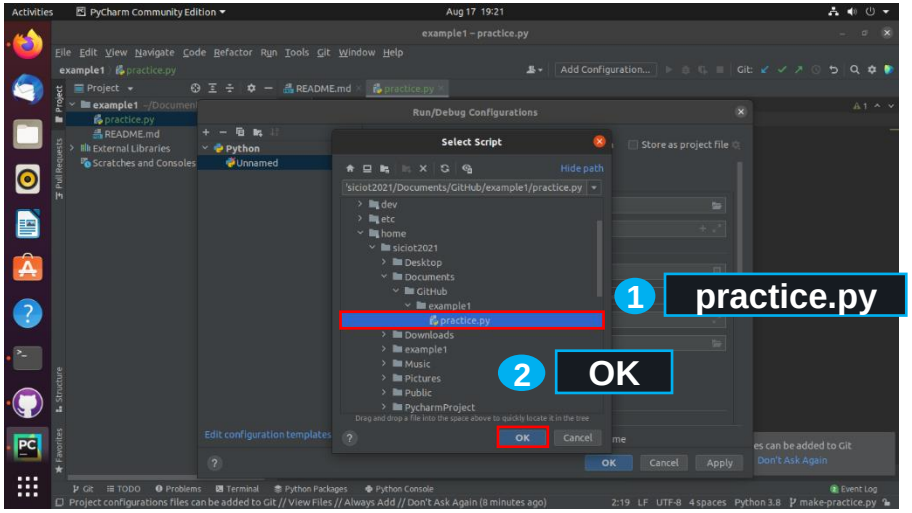
Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux

- Nhấp **Browse** (biểu tượng thư mục) để thêm đường dẫn tập lệnh.



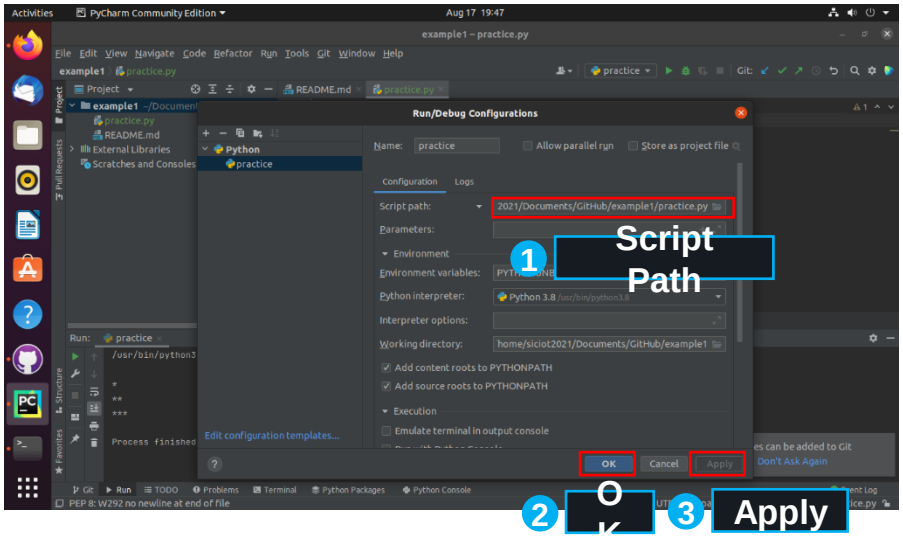
- Nhấp vào file practice.py trong my Local Path → OK



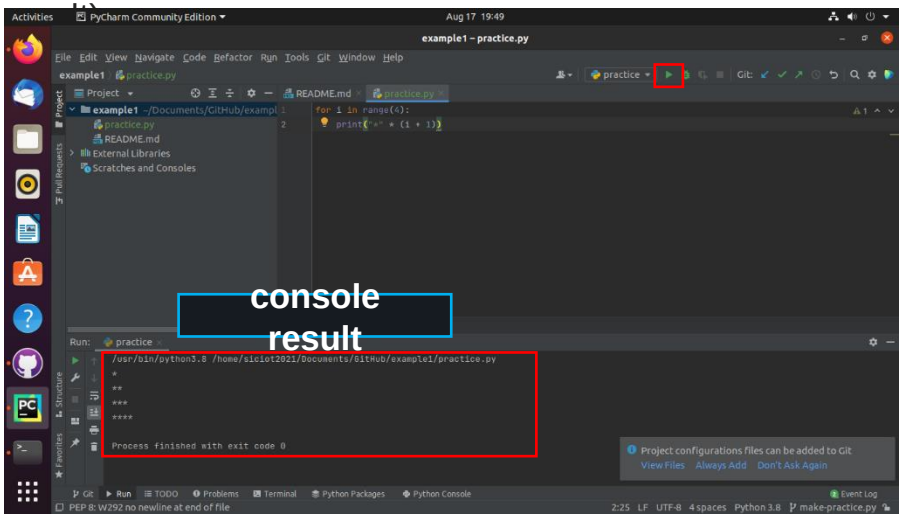
Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux

- Kiểm tra phần mới thêm vào Script Path → Apply → OK



- Chạy với cấu hình đã cài đặt
  → Kiểm tra kết quả qua bảng điều khiển (console

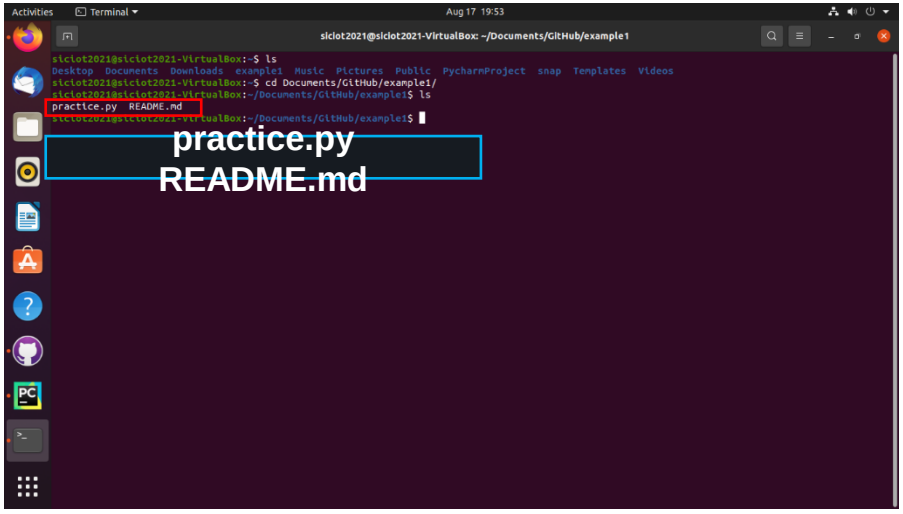


Sử dụng Nhánh trong GitHub Desktop (Linux)

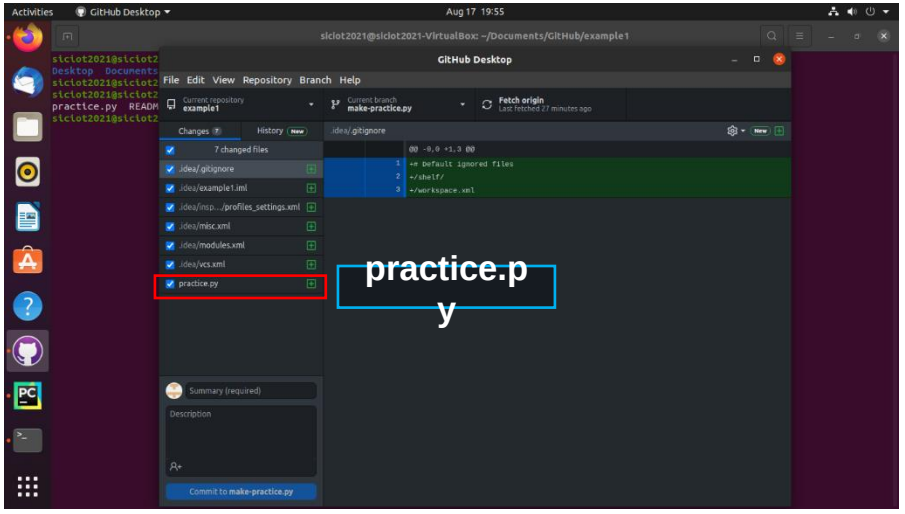
Sử dụng Nhánh trong GitHub Desktop trên Linux

- Truy cập tới Local Path và kiểm tra lại.

```
$ cd {local path}
$ ls
```



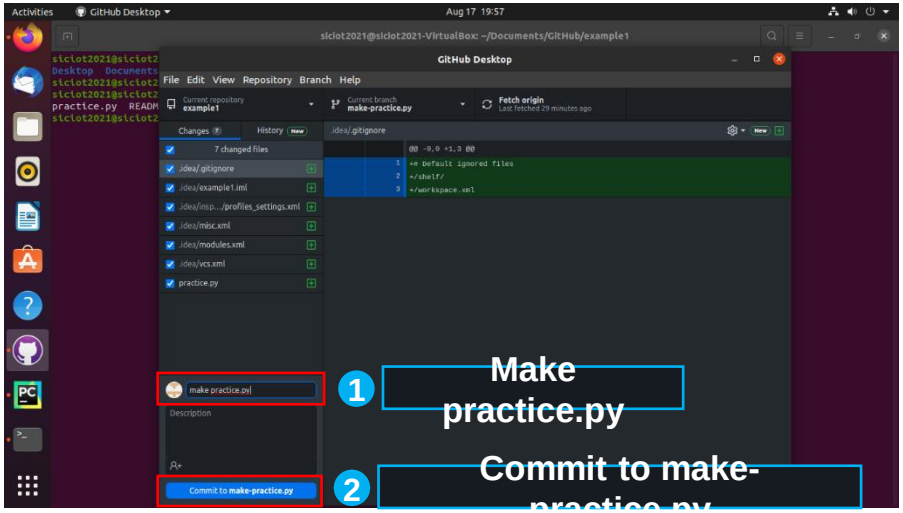
- Kiểm tra GitHub Desktop, bạn có thể thấy rằng một số file bao gồm cả practice.py đã được thêm vào nhánh make practice.py.Có thể thấy git hỗ trợ rất tốt trong việc quản lý các file ngoài file practice.py.



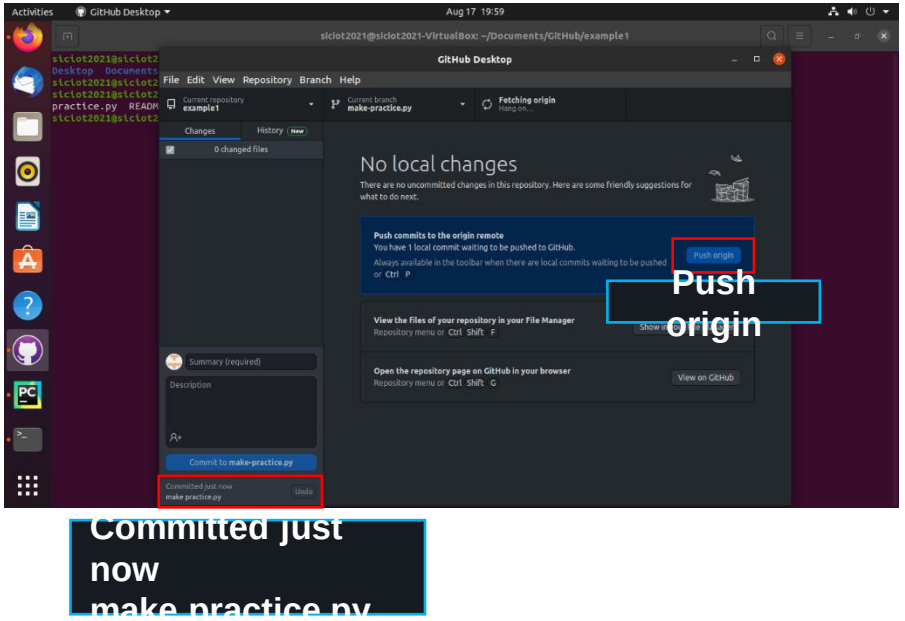
Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux

- Xác nhận nhánh make practice.py với thông điệp xác nhận "make practice.py".



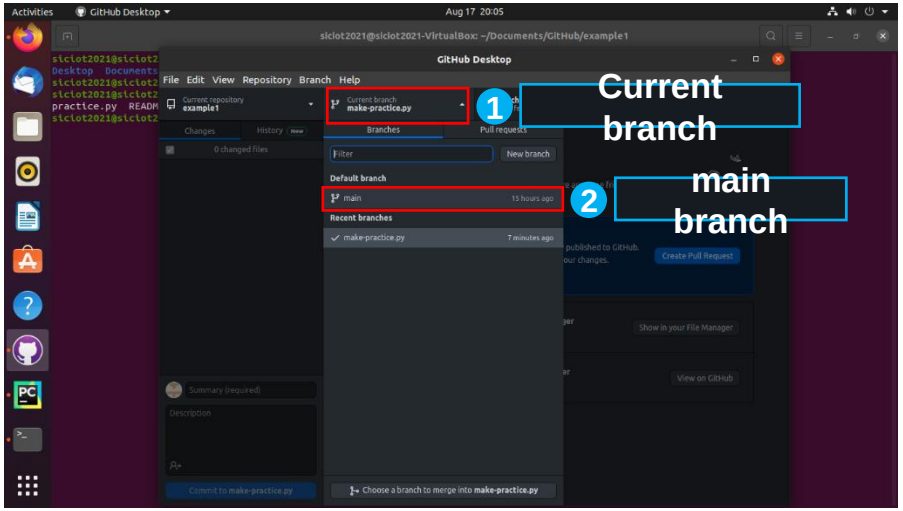
- Nhấp chọn **Push origin** để đẩy practice.py vào nhánh make practice.py.



Sử dụng Nhánh trong GitHub Desktop (Linux)

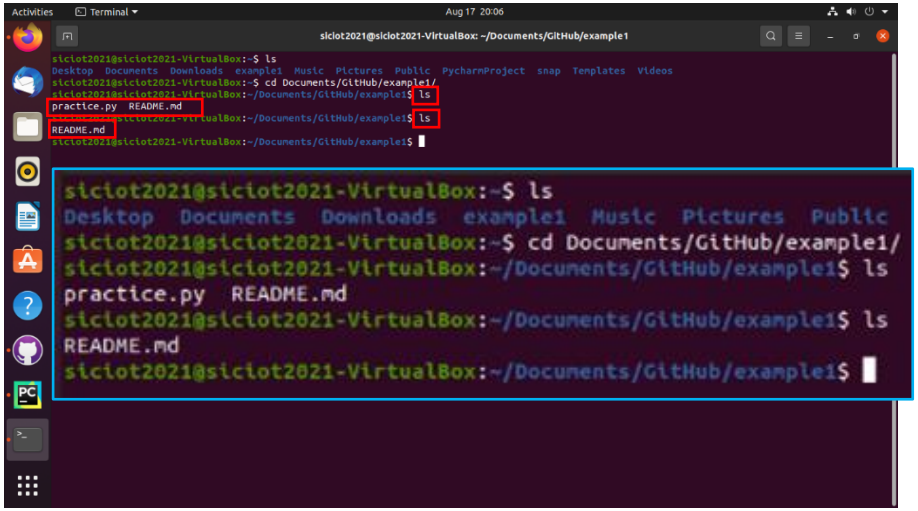
Sử dụng Nhánh trong GitHub Desktop trên Linux

- Tới thẻ current branch
  → Nhấp vào main để quay về nhánh chính



- Kiểm tra danh sách thông qua lệnh ls trong Local Path. Bạn có thể thấy practice.py đã biến mất.

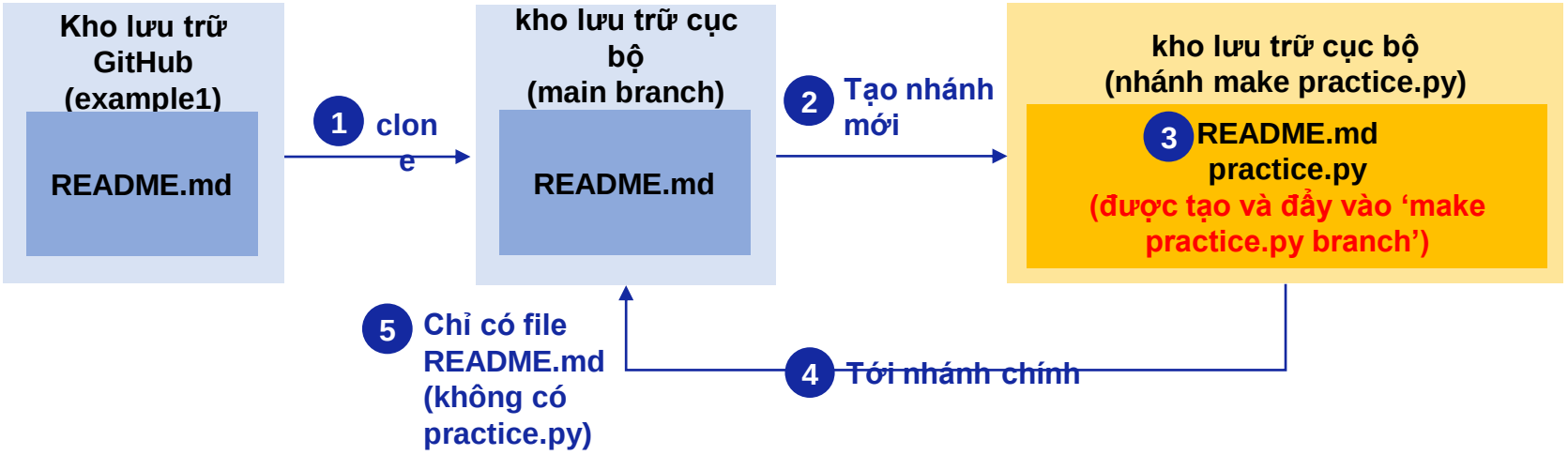
```
$ ls
```



Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux (1/2)

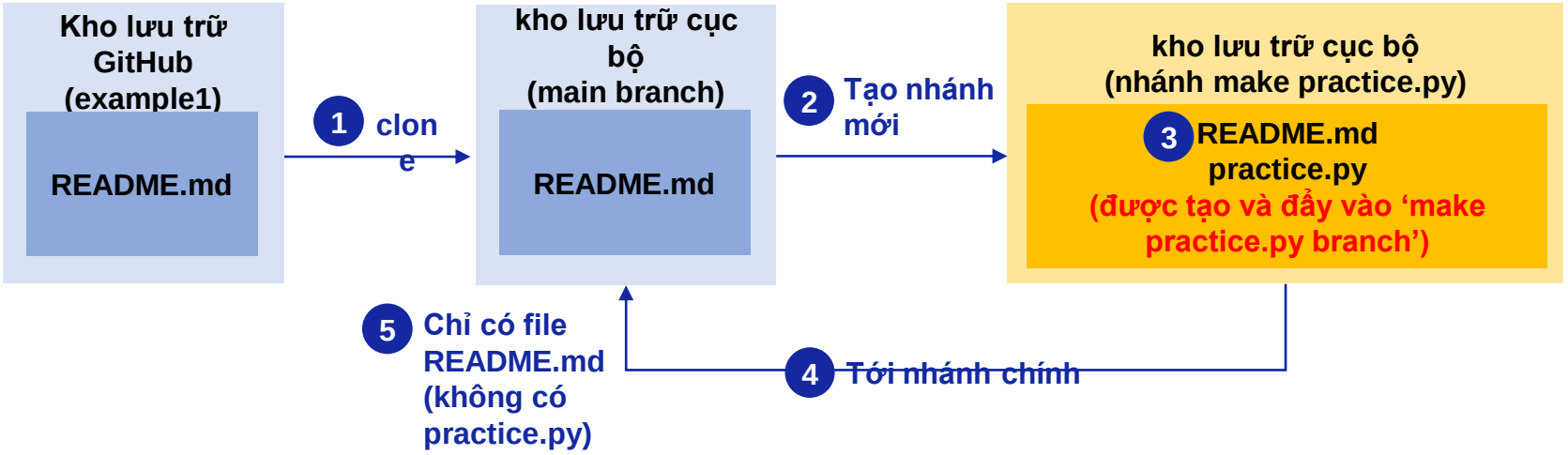
- Quá trình thực hành như sau:
  Nạp dự án trong Kho lưu trữ GitHub hiện có vào Kho lưu trữ cục bộ Linux bằng cách tạo bản sao.
  Tạo một nhánh mới có tên make practice.py để chạy thử nghiệm độ ổn định của thao tác trước khi chỉnh sửa.
  Tạo một tệp python trong nhánh make practice.py. Lưu nó rồi tiến hành quá trình Commit và Push.



Sử dụng Nhánh trong GitHub Desktop (Linux)

Sử dụng Nhánh trong GitHub Desktop trên Linux (2/2)

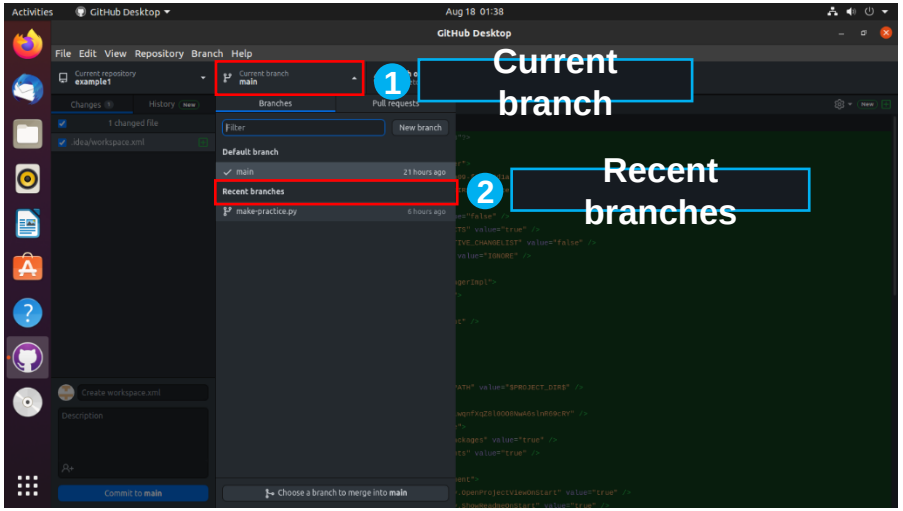
- Không có thay đổi trong nhánh chính.
  Điều này là do chúng ta đã thao tác trong nhánh make practice.py mới, là một không gian làm việc độc lập.
  Khi thực hiện công việc cá nhân hoặc tập thể, đều có thể chạy thử nghiệm trong khi làm việc độc lập mà không ảnh hưởng đến tính ổn định của dự án. Nếu kết quả thử nghiệm không ổn định, nó sẽ không được tích hợp vào nhánh chính. Khi bạn quay trở lại nhánh chính, nội dung của dự án cũng trở về trạng thái ban đầu.



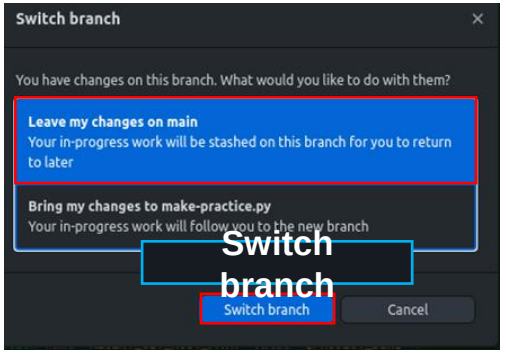
Sử dụng Nhánh trong GitHub Desktop (Linux)

Gây tác động tới một nhánh mới

- Sau khi chạy đủ các thử nghiệm xem xét các vấn đề có thể xảy ra trong việc tạo file practice.py không, ta hợp nhất kết quả vào nhánh chính.
- Tới current branch để di chuyển từ nhánh main tới nhánh make practice.py → current branch



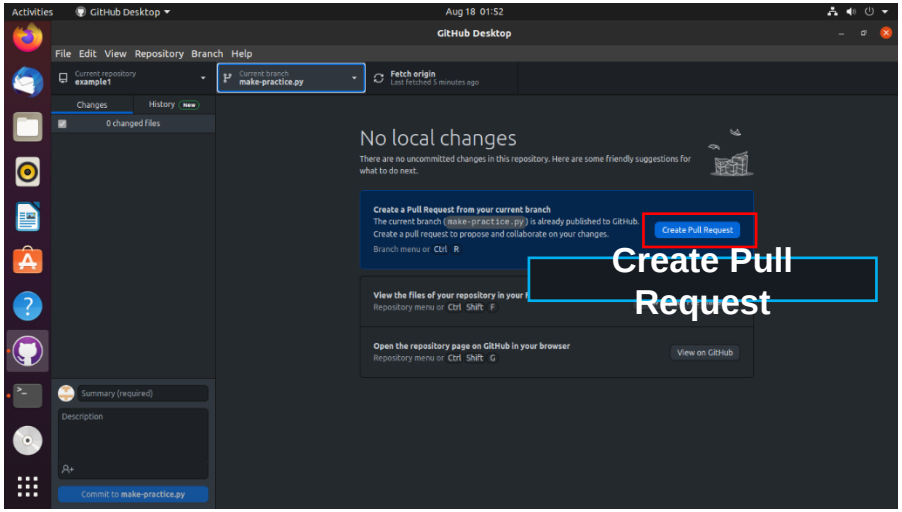
- Leave my changes on main có nghĩa là nội dung của nhánh chính sẽ không được ánh xạ trong nhánh practice.py. Mặt khác, Bring my changes to make practice.py. có nghĩa là nội dung của nhánh chính được ánh xạ trong nhánh make practice.py.
- Chọn Leave my changes on the main và nhấp vào biểu tượng switch branch. vì chúng ta cần giữ nguyên nội dung của nhánh chính.



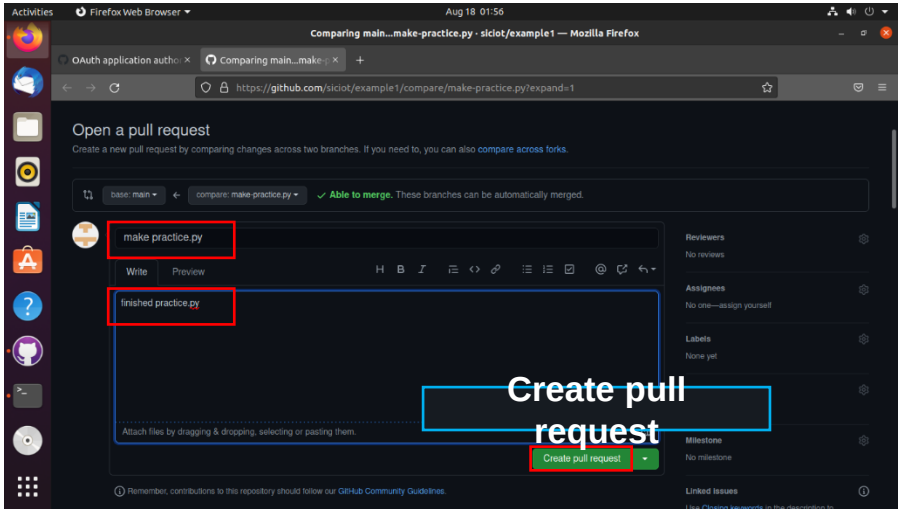
Sử dụng Nhánh trong GitHub Desktop (Linux)

Gây tác động tới một nhánh mới

- Chọn Create Pull Request để hợp nhất các thay đổi vào nhánh chính.



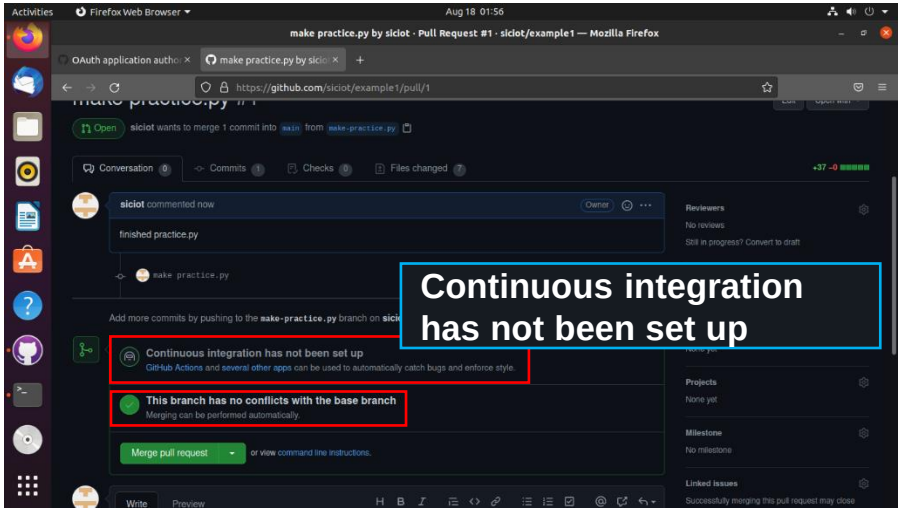
- Di chuyển đến trang Open a pull request.
- Viết tin nhắn yêu cầu pull và nhấp vào nút Create pull request button.



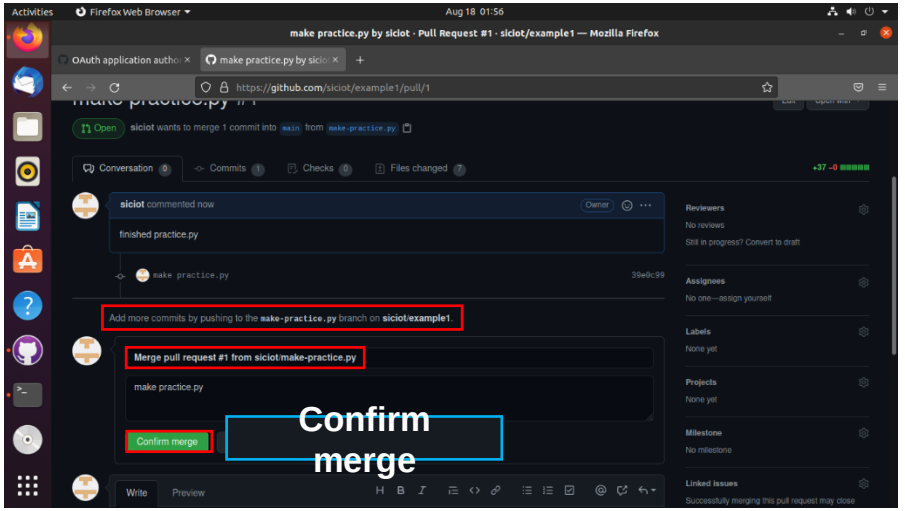
Sử dụng Nhánh trong GitHub Desktop (Linux)

Gây tác động tới một nhánh mới

- Kiểm tra tin nhắn “This branch has no conflicts with the base branch”
- Nhấp vào biểu tượng Merge pull request để hợp nhất nhánh make practice.py đã được nhận pull request vào main.



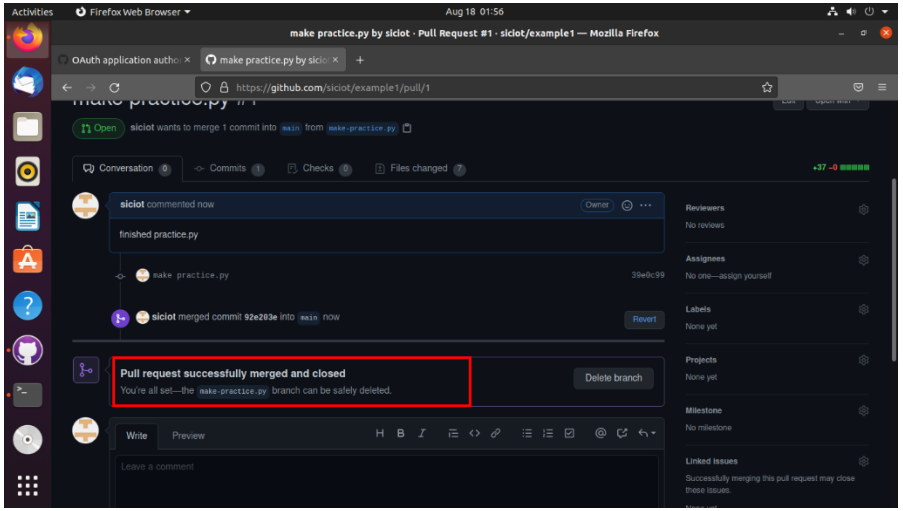
- Kiểm tra commit message để xác nhận các nội dung đã hợp nhất và nhấn vào Confirm merge.



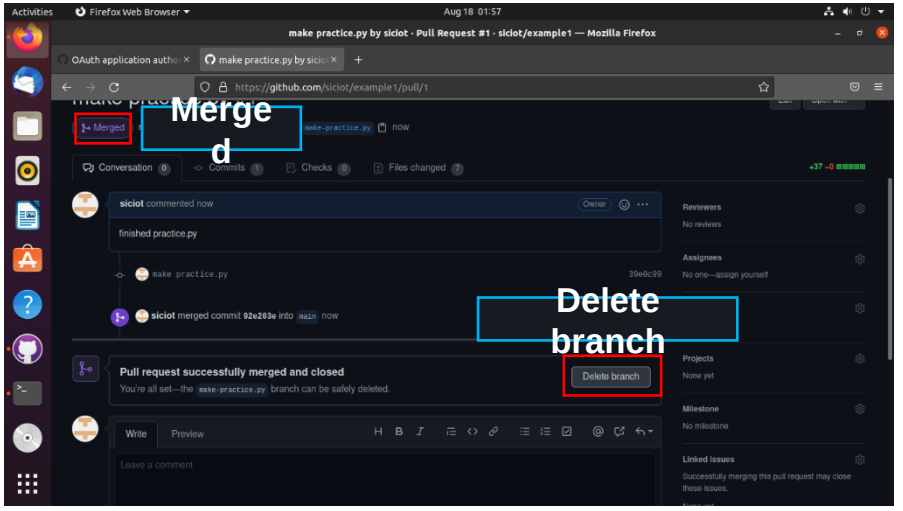
Sử dụng Nhánh trong GitHub Desktop (Linux)

Gây tác động tới một nhánh mới

- Kiểm tra tin nhắn “Pull request successfully merged and closed”.



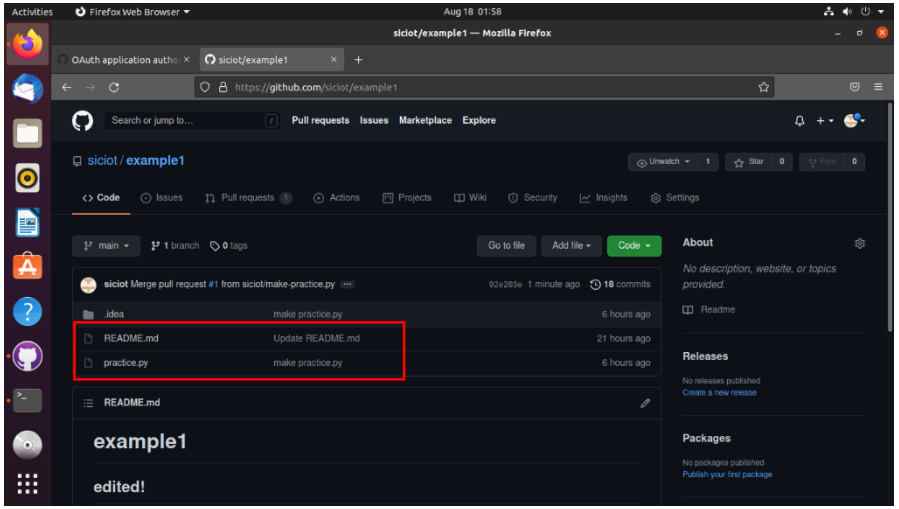
- Nhấp vào biểu tượng Delete branch ở bên phải để xóa nhánh make practice.py đã hoàn thiện.
- Kiểm tra phần trên cùng bên trái để xem trạng thái đã thay đổi thành Merged chưa.



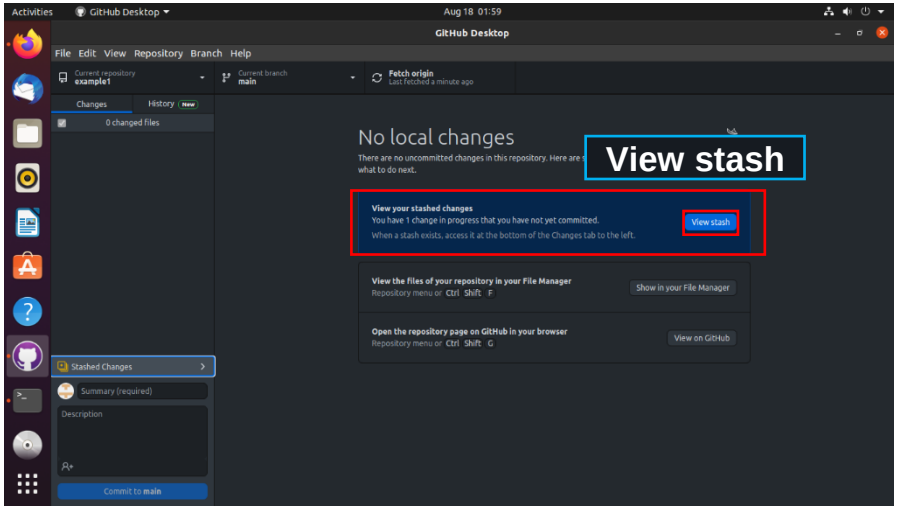
Sử dụng Nhánh trong GitHub Desktop (Linux)

Gây tác động tới một nhánh mới

- Quay trở lại Kho lưu trữ GitHub và kiểm tra xem practice.py có tồn tại hay không, mặc dù nó hiện đang nằm trong nhánh main.



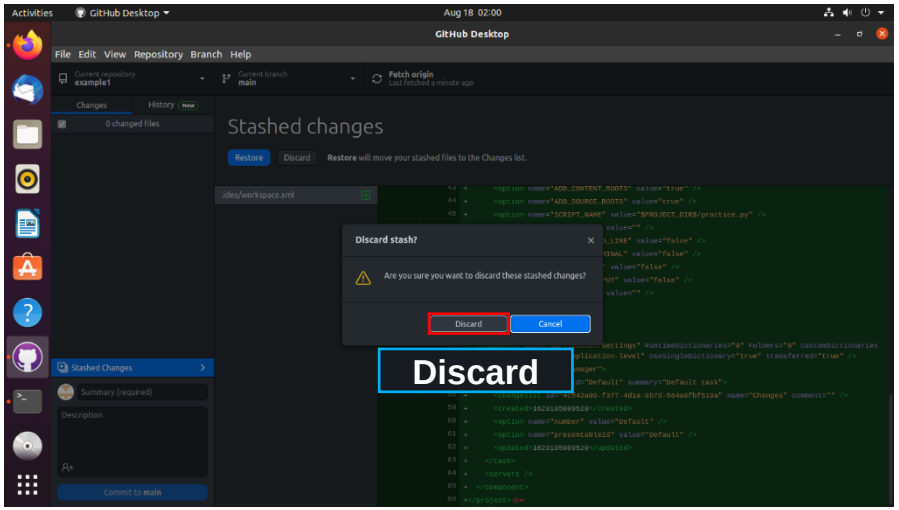
- Quay lại GitHub Desktop và kiểm tra nhánh chính. Bạn có thể thấy menu được gọi là View your stashed changes đang hoạt động. Nhấp vào biểu tượng View stash.



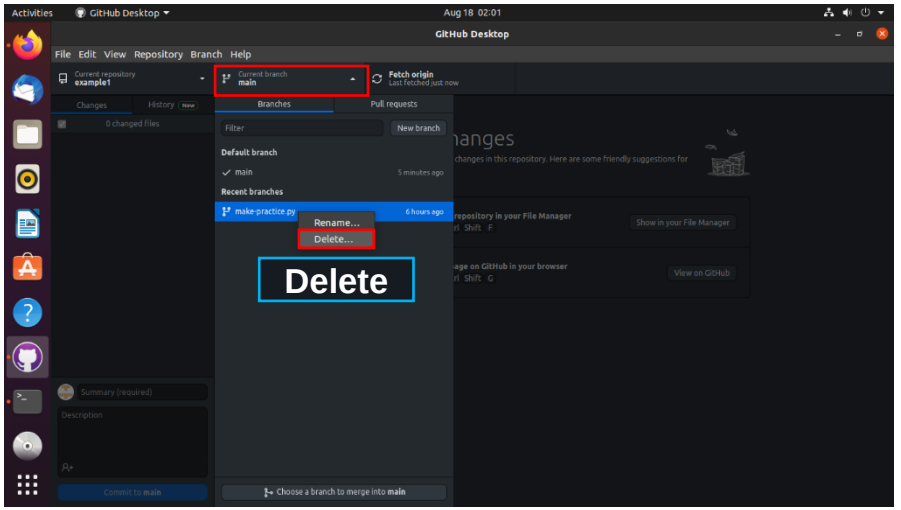
Sử dụng Nhánh trong GitHub Desktop (Linux)

Gây tác động tới một nhánh mới

- Chúng ta có thể tạm thời lưu các thay đổi mà không cần xác nhận chúng với một nhánh bằng cách lưu các thay đổi. Trong trường hợp này, chúng ta không cần phải lưu vùng nhớ stash. Và để loại bỏ chúng, nhấp vào biểu tượng Discard.



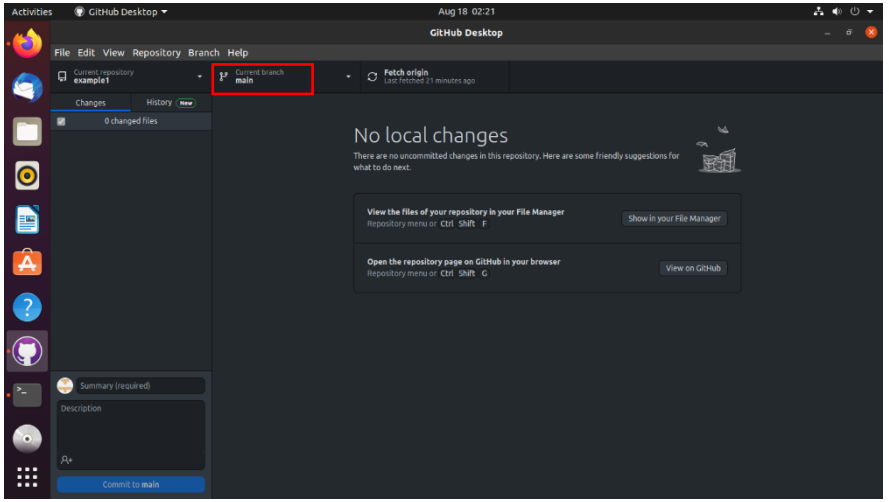
- Xóa nhánh make-practice.py vì bạn vẫn có thể thấy nó trên GitHub Desktop.
- Current branch → Bấm chuột phải make practice.py → Delete



Sử dụng Nhánh trong GitHub Desktop (Linux)

Gây tác động tới một nhánh mới

- Xóa nhánh make practice.py và quay lại nhánh chính. Cuối cùng, kiểm tra Kho lưu trữ cục bộ.



- Khi bạn di chuyển đến Local Path nơi example1 được lưu, bạn có thể thấy rằng practice.py và README.md tồn tại.
  Bạn cũng có thể kiểm tra nội dung của practice.py.

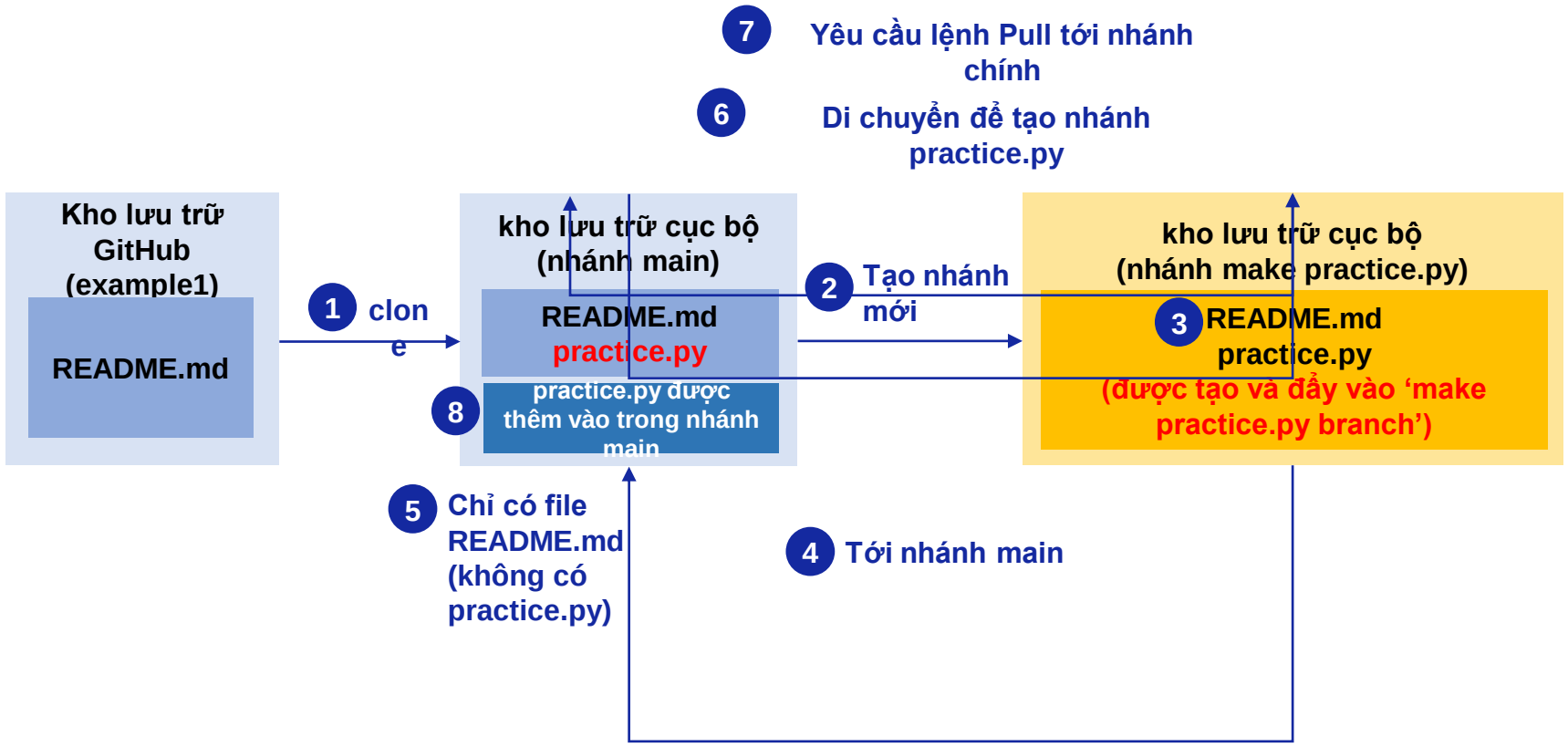
```
$ ls
```

```
$ cat practice.py
```

```
siciot2021@siciot2021-VirtualBox: ~/Documents/GitHub/example1
siciot2021@siciot2021-VirtualBox:~/Documents/GitHub/example1$ ls
practice.py  README.md
siciot2021@siciot2021-VirtualBox:~/Documents/GitHub/example1$ cat practice.py
for i in range(4):
    print("*" * (i + 1))siciot2021@siciot2021-VirtualBox:~/Documents/GitHub/example1$
```

Sử dụng Nhánh trong GitHub Desktop (Linux)

Lưu đồ của quy trình sử dụng Nhánh



Bài 5.

Cộng tác bằng GitHub

5.1. Bài thực hành đơn giản

Tổng quan Bài thực hành

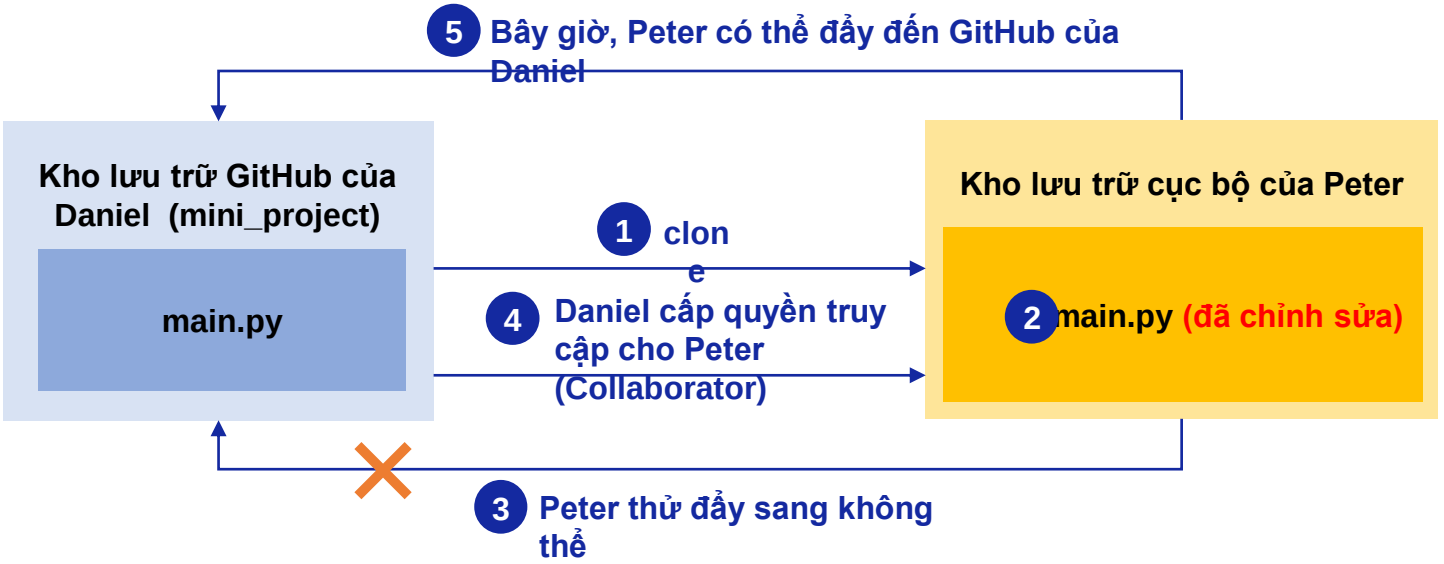
Tổng quan Bài thực hành



- Cùng thực hiện một Mini-Project về Python theo nhóm gồm Daniel (Project Manager - Quản lý dự án) và Peter (Developer - Lập trình viên).
- Daniel quản lý Kho lưu trữ trên GitHub và tổng thể dự án. Peter tạo ra một chương trình giải toán đơn giản với vai trò lập trình viên.
- Daniel sử dụng Window10, Peter sử dụng Linux (Ubuntu) và cả hai cùng sử dụng GitHub Desktop.
- Hãy cùng tìm hiểu cách thao tác Quản lý phiên bản và quản lý dự án khi làm việc cùng nhau trong một dự án.

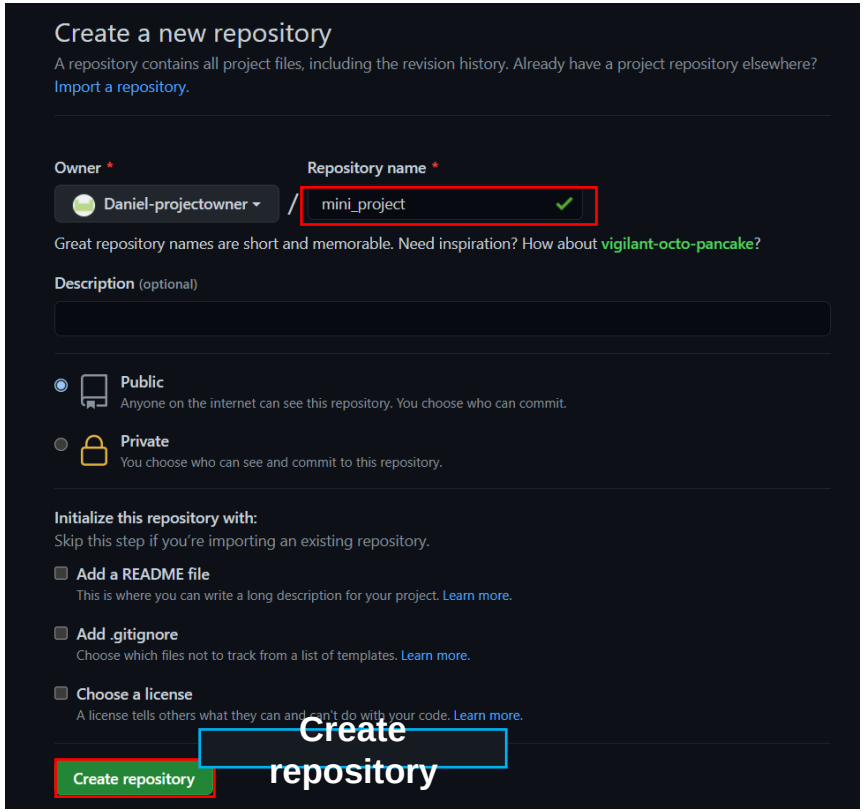
Lưu đồ thực hiện Mini-Project

Minh hoạ lưu đồ mô tả quá trình cộng tác



Bắt đầu với Mini-Project

Tạo kho lưu trữ dự án (Daniel)

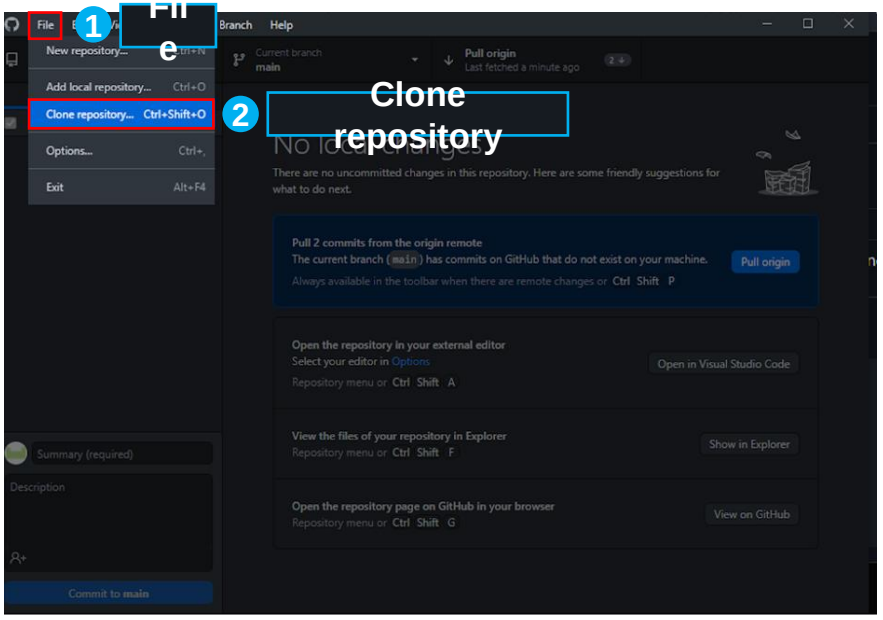


- Daniel tạo một kho lưu trữ để làm việc và quản lý các dự án trên GitHub.
- Tên của kho lưu trữ được đặt là mini_project rồi nhấp vào Create repository để khởi tạo.

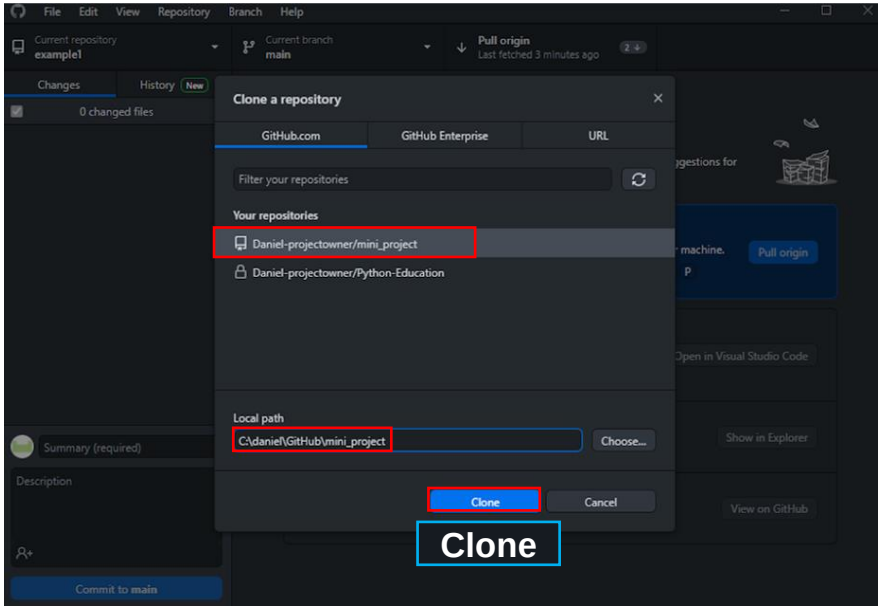
Bắt đầu với Mini-Project

Tạo kho lưu trữ dự án (Daniel)

- Daniel cần sao chép Kho lưu trữ GitHub của mình vào Kho lưu trữ cục bộ.
- File → Nhấp vào Clone repository



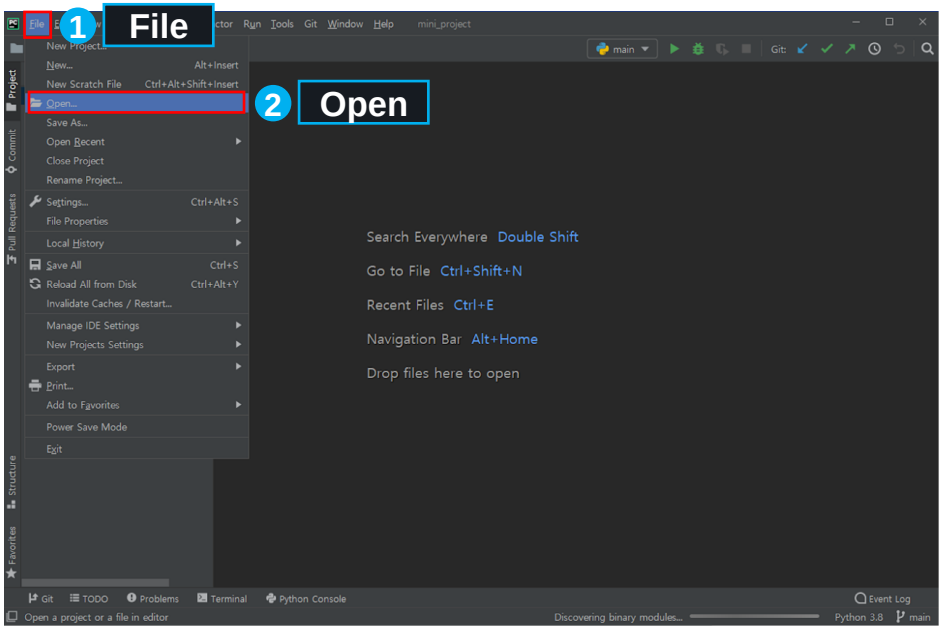
- Nhấp vào mini_project Repository trên GitHub Repository.
- Đặt Local Path rồi nhấp vào nút Clone.

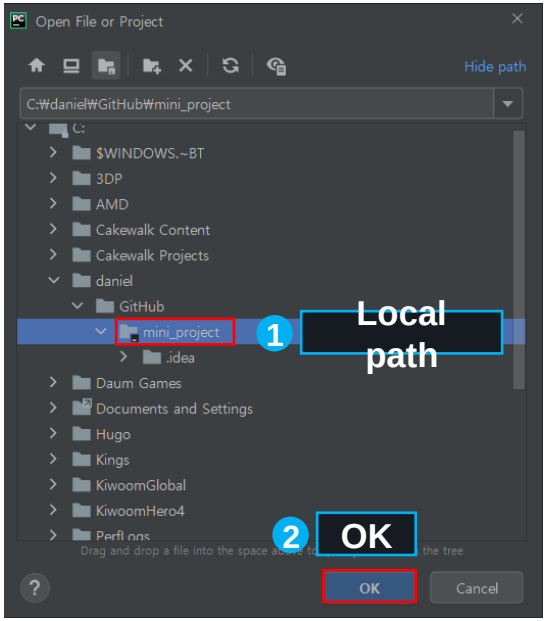


Bắt đầu với Mini-Project

Tạo file main.py (Daniel)

- Mở Pycharm để lập trình trong Kho lưu trữ cục bộ
- Nhấp vào File → Open
- Chọn mini_project trong Local Path kết nối với Kho lưu trữ GitHub → OK

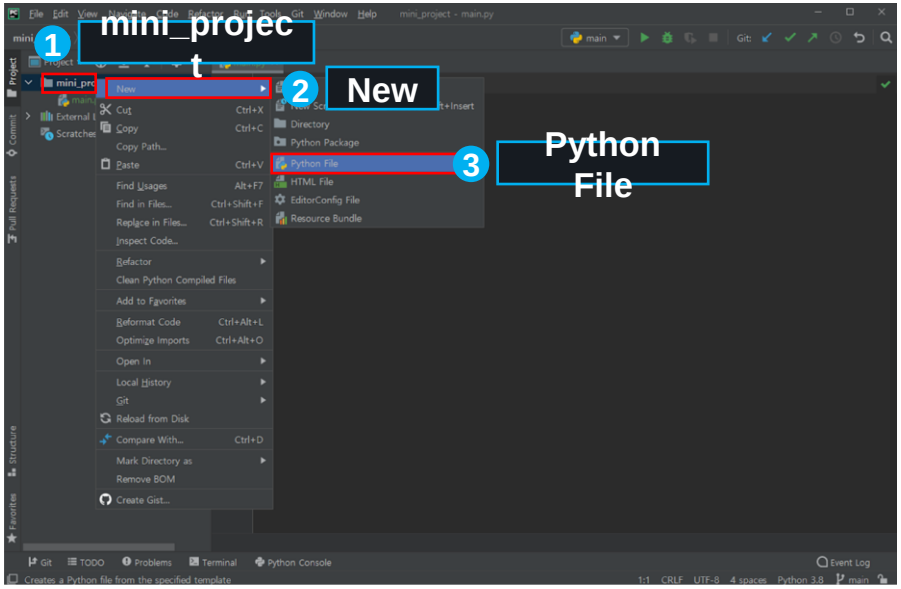




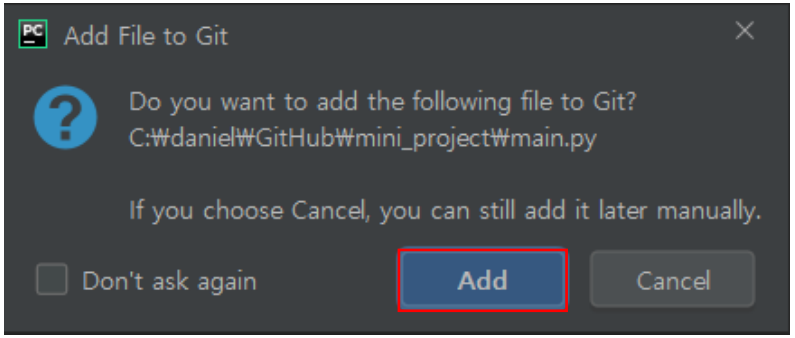
Bắt đầu với Mini-Project

Tạo file main.py (Daniel)

- Nhấp chuột phải mini_project → New → Click Python File → Nhập main là tên để khởi tạo main.py.



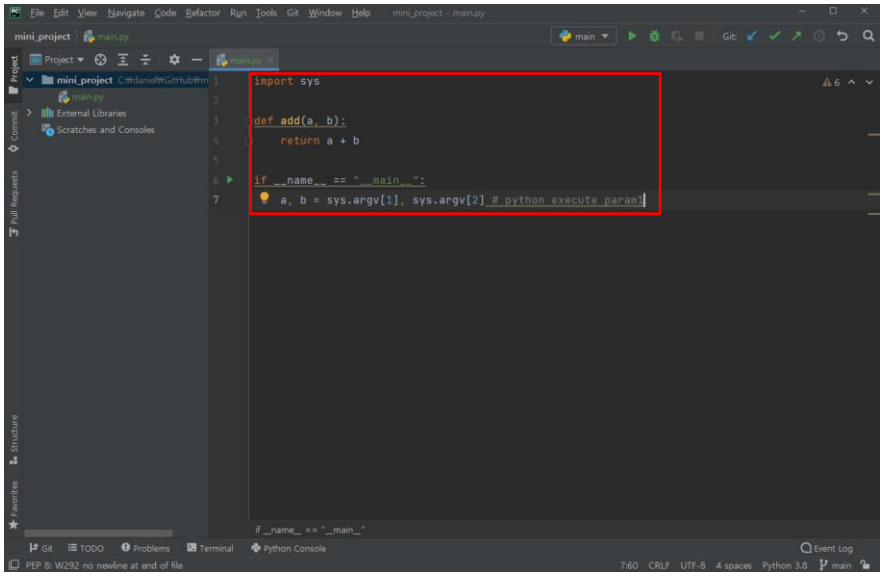
- Khi Daniel tạo file, chương trình sẽ hỏi liệu cậu ta có muốn Git theo dõi nó không. Nhấp vào **Add.**

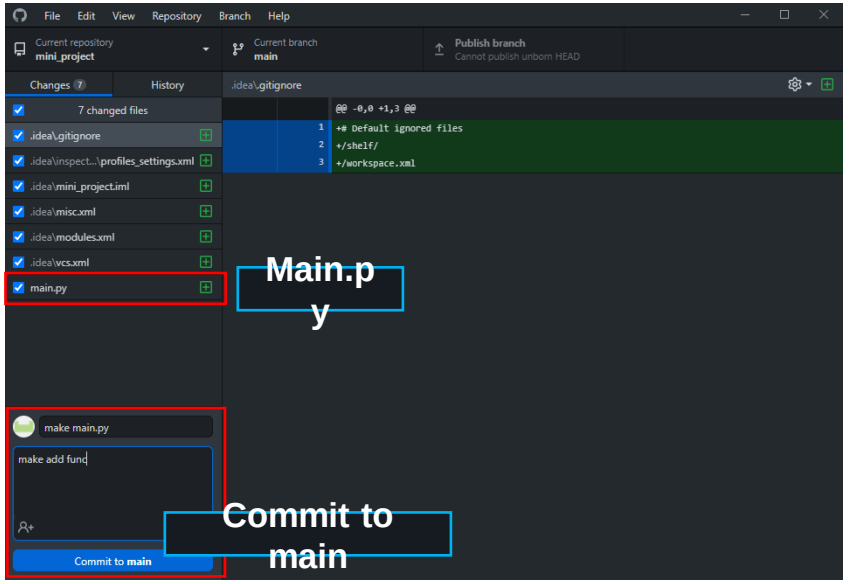


Bắt đầu với Mini-Project

Tạo file main.py (Daniel)

- Viết mã để tạo một chương trình tính toán các tham số thực thi Python làm đầu vào. sys.argv[1] và sys.argv[2] lần lượt là các tham số thực thi Python.
- Lưu main.py đã tạo và kiểm tra GitHub Desktop. Daniel có thể thấy rằng các tệp hỗ trợ quản lý git, bao gồm cả main.py, mới được thêm vào và theo dõi.
- Viết tin nhắn xác nhận và nhấp vào nút Commit to main.

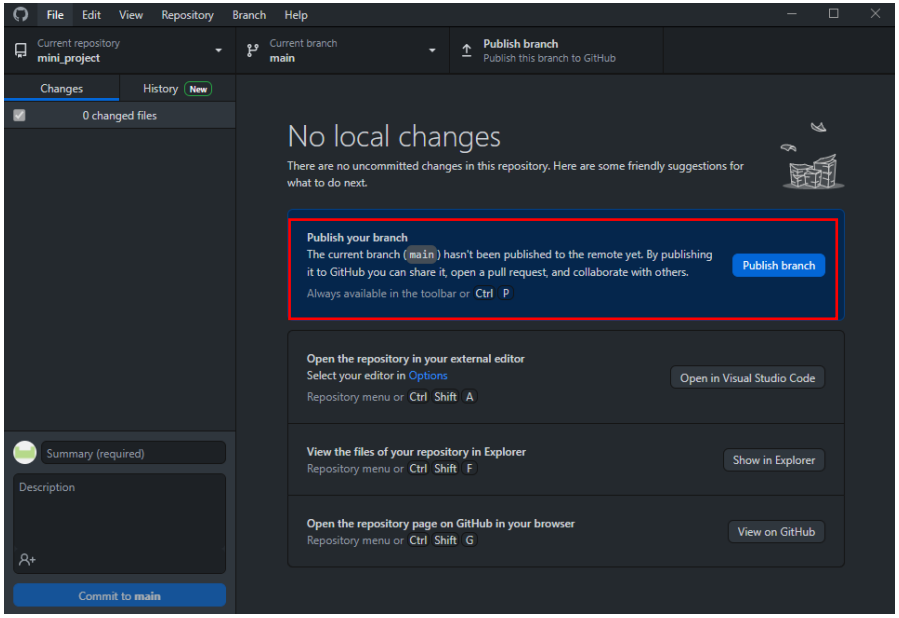




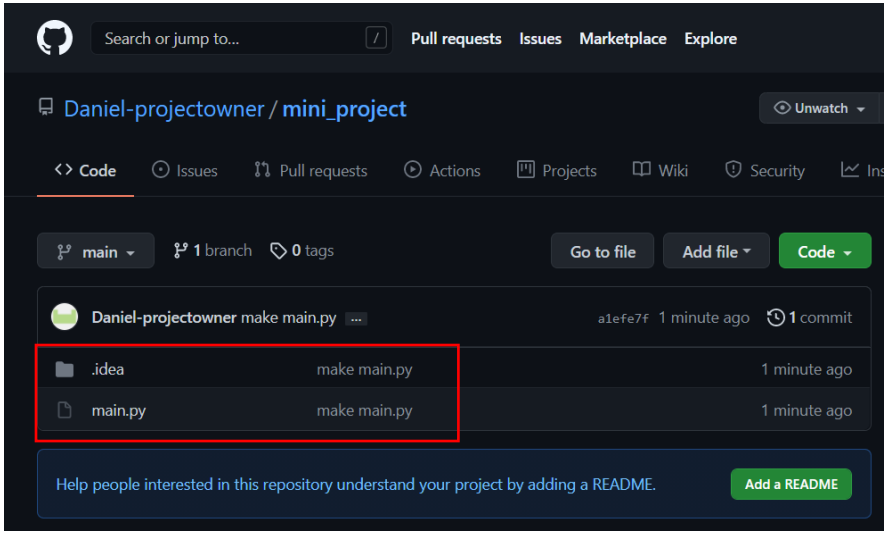
Bắt đầu với Mini-Project

Tạo file main.py (Daniel)

- Sau khi xác nhận, Daniel có thể thấy thông báo tạo nhánh. Chọn **Publish branch** để đưa nhánh đó vào Kho lưu trữ GitHub.



- Quay trở lại Kho lưu trữ GitHub. Daniel có thể kiểm tra xem file main.py có được tạo bằng thông báo xác nhận hay không, có nghĩa là mọi thứ hoạt động hoàn hảo.

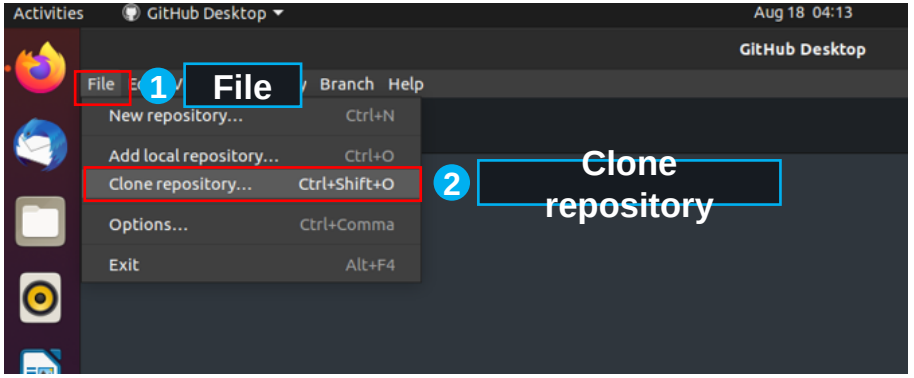


Quy trình thực hiện Mini-Project

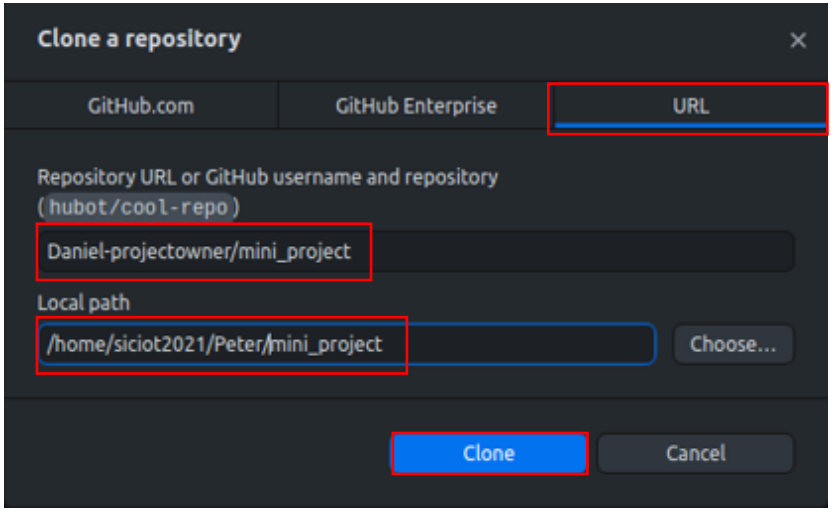
Sao chép Kho dữ liệu (Peter)

- Daniel ngừng phát triển main.py trong một thời gian để bắt đầu làm việc khác.
- Trong khi đó, Daniel hướng dẫn Peter để thêm và bổ sung một hàm trừ trong main.py để sử dụng thời gian của mình một cách hiệu quả.

- Chọn File trên GitHub Desktop để Peter sao chép Kho lưu trữ GitHub của Daniel → Clone repository



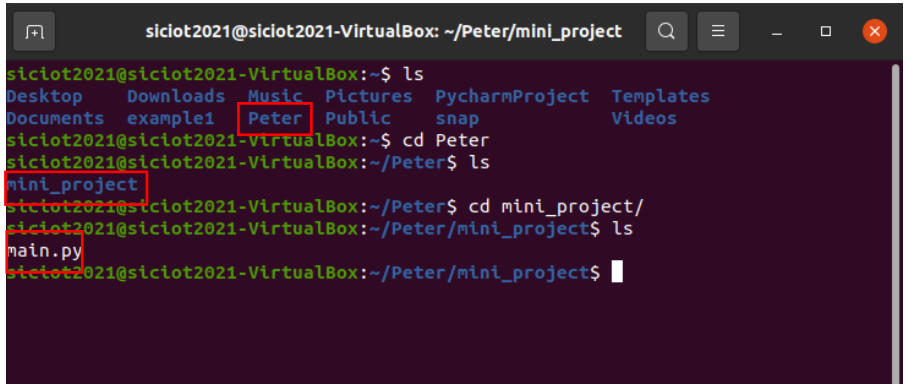
- Sử dụng URL để sao chép. Viết tên GitHub và tên Kho lưu trữ của Daniel. Đặt Local Path đến một vị trí mong muốn và nhấp vào nút **Clone**.



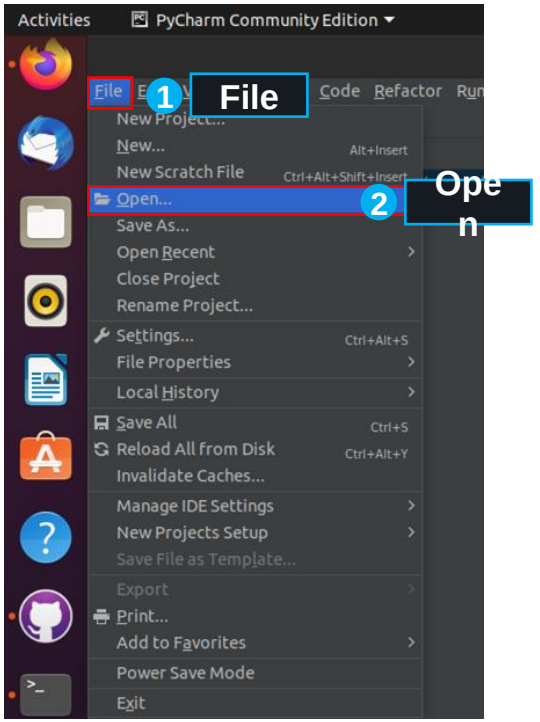
Bắt đầu với Mini-Project

Sao chép Kho dữ liệu (Peter)

- Duyệt lại Local Path để kiểm tra xem việc sao chép có thành công hay không. Peter có thể thấy main.py đang ở trong mini_project.



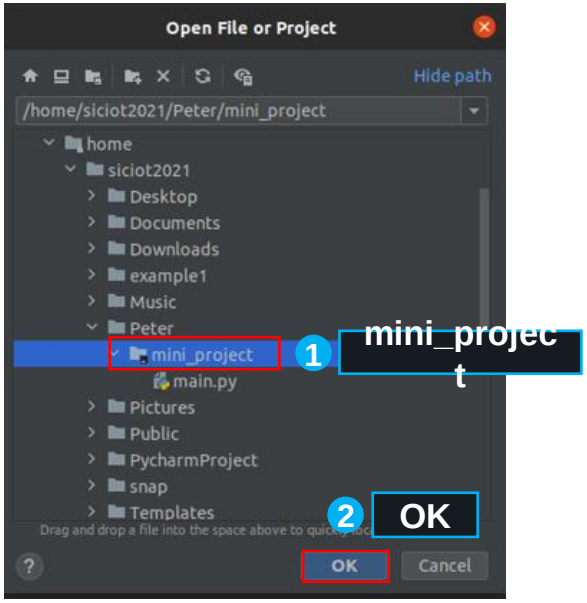
- Bắt đầu thao tác trên main.py bằng Pycharm.
- File → Open



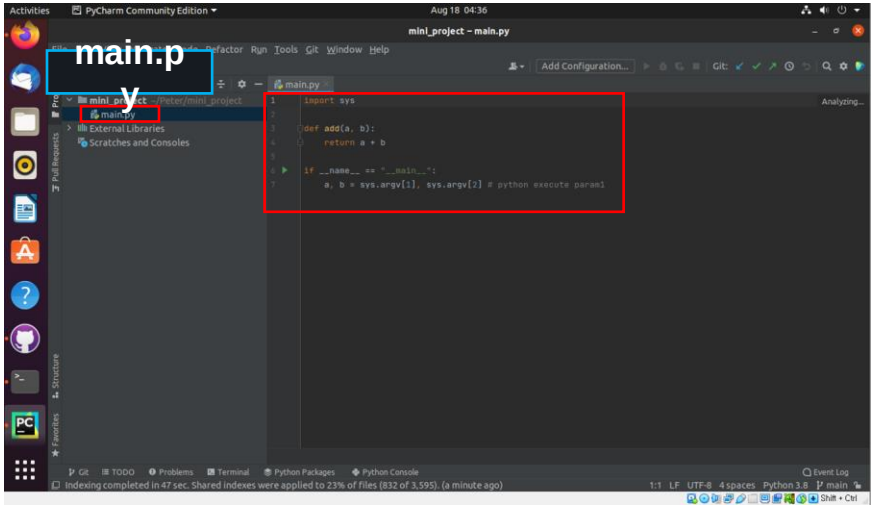
Bắt đầu với Mini-Project

Chỉnh sửa main.py(Peter)

- Sau khi tìm thấy vị trí tương ứng với Local Path, chọn thư mục mini_project → OK

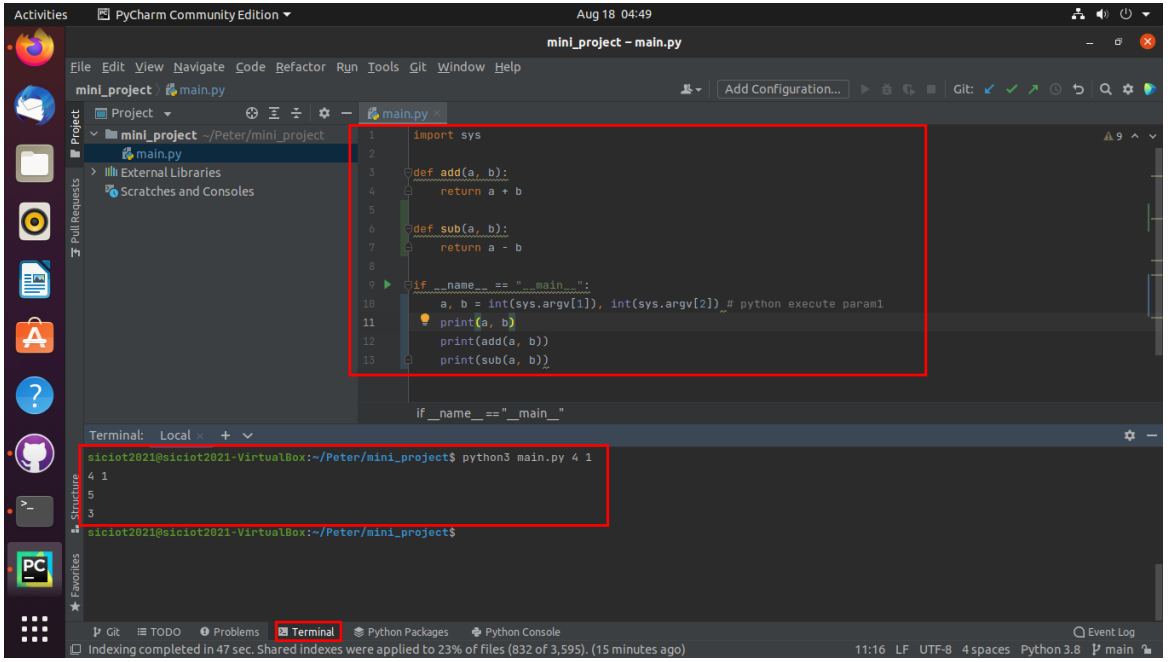


- Chọn mini_project và bấm đúp vào main.py để mở nó. Peter có thể kiểm tra đoạn mã Python do Daniel viết.



Bắt đầu với Mini-Project

Chỉnh sửa main.py(Peter)



- Do các tham số không phải là kiểu số nguyên, vì vậy Perter không thu được kết quả như mong đợi. Cậu ta đã thay đổi các tham số thành loại int, thêm hàm sub và thêm một câu lệnh xuất.
- Thực thi main.py với các tham số trong Terminal để kiểm tra. Lúc này, Peter xác nhận rằng kết quả đã đúng nhưmong đợi.

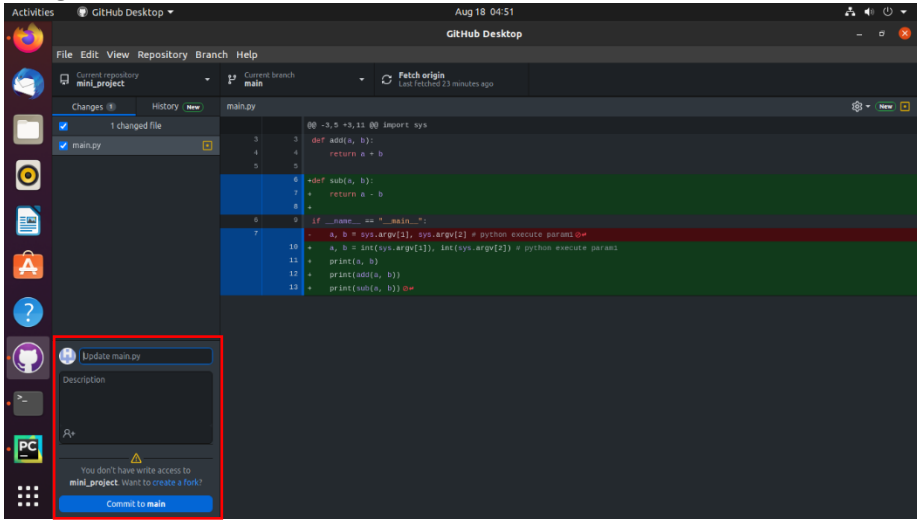
```
$ python3 main.py 4 1
```

- Lưu main.py và thoát.

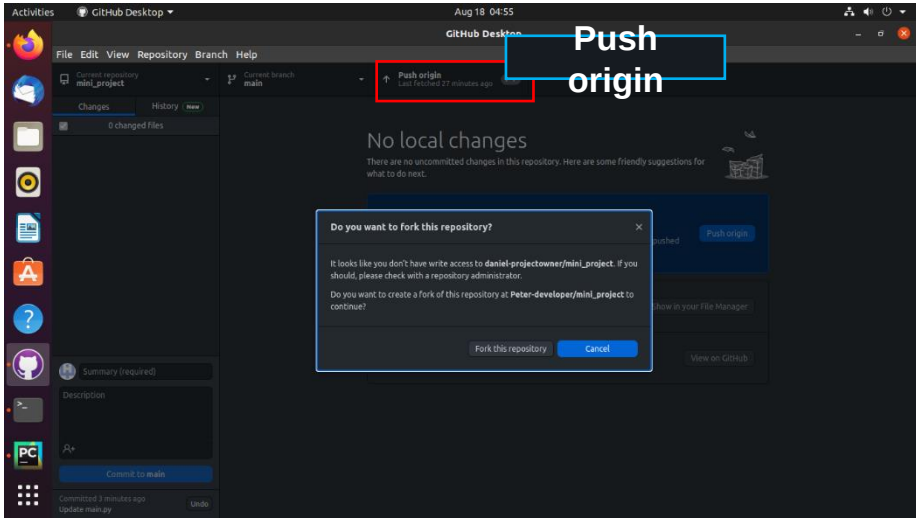
Bắt đầu với Mini-Project

Chỉnh sửa main.py(Peter)

- Quay lại GitHub Desktop. Khi Peter thử xác nhận thông qua một thông báo xác nhận, xuất hiện cảnh báo rằng cậu ta không có quyền chỉnh sửa. Lúc này cậu ta nhấp vào **Commit to main** để thử đẩy lên.



- Khi Peter cố gắng đẩy lên bằng cách nhấn nút push origin, một thông báo xuất hiện cho anh biết rằng không có quyền chỉnh sửa.



Bắt đầu với Mini-Project

Làm thế nào Peter có thể đẩy vào kho lưu trữ GitHub của Daniel?

- Vì kho lưu trữ GitHub mini_project thuộc sở hữu của Daniel, Peter không được phép đẩy vào kho lưu trữ này. Có hai cách chính để tránh tình trạng này.

Cách thứ nhất

- Cách đầu tiên là cấp cho Peter quyền thao tác trên Kho lưu trữ này. Khi Daniel chỉ định Peter làm Cộng tác viên, Peter có thể tự do đẩy vào Kho lưu trữ. Do kiểm soát khá lỏng lẻo, phương pháp này đòi hỏi sự quản lý chặt chẽ và thảo luận trong nhóm hơn.

Bắt đầu với Mini-Project

Làm thế nào Peter có thể đẩy vào kho lưu trữ GitHub của Daniel?

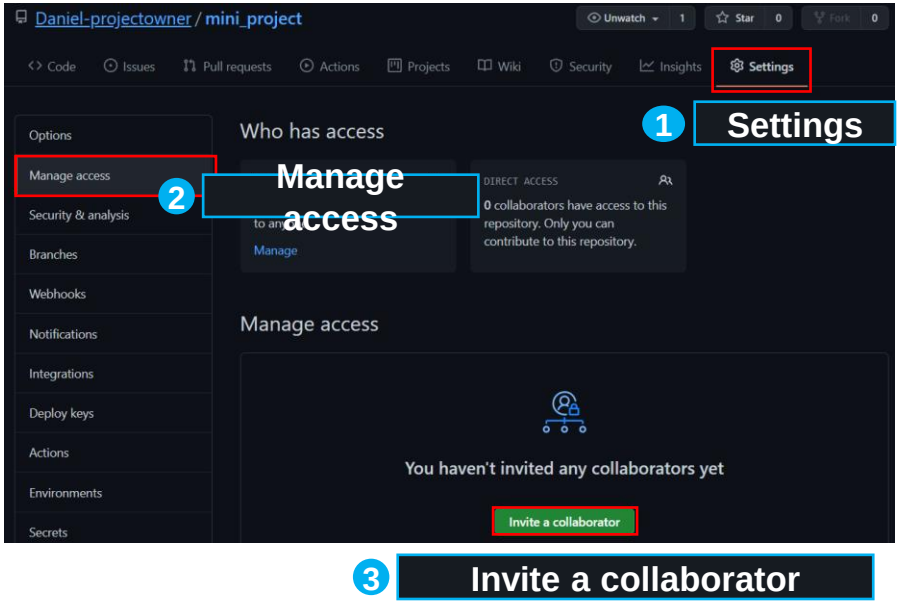
Cách thứ hai

- Cách thứ hai là Peter chia nhánh (fork) kho lưu trữ mini_project của Daniel.
  - Fork theo nghĩa đen có nghĩa là phân nhánh Kho lưu trữ của người khác và đưa nó vào Kho lưu trữ GitHub của riêng bạn.
  - Trong trường hợp này, Perter phân nhánh Kho lưu trữ của Daniel và đưa nó vào Kho lưu trữ GitHub của riêng mình, và nó khiến Kho lưu trữ GitHub của Peter tách biệt với Kho lưu trữ GitHub của Daniel.
  - Do đó, nếu Peter đẩy nội dung lên, chỉ có nội dung của Kho lưu trữ GitHub của Peter được ánh xạ, không phải Kho lưu trữ GitHub của Daniel.
- Làm thế nào Peter có thể ánh xạ tới Kho lưu trữ của Daniel với Kho lưu trữ mà anh ấy phân nhánh?
  - Có một phương thức được gọi là yêu cầu kéo (pull request). Đây là cách Kho lưu trữ GitHub của Daniel lưu các thay đổi. Kéo là một lệnh để lưu các thay đổi.
  - Sau khi Peter thực hiện Yêu cầu kéo tới Daniel và Daniel chấp thuận nó, Kho lưu trữ của Daniel ánh xạ những thay đổi có trong Yêu cầu kéo của Peter.
  - Phương pháp này thường được gọi là PR viết tắt từ Pull Request.
- Trong số hai phương pháp này, **chúng ta sẽ tìm hiểu phương pháp đầu tiên để mời Cộng tác viên**.

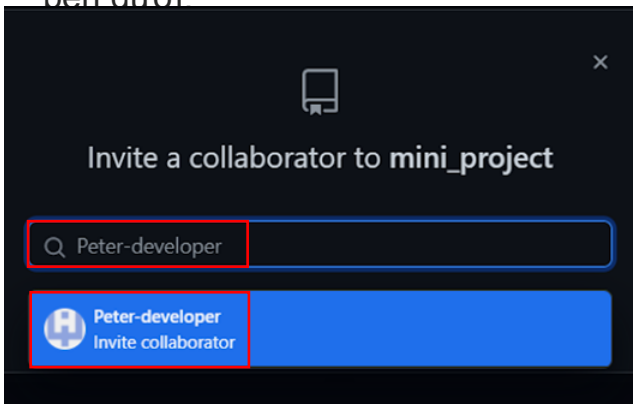
Quy trình thực hiện Mini-Project

Mời Peter (Daniel)

- Tới Settings trong Kho lưu trữ GitHub → Manage access → Invite a collaborator



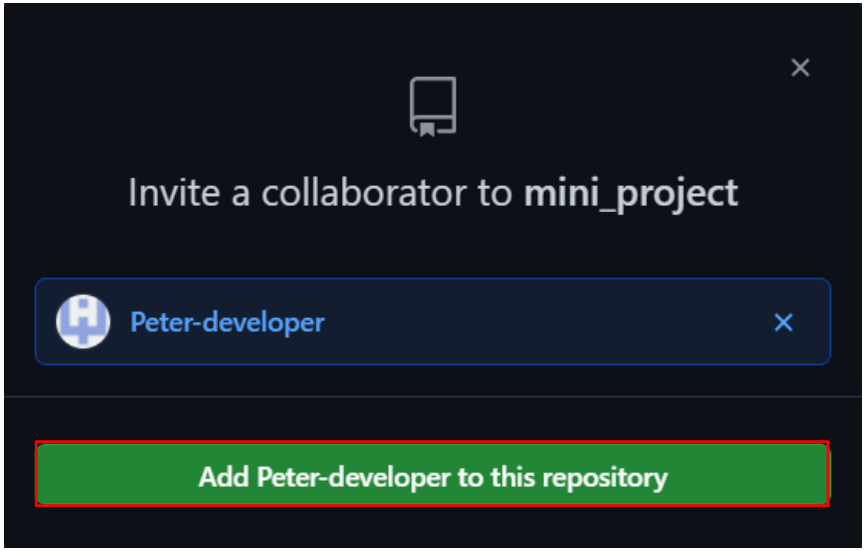
- Đây là nơi Daniel có thể mời một tài khoản GitHub khác. Sau khi nhập tên GitHub của Peter, hãy nhấp vào tên GitHub của Peter trong số các ứng cử viên bên dưới.



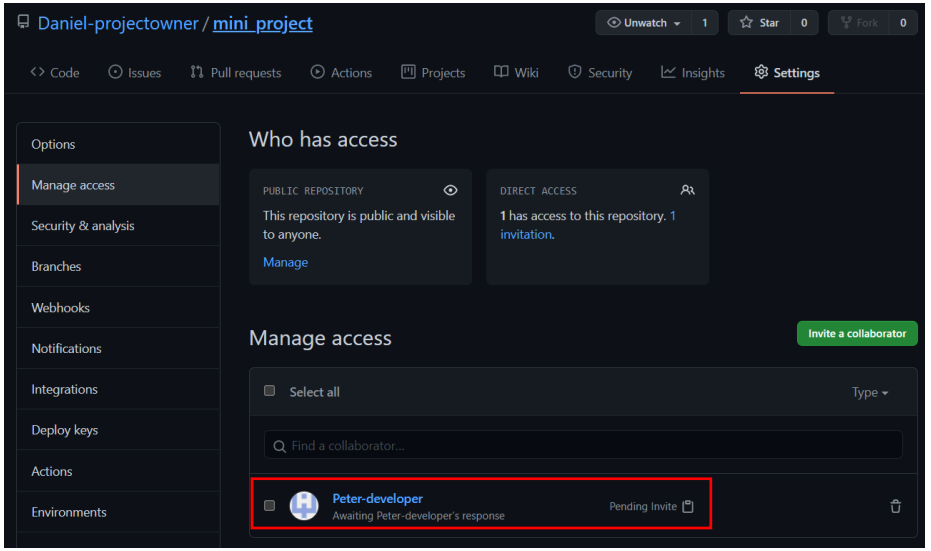
Quy trình thực hiện Mini-Project

Mời Peter (Daniel)

- Nhấp vào **Add Peter-developer to this repository** để mời Peter làm cộng tác viên.



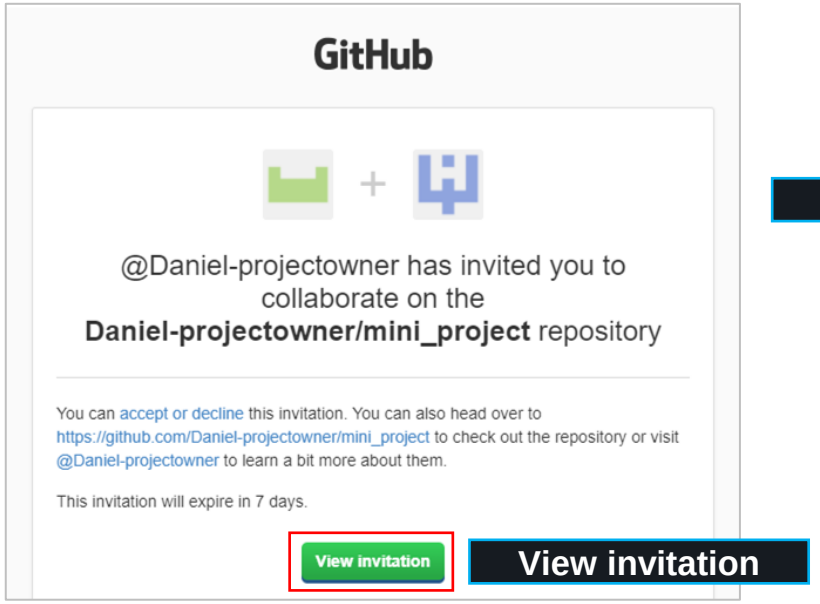
- Daniel có thể thấy rằng lời mời đã được gửi đến Peter. Và trạng thái đang chờ phản hồi của Peter. Peter cần kiểm tra email được gửi từ GitHub.

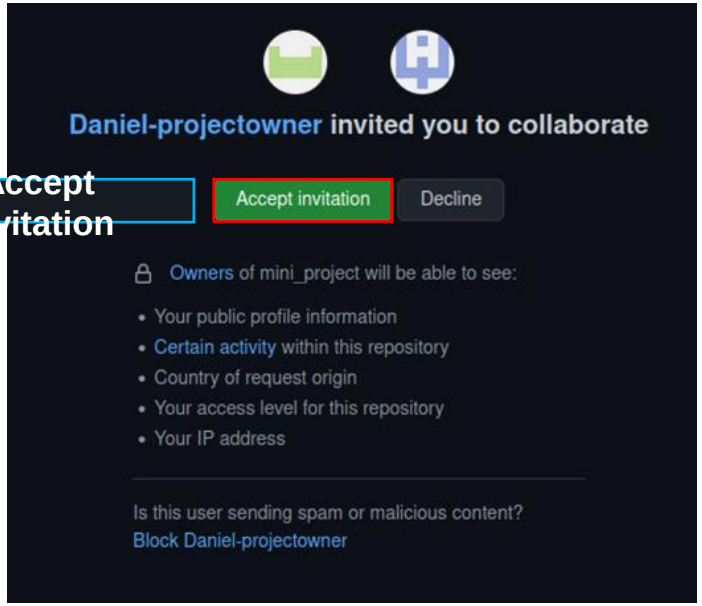


Quy trình thực hiện Mini-Project

Mời Peter (Daniel)

- Peter kiểm tra xem thư mời đã đến chưa. Nhấp vào biểu tượng **View invitation** để xác nhận.
- Bấm Accept invitation để chấp nhận lời mời và xin phép đẩy lên Kho lưu trữ của Daniel.

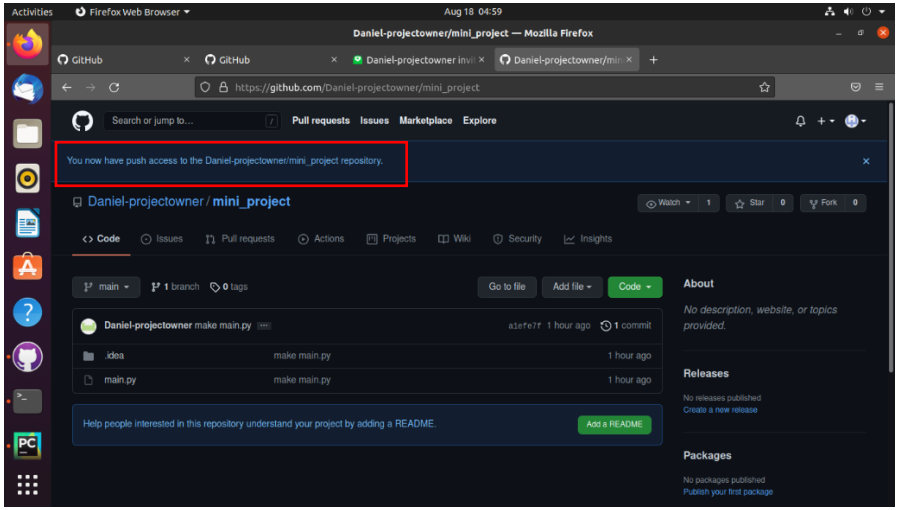




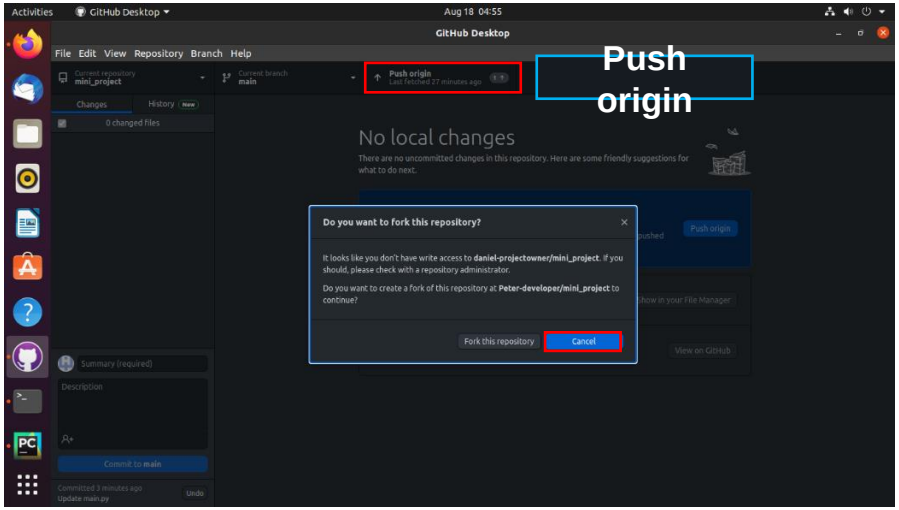
Quy trình thực hiện Mini-Project

Đẩy lên main.py đã chỉnh sửa (Peter)

- Kiểm tra thông báo push access đã được thực hiện.



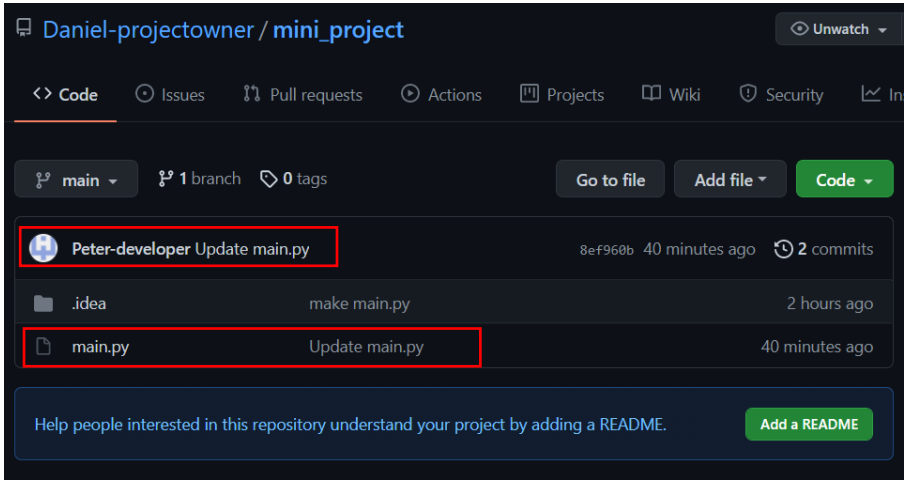
- Quay trở lại Github Desktop. Hủy cửa sổ bật lên liên quan đến đẩy không thành công và nhấp vào Push origin một lần nữa để tiến hành đẩy.

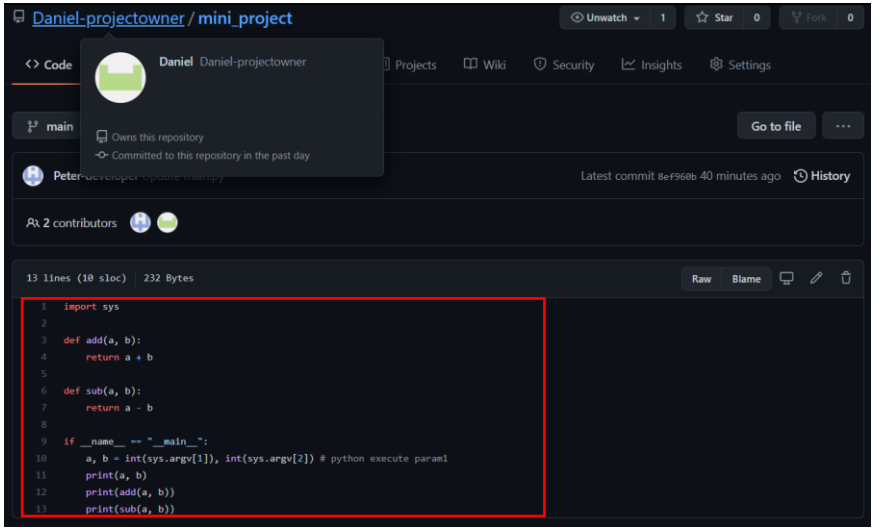


Quy trình thực hiện Mini-Project

Kiểm tra kho dữ liệu Project (Daniel & Peter)

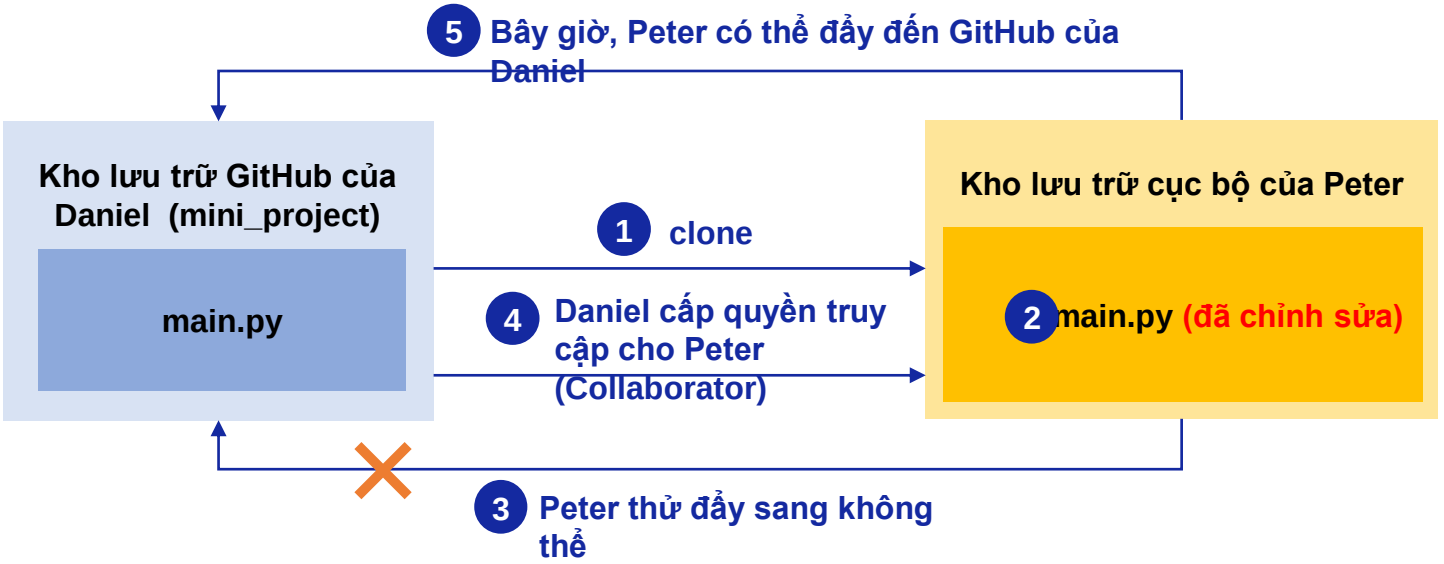
- Peter liên lạc với Daniel mà anh ấy đã thúc đẩy. Khi Daniel kiểm tra Kho lưu trữ GitHub của mình, được xác nhận rằng nó đã được ánh xạ cùng với thông điệp xác nhận của Peter. Bấm đúp vào main.py để kiểm tra main.py đã được chỉnh sửa.
- Cả Daniel và Peter đều có thể kiểm tra nội dung của main.py.





Lưu đồ thực hiện Mini-Project

Minh hoạ lưu đồ mô tả quá trình cộng tác




End of Document
Samsung Innovation Campus



ⓒ2022 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.